

# nox-mem: Pain-Weighted Hybrid Memory for LLM Agents

**Luiz Antonio Busnello** *Independent Researcher*

ORCID 0009-0007-5911-8141 · lab@generantis.com.br

**Version** v1.0.6 (2026-10-04) · DOI 10.5281/zenodo.23147633 (this version) ·  
all versions: 10.5281/zenodo.22649268 · changelog in [paper/CHANGELOG.md](#)

**Code and data:** [github.com/totobusnello/memoria-nox](https://github.com/totobusnello/memoria-nox) (MIT license)

---

## Abstract

We introduce nox-mem, a persistent memory system for autonomous LLM agents. Retrieval and retention are governed by an additive salience formula in which *pain* is a first-class signal. Pain is a severity in  $[0.1, 1.0]$ , persisted on every chunk, that an operator can assign and that a fixed keyword rule otherwise sets once at ingest (default 0.2, §3.4.2). Ranking changes pass a mandatory shadow phase before production activation. Each store is a single SQLite file (one per agent plus a shared workspace store), with provider-swappable embeddings, event-driven writeback (inotifywait, 2-second debounce), per-chunk\_type retention windows, chunk-level provenance, and a policy-gated pre-snapshot before destructive operations; the code is MIT-licensed. It has run in production since March 2026 (§2.4), serving six specialized agents at KG-path p50 = 2.5 ms, \$0 per KG-path query, and a 399 MB resident set in a single self-hosted process (§5.7). Our central result is a pre-specified (execution plan committed to the public repository before the first run; not registered with an external registry), same-corpus comparison against five competing memory systems, of which four produced head-to-head quality numbers (two in the 2026-06-15 canonical run, and EverOS and Zep in later runs over the same corpus and the full  $n = 2,482$  query set of the embedding-matched variant) and one was a documented deployment non-run (§6). Under each system’s native embedder, nox-mem and Mem0, the leaders of the 2026-06-15 run, split the datasets: Mem0 wins LoCoMo (nDCG@10 0.469 vs 0.426) and nox-mem wins LongMemEval. An embedding-matched variant (both Gemini 3072-d, full  $n = 2,482$ ) was a planned side experiment, not a post-hoc analysis (§6.7); it matches the embedder but does not cleanly isolate architecture (§6.3.2). This variant inverts the split: nox-mem leads on both datasets (LongMemEval 0.526 vs 0.403; LoCoMo 0.495 vs 0.441) and in all five represented categories, with four residual confounds declared. Neither leads the field: EverOS, run later over the same corpus and query set, outperforms nox-mem on both datasets (overall nDCG@10 0.646 vs 0.501), with a mandatory 4 B cross-encoder re-ranking stage that nox-mem’s pipeline lacks and whose share of the gap is unmeasured (§6.3.3); Zep ranks third, behind EverOS and nox-mem and ahead of Mem0 (§6.3.4). On EverMemBench, nox-mem reaches 63.28% Overall with Gemini-3-flash, 4.01 pp

above the 59.27% that Table 4 of the benchmark paper publishes for MemOS on the same backbone (not re-run by us), 4.50 pp in Table 4’s own aggregation (unweighted mean of nine sub-dimensions, where nox-mem scores 63.77%), and below that backbone’s 72.61% full-context baseline, so this is not a state-of-the-art claim (§5.1.10). Three findings cut against our own headline: *pain*’s isolated retrieval effect is directional but not statistically significant (§7.1); section-aware ranking, not pain, is the dominant empirical driver (§5.1.3); and on the same EverMemBench run the F\_MH multi-hop track sits at 6.02% (4–8% per batch), against 10.84% for MemOS on the same backbone in Table 4 (the best memory-augmented F\_MH in any Table 4 column is 18.88%, MemOS on GPT-4.1-mini; all LLM-judged); §5.4 offers task setup as the leading, not established, account.

---

## 1. Introduction

### 1.1 Problem Statement

Large Language Model (LLM) agents running in production are limited by an ephemeral context window. When a conversation ends or context is compacted, the agent loses accumulated knowledge, decisions, and operational state. In multi-agent systems where specialized agents collaborate on complex tasks, the loss compounds: agents cannot learn from each other’s experiences or recall past decisions, and they cannot build institutional knowledge over time.

### 1.2 Design Goals

nox-mem was designed with four objectives:

1. **Persistent Memory:** Survive context window resets, session boundaries, and agent restarts
2. **Intelligent Retrieval:** Return semantically relevant results in addition to keyword matches
3. **Cross-Agent Intelligence:** Enable knowledge sharing across isolated agent workspaces
4. **Operational Autonomy:** Self-maintain through automated consolidation, pruning, and indexing

### 1.3 Scope

The system runs inside an agent-hosting platform (OpenClaw) and serves 6 AI agents on a single VPS (a 4-vCPU / 16 GB host when the §5.7 measurements were taken; a 2-vCPU / ~8 GB host since 2026-08). Each agent has a distinct role and memory profile. The workspace (shared memory) and individual agent databases form a federated memory architecture.

## 1.4 Related Memory Systems and the Six Gaps

The published memory-for-LLM-agents literature spans roughly four families: (i) *vector-store wrappers with metadata layers*, such as mem0<sup>1</sup> and Letta<sup>2</sup>; (ii) *temporal- and provenance-aware memory services*, such as Zep<sup>3</sup>; (iii) *KG-augmented and graph-fused retrieval*, such as LightRAG<sup>4</sup> (HKU, Findings of EMNLP 2025) and HippoRAG2<sup>5</sup>; and (iv) *parametric-memory paradigms*, most recently MeMo<sup>6</sup>, which encodes new knowledge into the weights of a separately trained memory model rather than into an inspectable store, the opposite of our design. EverMind-AI/EverOS<sup>7</sup> occupies a distinct slot: its developers also publish a benchmark dataset of their own (EverMemBench<sup>8</sup>), which makes a same-benchmark comparison possible (§5.1.5–§5.1.10, §6). Four works outside these families bound the problem this design answers. *Lost in the Middle*<sup>9</sup> shows that simply extending the context window degrades mid-context recall, which is

<sup>1</sup>Chhikara, Khant, Aryan, Singh & Yadav, *Mem0: Building Production-Ready AI Agents with Scalable Long-Term Memory*, arXiv:2504.19413, 2025. Implementation: `mem0ai/mem0` — open-source memory layer for LLM agents (PostgreSQL + Qdrant backend, OpenAI embeddings by default), `github.com/mem0ai/mem0`. Used in §1.4, §5.3.2, §6.3, Table 2.

<sup>2</sup>Letta (formerly MemGPT) — agent-loop memory architecture with archival/recall memory separation. Original system paper: Packer et al., *MemGPT: Towards LLMs as Operating Systems*, arXiv:2310.08560. `github.com/letta-ai/letta`. Used in §1.4, §6.3, Table 2.

<sup>3</sup>Rasmussen, Paliychuk, Beauvais, Ryan & Chalef, *Zep: A Temporal Knowledge Graph Architecture for Agent Memory*, arXiv:2501.13956, 2025. Implementation: `github.com/getzep/zep`. Requires one paid LLM key to boot (OpenAI by default, Anthropic selectable); embeddings may run keyless against its own local embedder service. Verified in the v0.27.2 source — see [zep-stack]. Used in §1.4, §6.3, Table 2.

<sup>4</sup>Guo et al., *LightRAG: Simple and Fast Retrieval-Augmented Generation*, Findings of EMNLP 2025, pp. 10746–10761 (HKU); arXiv:2410.05779. `github.com/HKUDS/LightRAG` (~35k stars, MIT). Cited in §1.4 as a KG-augmented baseline. Also cited in §6.3.1 and in Table 2 (§6.8).

<sup>5</sup>Gutiérrez, Shu, Qi, Zhou & Su, *From RAG to Memory: Non-Parametric Continual Learning for Large Language Models* (HippoRAG 2), ICML 2025. arXiv:2502.14802. Graph-augmented retrieval with Personalized PageRank over an entity-relation graph; cited as a graph-baseline peer in §1.4, §1.5 and §6.

<sup>6</sup>Quek, Lee, Leong, Verma, Prakash, Chen, Low, Rus & Solar-Lezama, *MeMo: Memory as a Model*, 2026. arXiv:2605.15156v2. Encodes new knowledge into a dedicated, separately trained memory model while keeping the LLM’s own parameters unchanged. Cited in §1.4 and Table 1 as the parametric opposite of nox-mem’s externalized, inspectable memory.

<sup>7</sup>Hu, Gao, Zhou, Xu, Bai, Li, Zhang, Li, Zhang, Bing & Deng, *EverMemOS: A Self-Organizing Memory Operating System for Structured Long-Horizon Reasoning*, arXiv:2601.02163, 2026. Implementation: `github.com/EverMind-AI` (~5k stars, Apache 2.0). Publishes EverMemBench dataset and an EvoAgentBench-framed evolution loop. nox-mem’s EverMemBench numbers are in §5.1.5–§5.1.10; re-running the comparators in the same harness is F4 in §7.2. It is also the system paper for the system measured in §6.3.3. Its reported 93.05% LoCoMo / 83.00% LongMemEval are LLM-judged answer accuracy, a different quantity from the retrieval nDCG@10 of §6.3.3, and are not compared to it.

<sup>8</sup>Hu et al., *Evaluating Long-Horizon Memory for Multi-Party Collaborative Dialogues* (the EverMemBench paper), 2026. arXiv:2602.01313v3. Named in §1.5 as the multi-party long-horizon setting. §5.1.5–§5.1.10 report nox-mem numbers **on** this benchmark and §6.3.3 reports a number **of** the system whose authors also wrote it; both facts are disclosed rather than left implicit, and the disclosure applies symmetrically to our own use of it.

<sup>9</sup>Liu, Lin, Hewitt, Paranjape, Bevilacqua, Petroni & Liang, *Lost in the Middle: How Language Models Use Long Contexts*, TACL 2024. arXiv:2307.03172. Used in §1.4 as the bound on context-window scaling.

why the mechanism here is retrieval rather than a larger prompt. Reflexion<sup>10</sup> established verbal self-reflection as an agent-improvement loop, a loop nox-mem approximates only loosely: **reflect** synthesizes an answer over retrieved evidence and **crystallize** stores caller-supplied procedures (§3.4.4, §3.4.1). A-Mem<sup>11</sup> pursues agent-managed memory organisation, an axis orthogonal to our retrieval-side contribution. HaluMem<sup>12</sup> measures hallucination *in memory systems specifically*. We do not report that evaluation dimension, so we list it as a declared gap and claim no strength on it.

Across these systems, six recurring gaps appear in the design space, and each motivates a concrete subsystem below. The coverage table is compiled from each system’s own documentation rather than from measurement.

**Table 1 — Design-space coverage of the six gaps, as reported by each system’s own documentation.**

**Scope of the table.** Only the nox-mem column is measured here. Every other cell records what that system’s paper, README, or API reference *states* about itself, read between 2026-05 and 2026-06; we did not verify these claims by deployment. §6 deploys four of these six (Mem0, Zep, EverOS, Letta); three of them (Mem0, EverOS, Zep) produced same-corpus retrieval numbers, which test retrieval quality, not the properties tabulated here, and agentmemory, the fourth system with numbers in §6, is not in this table. A cell therefore answers “does the system claim this property?”, not “does the system have it?”. **n/r** means the source material does not address the gap either way; it records missing documentation and is not a negative finding. The table maps the design space that motivates §§2–4, and does not evaluate competing systems.

| # | Gap               | nox-mem<br>(measured)        | mem0    | Letta | Zep | EverOS           | LightRAG         | MeMo                       |
|---|-------------------|------------------------------|---------|-------|-----|------------------|------------------|----------------------------|
| 1 | Static injection  | yes — live writeback         | partial | yes   | yes | yes              | no — batch       | no — memory-model training |
| 2 | No temporal decay | yes — salience + retention   | no      | no    | yes | partial          | no               | no                         |
| 3 | No provenance     | yes — chunk_id + source_file | partial | yes   | yes | yes              | yes              | no — baked-in              |
| 4 | Flat memory       | yes — KG + section_boost     | no      | no    | yes | yes — hypergraph | yes — dual-level | no                         |

<sup>10</sup>Shinn, Cassano, Berman, Gopinath, Narasimhan & Yao, *Reflexion: Language Agents with Verbal Reinforcement Learning*, NeurIPS 2023 (vol. 36). arXiv:2303.11366. Used in §1.4.

<sup>11</sup>Xu et al., *A-Mem: Agentic Memory for LLM Agents*, 2025. arXiv:2502.12110. Used in §1.4.

<sup>12</sup>*HaluMem: Evaluating Hallucinations in Memory Systems of Agents*, 2025. arXiv:2511.03506. Used in §1.4 as a declared evaluation gap.

| # | Gap            | nox-mem<br>(measured)           | mem0    | Letta | Zep | EverOS         | LightRAG        | MeMo                       |
|---|----------------|---------------------------------|---------|-------|-----|----------------|-----------------|----------------------------|
| 5 | No write-back  | yes — consolidate + crystallize | partial | yes   | yes | yes — EvoAgent | no              | no                         |
| 6 | Indexing delay | yes — index wait, 2 s debounce  | n/r     | yes   | yes | n/r            | partial — batch | no — memory-model training |

Sources for the non-**nox-mem** columns: mem0 <sup>13</sup>, Letta <sup>14</sup>, Zep <sup>15</sup>, EverOS <sup>16</sup>, LightRAG <sup>17</sup>, MeMo <sup>18</sup>.

**How each gap is closed.** Live writeback addresses static injection (§3.1); typed `retention_days` folded into the salience formula addresses the absence of temporal decay (§3.4.3); `chunk_id` and `source_file` on every result, plus an append-only `ops_audit` table behind `withOpAudit()`, address provenance (§2.2; `withOpAudit()` is described in §7.1 L4); FTS5, typed retention, an extracted knowledge graph and section-aware `section_boost` address flat memory (§4.1); `consolidate` (extraction into re-ingested topic files) and `crystallize` (caller-supplied procedures) address the absence of writeback (§3.4); and event-driven re-ingestion after a 2-second debounce addresses indexing delay (§3.1).

<sup>13</sup>Chhikara, Khant, Aryan, Singh & Yadav, *Mem0: Building Production-Ready AI Agents with Scalable Long-Term Memory*, arXiv:2504.19413, 2025. Implementation: `mem0ai/mem0` — open-source memory layer for LLM agents (PostgreSQL + Qdrant backend, OpenAI embeddings by default), [github.com/mem0ai/mem0](https://github.com/mem0ai/mem0). Used in §1.4, §5.3.2, §6.3, Table 2.

<sup>14</sup>Letta (formerly MemGPT) — agent-loop memory architecture with archival/recall memory separation. Original system paper: Packer et al., *MemGPT: Towards LLMs as Operating Systems*, arXiv:2310.08560. [github.com/letta-ai/letta](https://github.com/letta-ai/letta). Used in §1.4, §6.3, Table 2.

<sup>15</sup>Rasmussen, Paliychuk, Beauvais, Ryan & Chalef, *Zep: A Temporal Knowledge Graph Architecture for Agent Memory*, arXiv:2501.13956, 2025. Implementation: [github.com/getzep/zep](https://github.com/getzep/zep). Requires one paid LLM key to boot (OpenAI by default, Anthropic selectable); embeddings may run keyless against its own local embedder service. Verified in the v0.27.2 source — see [^zep-stack]. Used in §1.4, §6.3, Table 2.

<sup>16</sup>Hu, Gao, Zhou, Xu, Bai, Li, Zhang, Li, Zhang, Bing & Deng, *EverMemOS: A Self-Organizing Memory Operating System for Structured Long-Horizon Reasoning*, arXiv:2601.02163, 2026. Implementation: [github.com/EverMind-AI](https://github.com/EverMind-AI) (~5k stars, Apache 2.0). Publishes EverMemBench dataset and an EvoAgentBench-framed evolution loop. nox-mem’s EverMemBench numbers are in §5.1.5–§5.1.10; re-running the comparators in the same harness is F4 in §7.2. It is also the system paper for the system measured in §6.3.3. Its reported 93.05% LoCoMo / 83.00% LongMemEval are LLM-judged answer accuracy, a different quantity from the retrieval nDCG@10 of §6.3.3, and are not compared to it.

<sup>17</sup>Guo et al., *LightRAG: Simple and Fast Retrieval-Augmented Generation*, Findings of EMNLP 2025, pp. 10746–10761 (HKU); arXiv:2410.05779. [github.com/HKUDS/LightRAG](https://github.com/HKUDS/LightRAG) (~35k stars, MIT). Cited in §1.4 as a KG-augmented baseline. Also cited in §6.3.1 and in Table 2 (§6.8).

<sup>18</sup>Quek, Lee, Leong, Verma, Prakash, Chen, Low, Rus & Solar-Lezama, *MeMo: Memory as a Model*, 2026. arXiv:2605.15156v2. Encodes new knowledge into a dedicated, separately trained memory model while keeping the LLM’s own parameters unchanged. Cited in §1.4 and Table 1 as the parametric opposite of nox-mem’s externalized, inspectable memory.

A single design principle ties these closures together, pain weighting under shadow discipline: every chunk carries an explicit **pain** severity, every ranking change rolls out first in `NOX_SALIENCE_MODE=shadow` with `/api/health` telemetry<sup>19</sup>, and only graduates to **active** after operator review.

---

## 1.5 Related Work

§1.4 compares nox-mem against deployable memory *products*. This section places it in the *literature*, and it marks what is not novel here as much as it locates the contribution.

**Where this sits on the map.** Two surveys chart the area this work belongs to. Zhang et al.<sup>20</sup> taxonomise agent memory by *what* is stored and *how* it is written, read and managed; Gao et al.<sup>21</sup> survey self-evolving agents along *what*, *when*, *how* and *where* to evolve. Read against either taxonomy, nox-mem is a narrow instance: a single-substrate store (SQLite, one embedding model) with a hand-specified write and retention policy, no learned component anywhere in the loop, and self-evolution limited to hand-written rules run on invocation or on a fixed nightly schedule (§3.4). Most of the axes those surveys enumerate are, for this system, fixed at their simplest setting. This is the relevant comparison: the results in §5–§6 come from a system that declines nearly every degree of freedom the literature has opened.

**Retrieval substrate: deliberately conventional.** Layer 1 is FTS5 BM25<sup>22</sup>, Layer 2 is dense retrieval over a single embedding model, and the two are combined by Reciprocal Rank Fusion<sup>23</sup> at the standard `k=60`. Each of those choices has a canonical source and a stronger alternative we did not adopt. Dense retrieval for open-domain QA was established by DPR<sup>24</sup> and extended to un-

---

<sup>19</sup>`NOX_SALIENCE_MODE` environment variable controls the three-state gate (`shadow` | `active` | `off`). Default is `shadow`. Telemetry exposed at `/api/health.salience`. See §3.4.3 and `staged/1.7a/edits/salience.ts`.

<sup>20</sup>Zhang, Bo, Ma, Li, Chen *et al.*, *A Survey on the Memory Mechanism of Large Language Model based Agents*, ACM TOIS 2025. arXiv:2404.13501. Used in §1.5.

<sup>21</sup>Gao, Geng, Hua, Hu, Juan *et al.*, *A Survey of Self-Evolving Agents: What, When, How, and Where to Evolve on the Path to Artificial Super Intelligence*, TMLR 2026. arXiv:2507.21046. Used in §1.5.

<sup>22</sup>Robertson & Zaragoza, *The Probabilistic Relevance Framework: BM25 and Beyond*, Foundations and Trends in Information Retrieval 3(4), 2009. doi:10.1561/15000000019. Cited in §1.5 and §2.2 for the BM25 ranking used by Layer 1 (FTS5, §4.1).

<sup>23</sup>Cormack, Clarke & Buettcher, *Reciprocal Rank Fusion Outperforms Condorcet and Individual Rank Learning Methods*, SIGIR 2009. doi:10.1145/1571941.1572114. Cited in §1.5 and §2.5; the source of the `k=60` constant used by the fusion layer (§4.1).

<sup>24</sup>Karpukhin, Oğuz, Min, Lewis, Wu, Edunov, Chen & Yih, *Dense Passage Retrieval for Open-Domain Question Answering*, EMNLP 2020. arXiv:2004.04906. The canonical source for the dense-retrieval half of Layer 2 (§1.5).

supervised training by Contriever<sup>25</sup>; late-interaction scoring as in ColBERT<sup>26</sup> is more expressive than the single-vector cosine we use; generative fusion over retrieved passages as in FiD<sup>27</sup> and the original RAG formulation<sup>28</sup> moves work into the reader, which nox-mem does not do at all: it returns ranked chunks and leaves generation to the calling agent. On the index side we run exact search via sqlite-vec<sup>29</sup> rather than an approximate structure such as HNSW<sup>30</sup>; §7.2 F6 expects, without having measured it, exact-search latency to start to matter near ~100k vectors (the main store held 67,724 on 2026-09-09), and it is a scaling limitation, not a design claim. One assumption underneath all of it is untested here: nox-mem retrieves on every query. Mallen et al.<sup>31</sup> show that for sufficiently popular facts a model’s parametric knowledge beats retrieval, and that retrieving anyway can make the answer worse. Always-retrieve is therefore a policy choice this paper never evaluates against the alternative. The retriever is not the contribution of this paper. Nothing in Layers 1–2 would surprise an IR reader, and that is intentional: it isolates what does change, which is the retention and ranking policy above them.

**Memory scoring: the closest prior art, and the delta.** Generative Agents<sup>32</sup> scores a memory stream by recency, importance and relevance, and retrieves the top-scoring items into a limited context, which is structurally the same shape as the additive salience formula of §3.4. We do not claim the shape as novel. Two things differ. First, nox-mem adds *pain*, an operator-assignable severity in  $[0.1, 1.0]$  *persisted on the chunk*. It is set explicitly or, by default, by a fixed keyword rule at ingest (§3.4.2), and is not inferred by the model; a hand-written rule, not the LLM, decides what hurt. Second, a ranking layer

<sup>25</sup>Izacard, Caron, Hosseini, Riedel, Bojanowski, Joulin & Grave, *Unsupervised Dense Information Retrieval with Contrastive Learning*, 2021. arXiv:2112.09118. Cited in §1.5 for unsupervised dense-retriever training.

<sup>26</sup>Khattab & Zaharia, *ColBERT: Efficient and Effective Passage Search via Contextualized Late Interaction over BERT*, SIGIR 2020. arXiv:2004.12832. Cited in §1.5 as the more expressive alternative to the single-vector scoring used here.

<sup>27</sup>Izacard & Grave, *Leveraging Passage Retrieval with Generative Models for Open Domain Question Answering*, EACL 2021. arXiv:2007.01282. Cited in §1.5 as generative fusion in the reader, which nox-mem does not perform.

<sup>28</sup>Lewis, Perez, Piktus, Petroni, Karpukhin, Goyal, Küttler, Lewis, Yih, Rocktäschel, Riedel & Kiela, *Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks*, NeurIPS 2020. arXiv:2005.11401. Engaged in §1.5 as the formulation nox-mem does **not** adopt: it returns ranked chunks and leaves generation to the caller.

<sup>29</sup>Garcia, *sqlite-vec: A Vector Search Extension for SQLite*, 2024. [github.com/asg017/sqlite-vec](https://github.com/asg017/sqlite-vec). Cited in §1.5 and §2.2 as the vector-table implementation; a software reference, so the repository is the identifier.

<sup>30</sup>Malkov & Yashunin, *Efficient and Robust Approximate Nearest Neighbor Search Using Hierarchical Navigable Small World Graphs*, 2016. arXiv:1603.09320. Cited in §1.5 as the approximate index nox-mem does **not** use — the exact-search scale limit of §7.2 F6 is a direct consequence.

<sup>31</sup>Mallen, Asai, Zhong, Das, Khashabi & Hajishirzi, *When Not to Trust Language Models: Investigating Effectiveness of Parametric and Non-Parametric Memories*, ACL 2023. arXiv:2212.10511. Used in §1.5.

<sup>32</sup>Park, O’Brien, Cai, Morris, Liang & Bernstein, *Generative Agents: Interactive Simulacra of Human Behavior*, UIST 2023. arXiv:2304.03442. The closest prior art to the salience formula of §3.4 (a weighted sum of recency, importance and relevance over a memory stream, all weights 1); §1.5 states the delta explicitly.

consumes the score in place of a paging loop, and changes to that layer must pass a shadow phase before activation (§3.4.3). The policy is therefore auditable and reversible in a way an in-prompt scoring heuristic is not.

**Learning the memory policy instead of fixing it.** A recent line of work makes the memory policy itself the object of training. Mem- $\alpha$ <sup>33</sup> learns *construction* (what to write and how to structure it) by reinforcement learning; Memory-R1<sup>34</sup> learns the management operations (add, update, delete, retain) with outcome rewards; Memory as Action<sup>35</sup> treats context curation as an action in the agent’s own action space; MEM1<sup>36</sup> trains memory and reasoning jointly so that the retained state stays constant in size across turns. Each of them replaces a hand-written rule with a learned one, and reports gains from doing so.

nox-mem takes the opposite position, and we state the trade here without defending it. The salience weights of §3.4 are constants chosen by hand; **pain** is set by the operator or, by default, once at ingest by a fixed keyword rule (§3.4.2); retention windows are integers per type. The rules never adapt, only the values they read. In exchange, an operator gets a policy that can be read in one screen, diffed in git, and reverted, plus a change process (the shadow gate of §3.4.3) that can hold a proposed ranking change against production traffic before it takes effect. The cost is what those four papers demonstrate: a fixed policy cannot discover, per corpus, what a trained one finds. This paper does not measure that gap, and the gap is real.

**Self-evolution: the same two moves, without a training loop.** ReasoningBank<sup>37</sup> distills reusable reasoning strategies out of an agent’s own successes and failures and retrieves them on later tasks; structurally, that is what **crystallize** (§3.4.1) stores when a caller records a solved incident as a procedure. Reflective Memory Management<sup>38</sup> splits the problem into *prospective* reflection (deciding at write time what will matter) and *retrospective* reflection (re-ranking retrieved evidence after the fact), which maps only loosely onto the write-side **pain** assignment and the **reflect** synthesis (§3.4.4) respectively. In both cases the mechanism in this paper is the cheaper half: **crystallize** records a procedure only when a caller invokes it, rather than on the agent’s own judgment of its trajectories (ReasoningBank) or an RL-refined policy (RMM), and **reflect** is a prompted LLM synthesis over hybrid-search results, run on request, rather

<sup>33</sup>Wang, Takanobu, Liang, Mao, Hu *et al.*, *Mem- $\alpha$ : Learning Memory Construction via Reinforcement Learning*, 2025. arXiv:2509.25911. Used in §1.5.

<sup>34</sup>Yan, Yang, Huang, Nie, Ding *et al.*, *Memory-R1: Enhancing Large Language Model Agents to Manage and Utilize Memories via Reinforcement Learning*, 2025. arXiv:2508.19828. Used in §1.5.

<sup>35</sup>Zhang, Shu, Ma, Lin, Wu & Sang, *Memory as Action: Autonomous Context Curation for Long-Horizon Agentic Tasks*, 2025. arXiv:2510.12635. Used in §1.5.

<sup>36</sup>Zhou, Qu, Wu, Kim, Prakash *et al.*, *MEM1: Learning to Synergize Memory and Reasoning for Efficient Long-Horizon Agents*, 2025. arXiv:2506.15841. Used in §1.5.

<sup>37</sup>Ouyang, Yan, Hsu, Chen, Jiang *et al.*, *ReasoningBank: Scaling Agent Self-Evolving with Reasoning Memory*, ICLR 2026. arXiv:2509.25140. Used in §1.5.

<sup>38</sup>Tan, Yan, Hsu, Han, Wang *et al.*, *In Prospect and Retrospect: Reflective Memory Management for Long-term Personalized Dialogue Agents*, ACL 2025. arXiv:2503.08026. Used in §1.5.

than a learned re-ranking step. WebCoach<sup>39</sup> closes the loop across *sessions*, feeding distilled advice from past episodes into a fresh agent that has no memory of them. That is the same cross-session role that §3.5’s brief plays, except that WebCoach’s advice is written by an LLM coach that decides when to inject it, while the brief is a ranked assembly with no generation step. The shapes are prior art; what is new here is the accounting of what they cost at \$0 of training.

**Forgetting: also prior art, with a different granularity.** MemoryBank<sup>40</sup> implements decay on an Ebbinghaus-style curve that forgets or reinforces each memory by the time elapsed and the memory’s relative significance (abstract, arXiv:2305.10250v3), so its forgetting is set per item. nox-mem instead uses typed retention windows per `chunk_type` (§3.4.3<sup>41</sup>), with NULL meaning never-decay for the `feedback` and `person` types. The trade is expressiveness for inspectability: a per-type integer in a column is coarser than a per-item curve, and it is also legible in `sqlite3` and changeable without re-deriving anything.

**Structure of the store.** Rezazadeh et al.<sup>42</sup> grow a *dynamic* tree of schemas over a conversation, deepening it as material accumulates. nox-mem’s entity files (§5.1.3) are the degenerate case of that idea: a fixed three-section shape (frontmatter, compiled truth, timeline) with a per-section retrieval boost and no restructuring at all. On the multi-agent side, MIRIX<sup>43</sup> coordinates several specialised memory components behind one interface; the architecture of §2.4 instead gives each of six agents its own database and reads across them with a single fan-out query (`cross_search`), which keeps per-agent provenance at the cost of any shared consolidation between them.

**Agent-memory architectures.** MemGPT<sup>44</sup> frames memory as an operating-system problem, paging between a fixed context window and archival storage. nox-mem does not page: the context window is populated by a brief (§3.5) assembled from a ranked query, and there is no eviction loop. The distinction matters for failure modes. A paging architecture can lose an item by evicting it, while a ranking architecture loses it by ranking it low, which is recoverable by changing the ranker; controlling such changes is the job of §3.4.3’s shadow gate.

<sup>39</sup>Liu, Geng, Li, Cui, Zhang *et al.*, *WebCoach: Self-Evolving Web Agents with Cross-Session Memory Guidance*, 2025. arXiv:2511.12997. Used in §1.5.

<sup>40</sup>Zhong, Guo, Gao, Ye & Wang, *MemoryBank: Enhancing Large Language Models with Long-Term Memory*, 2023. arXiv:2305.10250. The closest prior art to typed retention (§3.4.3), via Ebbinghaus-curve decay; §1.5 states the granularity difference.

<sup>41</sup>V8 schema typed retention defaults (in `chunks.retention_days`): `feedback` = 0 (never-decay), `person` = 0 (never-decay), `lesson` = 180d, `decision` = 365d, `project` = 365d, `team` = 120d, `daily` = 90d, `pending` = 30d, `graph_node` = 60d, `fallback` = 90d. See `staged/1.7a/edits/salience.ts:46-56` (DEFAULT\_RETENTION\_BY\_TYPE) and `CLAUDE.md` §“Schema v10”.

<sup>42</sup>Rezazadeh, Li, Wei & Bao, *From Isolated Conversations to Hierarchical Schemas: Dynamic Tree Memory Representation for LLMs*, 2024. arXiv:2410.14052. Used in §1.5.

<sup>43</sup>Wang & Chen, *MIRIX: Multi-Agent Memory System for LLM-Based Agents*, 2025. arXiv:2507.07957. Used in §1.5.

<sup>44</sup>Letta (formerly MemGPT) — agent-loop memory architecture with archival/recall memory separation. Original system paper: Packer et al., *MemGPT: Towards LLMs as Operating Systems*, arXiv:2310.08560. `github.com/letta-ai/letta`. Used in §1.4, §6.3, Table 2.

MemAgent<sup>45</sup> attacks the same context limit from a third direction, training an agent to rewrite a fixed-size memory across successive chunks of a long input; that is a *reading* strategy for one long document, where nox-mem’s brief is an *assembly* strategy over a persistent multi-source store. A fourth direction compresses instead of selecting: ACON<sup>46</sup> optimises *how* long-horizon context is condensed, trading fidelity for room. The brief does neither: it ranks and truncates at K, keeping every retained chunk verbatim. That makes a brief auditable (every line in it exists in the store, unaltered), and it is also why a brief cannot fit more evidence into the same token budget than selection allows. Whether compression would beat selection at equal budget is untested here.

**Graph-augmented retrieval.** HippoRAG<sup>47</sup> applies Personalized PageRank over an entity-relation graph, and HippoRAG 2<sup>48</sup> extends the approach toward non-parametric continual learning; LightRAG<sup>49</sup> merges an incremental KG using LLM-generated summaries. nox-mem’s KG path (§4.1) is far weaker by design: a SQL + regex entity walk over `kg_relations`, with no PPR and no LLM call, which is why it costs \$0 and runs at single-digit milliseconds (§5.7). It buys latency and cost at the price of multi-hop expressiveness; that price is not isolated here, and §5.4 offers task setup, not the KG path, as the leading account of the F\_MH gap.

**Benchmarks and evaluation.** The long-term conversational memory setting is measured here on LoCoMo<sup>50</sup> and LongMemEval<sup>51</sup>, both of which follow the multi-session conversation setting introduced by Beyond Goldfish Memory<sup>52</sup>; classical multi-hop QA on MuSiQue<sup>53</sup> against the IRCOT<sup>54</sup> baseline. Retrieval-

<sup>45</sup>Yu, Chen, Feng, Chen, Dai *et al.*, *MemAgent: Reshaping Long-Context LLM with Multi-Conv RL-based Memory Agent*, ICLR 2026. arXiv:2507.02259. Used in §1.5.

<sup>46</sup>Kang, Chen, Han, Inan, Wutschitz *et al.*, *ACON: Optimizing Context Compression for Long-horizon LLM Agents*, 2025. arXiv:2510.00615. Used in §1.5.

<sup>47</sup>Gutiérrez, Shu, Gu, Yasunaga & Su, *HippoRAG: Neurobiologically Inspired Long-Term Memory for Large Language Models*, NeurIPS 2024. arXiv:2405.14831. Cited in §1.5 as the PPR-over-entity-graph approach the KG path deliberately does not implement.

<sup>48</sup>Gutiérrez, Shu, Qi, Zhou & Su, *From RAG to Memory: Non-Parametric Continual Learning for Large Language Models* (HippoRAG 2), ICML 2025. arXiv:2502.14802. Graph-augmented retrieval with Personalized PageRank over an entity-relation graph; cited as a graph-baseline peer in §1.4, §1.5 and §6.

<sup>49</sup>Guo *et al.*, *LightRAG: Simple and Fast Retrieval-Augmented Generation*, Findings of EMNLP 2025, pp. 10746–10761 (HKU); arXiv:2410.05779. github.com/HKUDS/LightRAG (~35k stars, MIT). Cited in §1.4 as a KG-augmented baseline. Also cited in §6.3.1 and in Table 2 (§6.8).

<sup>50</sup>Maharana, Lee, Tulyakov, Bansal, Barbieri & Fang, *Evaluating Very Long-Term Conversational Memory of LLM Agents*, ACL 2024. aclanthology.org/2024.acl-long.747. The LoCoMo benchmark used in §5.3 and §6.

<sup>51</sup>Wu, Wang, Yu, Zhang, Chang & Yu, *LongMemEval: Benchmarking Chat Assistants on Long-Term Interactive Memory*, ICLR 2025. arXiv:2410.10813. The cross-bench validation set of §5.6.

<sup>52</sup>Xu, Szlam & Weston, *Beyond Goldfish Memory: Long-Term Open-Domain Conversation*, ACL 2022. arXiv:2107.07567. Used in §1.5.

<sup>53</sup>Trivedi, Balasubramanian, Khot & Sabharwal, *MuSiQue: Multihop Questions via Single-hop Question Composition*, TACL 2022. arXiv:2108.00573. The multi-hop QA dataset used in §5.2.

<sup>54</sup>Trivedi, Balasubramanian, Khot & Sabharwal, *Interleaving Retrieval with Chain-of-Thought Reasoning for Knowledge-Intensive Multi-Step Questions*, ACL 2023. arXiv:2212.10509. The IRCOT baseline compared against in §5.2.

quality methodology follows the zero-shot heterogeneous-benchmark discipline of BEIR<sup>55</sup> and the embedding-model evaluation conventions of MTEB<sup>56</sup>, which is why §6 reports nDCG@10 under each system’s native embedder *and* an embedding-matched variant: a single-embedder comparison conflates retriever quality with embedder quality. Multi-party long-horizon collaborative memory is the setting of EverMemBench, introduced by Hu et al.<sup>57</sup>; §5.1.5–§5.1.10 report nox-mem on its five released batches, compared against the MemOS numbers that paper publishes rather than against a re-run (§5.8.5).

**Agent reasoning loops.** The orchestration experiments of §5.5 are implementations of published loops, not new ones: IterB follows ReAct<sup>58</sup> and IterC follows the Self-Ask<sup>59</sup> decomposition. Their contribution in this paper is the measurement of where each loop’s ceiling sits on top of this retriever, not the loops themselves. The newer alternative is to train the loop rather than script it: MemSearcher<sup>60</sup> learns jointly to reason, issue searches and prune its own context end-to-end. Untrained scripted loops are what §5.5 measures, and the ceilings it reports should be read as ceilings *of that class*.

## 2. System Architecture

### 2.1 Infrastructure Overview

The system runs on a single VPS (a Hostinger KVM4 at initial deployment) reachable over a Tailscale VPN at a private Tailscale address. Five systemd-managed services make up the runtime:

| Service          | Port  | Type        | Function                    |
|------------------|-------|-------------|-----------------------------|
| openclaw-gateway | 18789 | WebSocket   | Agent communication gateway |
| nox-mem-watcher  | —     | inotifywait | Filesystem event monitor    |

<sup>55</sup>Thakur, Reimers, Rücklé, Srivastava & Gurevych, *BEIR: A Heterogeneous Benchmark for Zero-shot Evaluation of Information Retrieval Models*, NeurIPS 2021 (Datasets & Benchmarks). arXiv:2104.08663. The zero-shot evaluation discipline §6 follows.

<sup>56</sup>Muennighoff, Tazi, Magne & Reimers, *MTEB: Massive Text Embedding Benchmark*, EACL 2023. arXiv:2210.07316. The embedding-evaluation convention behind §6’s native-embedder plus embedding-matched design.

<sup>57</sup>Hu et al., *Evaluating Long-Horizon Memory for Multi-Party Collaborative Dialogues* (the EverMemBench paper), 2026. arXiv:2602.01313v3. Named in §1.5 as the multi-party long-horizon setting. §5.1.5–§5.1.10 report nox-mem numbers **on** this benchmark and §6.3.3 reports a number **of** the system whose authors also wrote it; both facts are disclosed rather than left implicit, and the disclosure applies symmetrically to our own use of it.

<sup>58</sup>Yao, Zhao, Yu, Du, Shafraan, Narasimhan & Cao, *ReAct: Synergizing Reasoning and Acting in Language Models*, ICLR 2023. arXiv:2210.03629. The loop implemented as IterB in §5.5.

<sup>59</sup>Press, Zhang, Min, Schmidt, Smith & Lewis, *Measuring and Narrowing the Compositionality Gap in Language Models*, 2022. arXiv:2210.03350. The decomposition implemented as IterC in §5.5.

<sup>60</sup>Yuan, Lou, Li, Chen, Lu *et al.*, *MemSearcher: Training LLMs to Reason, Search and Manage Memory via End-to-End Reinforcement Learning*, ACL 2026. arXiv:2511.02805. Used in §1.5.

| Service     | Port                                                       | Type      | Function                                                                                                                                                      |
|-------------|------------------------------------------------------------|-----------|---------------------------------------------------------------------------------------------------------------------------------------------------------------|
| nox-mem-api | 18802<br>(NOX_API_PORT;<br>18800 at initial<br>deployment) | HTTP/JSON | Search, health, brief and<br>dashboard API                                                                                                                    |
| ollama      | 11434                                                      | HTTP      | Local LLM inference<br>(llama3.2:3b), initial<br>deployment only; since<br>removed, and consolidation<br>now uses Gemini 2.5<br>Flash-Lite with Groq fallback |
| tailscaled  | —                                                          | WireGuard | VPN mesh connectivity                                                                                                                                         |

## 2.2 Database Schema

Storage is SQLite 3 in WAL (Write-Ahead Logging) mode for concurrent access. The listing below is the initial-deployment schema (version 3). The current schema is v18, which adds to `chunks` the columns used by §3.4, §4 and §5 (among them `retention_days` (v8), `pain` (v9), `section/section_boost` (v10), `importance`, `access_count`, `last_accessed_at`, `tier` and `source_type`). Version 3 contains:

### Core Tables:

- `chunks` — Memory fragments with full-text indexing
  - `id` (INTEGER PK), `source_file` (TEXT), `chunk_text` (TEXT), `chunk_type` (TEXT)
  - `source_date` (TEXT), `is_consolidated` (INTEGER), `memory_type` (TEXT)
  - `created_at`, `updated_at` (TEXT, ISO 8601)
  - `metadata` (TEXT, JSON)
- `chunks_fts` — FTS5 virtual table with porter unicode61 tokenizer (replaced by `unicode61 remove_diacritics 2` in schema v5, §4.1)
  - Content-sync triggers (INSERT, UPDATE, DELETE) maintain index consistency
  - BM25 ranking<sup>61</sup> with configurable column weights (1.0, 0.5, 0.5)
- `consolidated_files` — Processing state tracker
  - `source_file` (TEXT PK), `status` (INTEGER: 0=pending, 1=done, -1=failed)
- `meta` — Key-value configuration store (`schema_version`, `cursors`, `metrics`)

### Knowledge Graph Tables:

- `kg_entities` — Named entities with type classification and mention counting
  - UNIQUE constraint on (name, entity\_type)

<sup>61</sup>Robertson & Zaragoza, *The Probabilistic Relevance Framework: BM25 and Beyond*, Foundations and Trends in Information Retrieval 3(4), 2009. doi:10.1561/15000000019. Cited in §1.5 and §2.2 for the BM25 ranking used by Layer 1 (FTS5, §4.1).

- TTL tracking via `first_seen`, `last_seen`
- `kg_relations` — Typed relationships between entities
  - Confidence scoring (0.0-1.0) with temporal decay
  - TTL via `expires_at` (90-day default), `last_confirmed`
  - Evidence linking via `evidence_chunk_id`
- `decision_versions` — Architectural decision version history
  - Supersession chain via `is_current` flag and `superseded_at` timestamp

#### Vector Tables (sqlite-vec<sup>62</sup>):

- `vec_chunks` — Virtual table storing float32 embeddings (3072 dimensions)
- `vec_chunk_map` — Rowid-to-`chunk_id` mapping (sqlite-vec requires rowid-based access)
- `dedup_log` — Suppressed duplicate tracking for audit

### 2.3 Chunk Type Taxonomy

Each chunk is assigned one of 10 types from the path of its source file:

| Type     | Source Pattern                   | Count (workspace store, early snapshot; not current) | Purpose                                 |
|----------|----------------------------------|------------------------------------------------------|-----------------------------------------|
| team     | <code>shared/</code>             | 499                                                  | Shared team knowledge, cross-agent docs |
| daily    | <code>memory/YYYY-MM-DD</code>   | 161                                                  | Daily operational notes                 |
| other    | (default)                        | 126                                                  | Unclassified content                    |
| decision | <code>memory/decisions.md</code> | 34                                                   | Architectural and strategic decisions   |
| lesson   | <code>memory/lessons.md</code>   | 21                                                   | Errors, corrections, lessons learned    |
| project  | <code>memory/projects.md</code>  | 11                                                   | Active project tracking                 |
| pending  | <code>memory/pending.md</code>   | 8                                                    | Incomplete tasks and blockers           |
| feedback | <code>memory/feedback/</code>    | 6                                                    | User and system feedback                |
| person   | <code>memory/people.md</code>    | 6                                                    | People profiles and contacts            |
| digest   | <code>memory/digests/</code>     | 2                                                    | Weekly summary reports                  |

<sup>62</sup>Garcia, *sqlite-vec: A Vector Search Extension for SQLite*, 2024. [github.com/asg017/sqlite-vec](https://github.com/asg017/sqlite-vec). Cited in §1.5 and §2.2 as the vector-table implementation; a software reference, so the repository is the identifier.

## 2.4 Multi-Agent Memory Architecture

Each of the 6 agents has its own database at `agents/{name}/tools/nox-mem/nox-mem.db`, relative to the workspace root. Every module resolves paths through the `OPENCLAW_WORKSPACE` environment variable, so the same nox-mem binary opens a different database depending on the calling context.

### Agent Memory Distribution (as of March 23, 2026):

| Agent            | Role                   | Chunks     | DB Size        | Dominant Type     |
|------------------|------------------------|------------|----------------|-------------------|
| Nox              | Chief of Staff         | 185        | 268 KB         | daily (91)        |
| Boris            | Head of Communications | 148        | 268 KB         | team (50)         |
| Forge            | Code Reviewer          | 182        | 292 KB         | daily (135)       |
| Atlas            | Research               | 30         | 128 KB         | other (17)        |
| Cipher           | Security               | 31         | 132 KB         | other (12)        |
| Lex              | Legal/Compliance       | 31         | 132 KB         | other (12)        |
| <b>Workspace</b> | <b>Shared</b>          | <b>874</b> | <b>25.2 MB</b> | <b>team (499)</b> |

Total system memory: 1,481 chunks across 7 databases (initial-deployment snapshot, March 2026; the main store measured 67,724 chunks on 2026-09-09).

## 2.5 User-Facing Primitives

nox-mem exposes a deliberately small public contract of three primitives. All three are backed by the same SQLite store and exposed through the CLI and the HTTP API; `search` and the temporal filter are also exposed through MCP (the MCP tool for `answer` is pending). The advanced verbs (`reflect`, `cross-search`, `kg-path`, `crystallize`) are separate operations over the same store, not compositions of these primitives.

**Primitive 1: `search` (hybrid retrieval).** FTS5 BM25 and Gemini semantic retrieval (3072d) are fused by Reciprocal Rank Fusion (RRF)<sup>63</sup>,  $k=60$ , with optional section and source-type boost weights summed as deltas (§4.1). The source-type boost is skipped for chunks that already carry a section boost, except for queries that name more entities than a configured threshold. Returns ranked chunks with `score`, `match_type`, and provenance fields (`source_file`, `section`, `created_at`, `updated_at`). Fusion is detailed in §4; the boosts are evaluated in §5.1.

**Primitive 2: `answer` (grounded RAG).** Internally calls `search` with `topK = 8` by default (`NOX_ANSWER_TOPK`, 1–20), builds a citation-anchored prompt over the retrieved chunks, invokes the configured LLM (`gemini-2.5-flash-lite` by default), and parses inline `[chunk_<id>]` citations. As an anti-hallucination guard, a citation that points to a chunk outside the retrieved set triggers a single retry with a stricter prompt; a second failure

<sup>63</sup>Cormack, Clarke & Buettcher, *Reciprocal Rank Fusion Outperforms Condorcet and Individual Rank Learning Methods*, SIGIR 2009. doi:10.1145/1571941.1572114. Cited in §1.5 and §2.5; the source of the  $k=60$  constant used by the fusion layer (§4.1).

raises `AnswerError('hallucination_after_retry')`. When no chunks match, the call returns before invoking the LLM, so empty retrieval costs no LLM spend. On an offline bench with a mock LLM fixed at 100 ms, the pipeline overhead outside the LLM call is ~1.7 ms at p95 (101.74 ms total; `staged/P1/edits/benchmark/answer-latency.ts`); end-to-end latency is dominated by the LLM round-trip, which we did not benchmark. Implementation: `staged/P1/edits/src/lib/answer/{index,retrieval,prompt,provider,config}.ts`.

**Primitive 3: temporal filter (`--as-of` / `--changed-since`).** These are time-travel and recency-window selectors, implemented as hard SQL pre-filters on the candidate set rather than as ranking boosts. `--as-of <date>` restricts to chunks satisfying `created_at <= date AND (deleted_at IS NULL OR deleted_at > date)`; `--changed-since <date>` restricts to chunks satisfying `updated_at > date OR created_at > date`. When both are given, the two clauses are combined with AND. Accepted formats: ISO 8601 (2026-05-01 or full 2026-05-01T00:00:00Z) and relative (7d, 1w, 30d, 2h, 15m). The filter uses the existing `chunks.created_at` and `chunks.updated_at` columns from schema v18 and changes neither the schema nor the ranking. The filter is orthogonal to ranking-time recency signals (the 7-day recency boost of §4.1 and the recency component of salience, §3.4.3), which reweight ranking rather than restricting the candidate set. Implementation: `staged/P3/edits/{dates,search,api-server}.ts`.

**Composition.** The three primitives compose orthogonally. For example, `answer "what incidents happened last week?" --changed-since 7d` retrieves only chunks updated in the last seven days and then synthesizes a grounded answer over that restricted candidate set. Temporal filtering is deterministic for a fixed store and resolved cutoff timestamp. Search also depends on time-dependent ranking signals and, when enabled, LLM query expansion; `answer` additionally invokes an answer-generation LLM. All three share the CLI and HTTP surfaces.

## 3. Memory Pipeline

### 3.1 Ingestion

A file created or modified in a monitored directory triggers the `inotifywait`-based watcher service<sup>64</sup>, which applies:

- **Debounce logic:** 2-second delay to batch rapid successive writes
- **File filtering:** Only `.md` and `.txt` files are processed (`.json/.jsonl`, `database`, `log` and dot-files are skipped)
- **Recursion prevention:** `MEMORY.md` and `SESSION-STATE.md` are excluded to avoid feedback loops
- **Heartbeat:** Touches `/tmp/nox-mem-watcher-heartbeat` on every event, for liveness monitoring

<sup>64</sup>`nox-mem-watcher` systemd service running `inotifywait` on `memory/` directories. See §3.1.

Upon trigger, `ingestFile()` executes:

1. Read file content with UTF-8 sanitization (fixes common mojibake patterns for Portuguese text)
2. Detect chunk type from relative file path
3. Extract date from filename pattern (YYYY-MM-DD)
4. Split content into semantic chunks:
  - Markdown: Split on H2/H3 headers, with sub-splitting for chunks exceeding 500 words
  - JSON: Array items become individual chunks; object entries become key-value pairs
  - Small chunks (<20 words) are merged with the previous chunk
5. Delete existing chunks for the same source file (idempotent re-ingestion)
6. Insert new chunks via prepared statement transaction
7. Auto-vectorize if GEMINI\_API\_KEY is available (up to 20 chunks per file)

### 3.2 Consolidation

Nightly consolidation (23:00, 5-minute stagger across agents) processes daily notes into structured topic files:

1. **Reindex:** Scan all `.md/.json` files in memory directories, rebuild chunk index
2. **Extract:** Use Gemini 2.5 Flash-Lite (Groq Llama-3.3-70B as fallback; Olama llama3.2:3b at initial deployment) to identify facts, decisions, lessons, and action items from daily notes
3. **Append:** Add extracted content to topic files (decisions.md, lessons.md, people.md, projects.md, pending.md)

Deployment-specific housekeeping steps that follow do not write to the store and are omitted here.

### 3.3 Deduplication

Items produced by consolidation (§3.2) and session distillation are checked for duplicates before they are appended, using a two-tier strategy:

- **Primary:** Gemini cosine similarity with 0.85 threshold (when embeddings are available)
- **Fallback:** Word-overlap ratio above 70%
- **Audit:** Suppressed duplicates are logged to `dedup_log` table with reason and preview

### 3.4 Write-Side Maintenance: Rule-Based Mechanisms

Some memory systems change their write side by training a parametric memory model (MeMo) or through a closed evolution loop (EverOS EvoAgentBench

<sup>65</sup>). nox-mem has no learned component here: its write side is hand-written mechanisms (§3.4.1–§3.4.5), each inspectable in the SQLite file. They address Gap #5 (no writeback) in Table 1.

**3.4.1 Crystallize — procedure capture** The `crystallize` command implements procedure capture. It is exposed through the CLI, the Model Context Protocol (MCP) server tool, and `POST /api/crystallize` + `POST /api/crystallize/validate` on the HTTP API (see the HTTP API server and the consolidation module <sup>66</sup>). A caller (CLI, MCP tool, or `POST /api/crystallize`) supplies a title and an ordered list of steps, which are stored as a single `chunk_type = procedure` chunk (embedded best-effort so it is hybrid-searchable). `POST /api/crystallize/validate` appends success/failure outcomes to the procedure’s metadata. No LLM is involved, and chunks are not promoted between types; the pending→lesson promotion with pain accumulation described in earlier drafts is not implemented.

**3.4.2 Pain — assigned once at ingest** The `pain` field on `chunks` is an explicit severity in  $[0.1, 1.0]$  (0.1 trivial, 1.0 production outage). The author can set it explicitly; otherwise `inferPain()` assigns it once at ingest as a per-`chunk_type` base (0.2 when the type has no entry), raised by 0.5 when the chunk text matches an incident/outage keyword pattern. No mechanism currently adjusts `pain` after ingest in response to retrieval or feedback, so `pain` measures whether a chunk *talks about* a failure, not an observed severity.

**3.4.3 Saliency decay computed on the read path — recency × pain × importance** The saliency function is in `staged/1.7a/edits/saliency.ts` <sup>67</sup>. The principle behind it is multiplicative: a memory scores high only when it is recent, painful, and important at the same time:

`saliency = recency × pain × importance`

with components:

---

<sup>65</sup>Hu, Gao, Zhou, Xu, Bai, Li, Zhang, Li, Zhang, Bing & Deng, *EverMemOS: A Self-Organizing Memory Operating System for Structured Long-Horizon Reasoning*, arXiv:2601.02163, 2026. Implementation: [github.com/EverMind-AI](https://github.com/EverMind-AI) (~5k stars, Apache 2.0). Publishes EverMemBench dataset and an EvoAgentBench-framed evolution loop. nox-mem’s EverMemBench numbers are in §5.1.5–§5.1.10; re-running the comparators in the same harness is F4 in §7.2. It is also the system paper for the system measured in §6.3.3. Its reported 93.05% LoCoMo / 83.00% LongMemEval are LLM-judged answer accuracy, a different quantity from the retrieval nDCG@10 of §6.3.3, and are not compared to it.

<sup>66</sup>HTTP handlers for `/api/crystallize` and `/api/crystallize/validate` in `staged/1.6/edits/api-server.ts`:253–273; core logic exports `crystallize()`, `validateProcedure()` and `listProcedures()`.

<sup>67</sup>`staged/1.7a/edits/saliency.ts` — versioned implementation of the saliency formula (multiplicative principle in §3.4.3; weighted-additive v2 production form `W_IMPORTANCE·importance + W_RECENCY·recency + W_PAIN·pain + W_ACCESS·access_score` in §5.1); lines 1–27 contain the module docstring (multiplicative principle and the three-state mode gate), lines 45–56 the per-type retention defaults, and lines 229–232 the additive v2 weights).

- **recency** in  $[0,1]$  — half-life-style decay over the chunk’s `retention_days` window (`feedback/person` → never-decay → `recency = 1.0`; everything else decays per the per-type defaults listed in <sup>68</sup>). Decay is *continuous*: it is recomputed at every retrieval from the age of `last_accessed_at` (falling back to `source_date`), so no batch step “ages” memory, and each access resets a chunk’s decay clock. Aging is a property of the read path.
- **pain** in  $[0,1]$  — severity as described in §3.4.2.
- **importance** in  $[0,1]$  — `chunk_type` / `source_type` / tier signal (manual mapping; e.g., `decision` and `lesson` rank above `daily`).

Since Wave A (§5.1), production implements this principle in a weighted-additive form, `salience = W_IMPORTANCE·importance + W_RECENCY·recency + W_PAIN·pain + W_ACCESS·access_score` (weights 0.55 / 0.15 / 0.10 / 0.20). The additive form was adopted because the multiplicative product compressed 99.7% of chunks into the  $[0.05, 0.40]$  range (§5.1.2). §5.1 reports active-versus-shadow for the additive form, not a head-to-head against the multiplicative product under an otherwise identical stack. The multiplicative expression above states the principle; the additive form is what ships.

`NOX_SALIENCE_MODE` selects the mode: `shadow` (default; compute and log to `/api/health.salience` without applying to retrieval rankings), `active` (add `salience - 0.5`, bounded to  $[-0.5, +0.5]$ , to each retrieval layer’s boost sum before RRF fusion, §4.1), and `off` (short-circuit to 0 for ablation experiments). This three-state gate is how *shadow discipline* (§1.4) is implemented; §5.1.2 measures active against shadow.

#### 3.4.4 Reflect — on-demand synthesis over retrieved evidence

The `reflect` command (CLI / MCP / POST `/api/reflect`, backed by the `reflect` module and cached via `getReflectCacheStats()` exposed in `/api/health.reflectCache` <sup>69</sup>) answers a question in two steps: it queries hybrid search (falling back to FTS), then asks Gemini to synthesize over the top results, and it returns the answer with its source files. Responses are cached in a `reflect_cache` table with a per-entry TTL and exact- or semantic-key lookup (statistics exposed via `/api/health.reflectCache`), so the cost of a repeated question is amortized. Reflect does not write its output back into `chunks`.

#### 3.4.5 Consolidate — nightly extraction into topic files

Nightly consolidation (23:00, 5-minute stagger across agents; see §3.2) is the extraction step of

<sup>68</sup>V8 schema typed retention defaults (in `chunks.retention_days`): `feedback` = 0 (never-decay), `person` = 0 (never-decay), `lesson` = 180d, `decision` = 365d, `project` = 365d, `team` = 120d, `daily` = 90d, `pending` = 30d, `graph_node` = 60d, `fallback` = 90d. See `staged/1.7a/edits/salience.ts:46-56` (`DEFAULT_RETENTION_BY_TYPE`) and `CLAUDE.md` §“Schema v10”.

<sup>69</sup>The `reflect` module exports `reflect()` and `getReflectCacheStats()` (wired in `staged/1.6/edits/api-server.ts:12`). Cache statistics are surfaced at `/api/health.reflectCache`.

§3.2: it sends recent daily notes to Gemini 2.5 Flash-Lite (Groq Llama-3.3-70B as fallback) to extract decisions, lessons, people, projects and pending items, and appends them to the topic files (`decisions.md`, `lessons.md`, `people.md`, `projects.md`, `pending.md`), which the watcher then re-ingests. It does not invoke `reflect` or `crystallize` and is not currently wrapped in `withOpAudit()`.

These mechanisms keep the write side inspectable without retraining a model: `consolidation` extracts durable items from daily notes into topic files that the watcher re-ingests, `crystallize` records procedures a caller supplies, `pain` is fixed at ingest and enters salience, and `decay` is computed on the read path. None of them promotes chunks between types or raises `pain` after ingest. Each step can be checked by opening `nox-mem.db` in `sqlite3` and inspecting `chunks.pain`, `chunks.retention_days`, the topic files and, for the operations wrapped in `withOpAudit()`, `ops_audit`.

### 3.5 Session Priming: the read side

The mechanisms above act on the write side: they decide what the store keeps and how it ranks. A store that only grows is of little use if every new session starts without context and has to recover it through cold search. Session priming covers the read side: each session (any agent, any MCP/HTTP client) starts with context already loaded. `GET /api/brief` returns a salience-ranked, scope-filtered digest of the top-N chunks, injected at session start. The brief is strictly read-only over `chunks`. It never touches `access_count` or `last_accessed_at`, so serving a brief does not contaminate the organic-use signal that feeds salience (§3.4); serve history is tracked in a separate `brief_log` table. With an `agent` scope the digest is a guaranteed union (~half agent-specific, half global high-salience), and near-duplicate variants are collapsed by token-containment so the budget is spent on distinct content.

**Measured behaviour of the brief (summary; full account in the operational supplement, §S3.5).** In production (~6.7k serves/day), the salience-ranked brief first served only 83 distinct chunks across 46.8k serves in 7 days (0.18% diversity), while 931 recent, relevant chunks were never served. We could not measure a downstream follow-up rate, because injected context is not re-queried (3 genuine searches and 0 `answer` calls in 7 days); the utility of an injection-based primer is therefore a property of the served set. A diversity term re-ranks inside the brief only, behind `NOX_BRIEF_DIVERSITY = off | shadow | active`. It combines a log-saturating novelty penalty, a freshness quota and a high-pain floor (`pain >= 0.9` immune). Active-mode gates refuted two slot designs (a hard anti-repeat that dried up within a day; a capped soft penalty that fell  $146 \rightarrow 67 \rightarrow 3$  distinct curated chunks per day) before ranking by time since last serve held: on 2026-06-27 184 of 184 eligible entity files were served (190 distinct curated chunks). Saturating de-duplication reconverges under production volume, which only a multi-day active gate exposed.

## 4. Hybrid Search System

### 4.1 Architecture

Search runs in three layers:

#### Layer 1: FTS5 BM25 (Keyword)

SQLite FTS5 with the `unicode61 remove_diacritics 2` tokenizer (accent-folding, no stemming; it replaced the Porter stemmer in schema v5 because Porter mis-stemmed Portuguese) provides fast keyword matching. Results are scored with BM25 using column weights (`chunk_text`: 1.0, `source_file`: 0.5, `chunk_type`: 0.5). The following post-retrieval boosts apply:

- Type boost: `decision`, `lesson`, `person`, `project` and `pending` chunks receive  $\delta = +1.0$  (FTS layer) /  $+0.5$  (semantic layer)
- Recency boost: chunks from the last 7 days receive  $\delta = +0.5$  (FTS) /  $+0.2$  (semantic)

(These replaced the earlier  $2.0\times/1.5\times$  and  $1.5\times/1.2\times$  multipliers in Wave A.)

**Boost composition.** Every boost enters the score as a delta  $\delta$ , and the deltas are summed before they touch it:  $score = base \times (1 + \sum \delta)$ , where `base` is the layer’s BM25 or similarity score (`staged/1.7a/edits/search.ts`). A boost with weight  $w$  contributes  $\delta = w - 1$ . The section weights are  $w = 2.0$  (compiled),  $1.5$  (frontmatter) and  $0.8$  (timeline), with  $1.0$  for chunks without a section; source-type and tier weights (tier off by default) enter the same way; the type and recency deltas are the values above; and the salience delta lies in  $[-0.5, +0.5]$  (§3.4.3). Weights are never multiplied by each other.

The query sanitizer replaces every character that is not a Unicode letter, digit or whitespace with a space, so a compound term such as “nox-mem” is searched as the two tokens “nox” and “mem”.

#### Layer 2: Gemini Semantic (Vector)

Each chunk is embedded using Google’s `gemini-embedding-001` model<sup>70</sup> (3072 dimensions) with task type `RETRIEVAL_DOCUMENT`. Query embeddings use task type `RETRIEVAL_QUERY` for asymmetric similarity optimization.

Vectors are stored in `sqlite-vec` virtual tables. Retrieval uses `sqlite-vec`’s default L2 distance (rank-equivalent to cosine only if the embeddings are unit-normalized) with a map table (`vec_chunk_map`) that maps `vec_chunks` rowids to `chunks.id` values, because `sqlite-vec` addresses vectors only by rowid.

Scoring normalizes distances to a 0-10 scale with type and recency deltas ( $\delta = +0.5$  and  $+0.2$ , lower than FTS5 to avoid double-boosting in fusion).

#### Layer 3: Reciprocal Rank Fusion (RRF)

---

<sup>70</sup>Lee, Chen, Dua *et al.* (Google), *Gemini Embedding: Generalizable Embeddings from Gemini*, 2025. arXiv:2503.07891; served as `gemini-embedding-001` (model card at `ai.google.dev`). Cited in §4.1 for the 3072-dimension embedding used by Layer 2.

FTS5 and semantic results are merged using RRF with  $k=60$ :

$$\text{RRF\_score}(d) = \sum 1/(k + \text{rank}_i(d))$$

Documents that appear in both result sets receive combined scores and are marked `match_type: "hybrid"`. Content-prefix deduplication (first 50 characters) prevents near-duplicate results.

## 4.2 Performance Characteristics

Three production queries illustrate the qualitative difference between FTS5-only and hybrid retrieval (anecdotal, not a measured quality gain; measured comparisons are in §5):

| Query                  | FTS5 Only | Hybrid                  | Analysis                                       |
|------------------------|-----------|-------------------------|------------------------------------------------|
| “qual o proximo passo” | 0 results | ROADMAP + PHASE-3       | Semantic captures intent without keyword match |
| “nox-mem”              | 0 results | decisions.md + docs     | Vector bypasses tokenizer hyphen issues        |
| “quem e o [name]”      | people.md | people.md + TEAM_MEMORY | RRF combines exact match + semantic context    |

The three queries are verbatim from production usage and are in Portuguese, the operator’s working language; they are reported unchanged because the FTS5 and hybrid outcomes in the table were measured on these exact strings. In the third query a personal name occurring in the corpus is redacted as `[name]`; the retrieval behaviour reported for it was measured on the unredacted string.

## 4.3 Cross-Agent Search

The `crossSearch()` function opens all 7 databases in read-only mode, executes FTS5 queries in each, and merges results with agent attribution. Deduplication uses content-prefix comparison to handle shared documents that appear across multiple agent databases.

# 5. Empirical Evaluation

**Headline results of §5.** EverMemBench figures use the 5-batch protocol (§5.8.1); MemOS figures are the MemOS row of the benchmark paper’s Table 4, not re-run. - **EverMemBench, GPT-4.1-mini (§5.1.6):** Overall 51.68% (95% CI [49.87, 53.48]) against 42.55% for MemOS on the same backbone, +9.13 pp. - **EverMemBench, Gemini-3-flash (§5.1.10):** Overall 63.28% against 59.27% for MemOS on the same backbone, +4.01 pp (+4.50 pp in Table 4’s aggregation); that backbone’s 72.61% full-context baseline

is higher than both, so this is not a state-of-the-art result. - **Classical multi-hop QA (§5.2):** MuSiQue-Ans dev answer F1 58.62% and HotPotQA dev distractor answer F1 73.37%, above the conventional reference readers and about 10–12 pp below Beam Retrieval<sup>71</sup>. - **EverMemBench multi-hop (§5.4):** F\_MH 3.21% (GPT-4.1-mini) and 6.02% (Gemini-3-flash), against 18.88% and 10.84% for MemOS on those backbones; task setup is the leading, not established, account. - **Orchestration (§5.5.2):** a ReAct loop (IterB) raises F\_MH on Gemini-3-flash from 6.02% to 8.03% (+2.01 pp), directional and not significant. - **Operational (§5.7):** KG-path p50 2.5 ms, \$0 per KG-path query and a 399 MB resident set, in one process.

This section reports three evaluation tracks that test the same architectural claims. The Wave A ablation series (§5.1.1–§5.1.4, entity-flavored golden set, nDCG@10) exercises the V10 schema’s section/source-type/salience drivers. The EverMemBench cross-system series (§5.1.5–§5.1.10, n=3,121 queries, task-accuracy vs MemOS Table 4 baselines<sup>72</sup>) covers Phase D / H v2 / G / standalone knobs / Wave B/C composability / Backbone Matrix. The classical multi-hop QA series (§5.2, MuSiQue + HotPotQA) bounds multi-hop reasoning as competent on standard benchmarks, above their conventional reference readers and below current SOTA where corpus structure and scoring are not adversarial. Of the remaining subsections, §5.3 cross-validates on LoCoMo (memory-bench, conversational); §5.4 refines the EverMemBench F\_MH paradox using the §5.2 and §5.3 evidence; §5.5 reports orchestration mechanism-class findings, including the IterB ReAct result on Gemini-3-flash (§5.5.2), the backbone portability of the retrieval knobs (§5.5.5–§5.5.6), an adapter constraint on combining them with IterB (§5.5.7), and an aborted composition test (§5.5.8); §5.6 cross-validates on LongMemEval; §5.7 reports operational characteristics (latency / cost / footprint); and §5.8 documents the methodology and limitations.

**Dual-baseline reporting convention (adopted 2026-05-31).** Orchestration-mechanism evaluations in this revision report against two baselines: (a) the project-convention Phase H v2 GPT-4.1-mini baseline, which preserves comparability with prior revisions but conflates mechanism lift with any backbone swap; and (b) the strongest in-matrix bare baseline (Gemini-3-flash, §5.1.10), which isolates the mechanism’s effect on the strongest available backbone. Mechanism claims rest on the (b) number. Reporting only (a) would conflate ReAct mechanism lift with the +11.60 pp Overall and +15.08 pp MA composite

<sup>71</sup>Zhang, Zhang, Zhang, Liu & Huang, *End-to-End Beam Retrieval for Multi-Hop Question Answering*, NAACL 2024. arXiv:2308.08973. The MuSiQue leaderboard system referenced as the upper bound in §5.2.1.

<sup>72</sup>“MemOS Table 4” throughout §5 means the MemOS row of Table 4 in the **EverMemBench paper**[<sup>longhorizon</sup>] — *not* a table in the MemOS paper itself. The MemOS system is Li, Xi, Li, Chen, Chen, Song, Niu, Wang *et al.*, *MemOS: A Memory OS for AI System*, arXiv:2507.03724, 2025; we compare against its published numbers and did not re-run it (§5.8.5).

that the Gemini-3-flash backbone already gives nox-mem over GPT-4.1-mini (§5.1.10). The convention applies to §5.5 IterB (this revision) and to future MAS-class orchestration evaluations.

## 5.1 EverMemBench + Wave A ablation series

**5.1.1 Wave A ablation — setup and headline** The Wave A evaluation uses an entity-flavored golden set of 100 queries, intended to exercise the V10 schema’s `section` and `pain` dimensions; internal run records disagree on whether the G5 V3 run searched a ~69k-chunk clone of the production store (`g5.db`) or the 500-chunk `entity-eval.db`, and we report the run as recorded without resolving this. No record documents how the 100 queries were selected. Configurations are toggled via environment-variable feature gates (`NOX_SALIENCE_MODE`, `NOX_DISABLE_TIER_BOOST`, `NOX_ENABLE_TIER_BOOST`, `NOX_DISABLE_SECTION_BOOST`, etc.), so individual ranking components can be isolated without code changes between runs. All measurements occur after deployment of the salience formula with `tier_boost` off by default, the `source_type` backfill of 67,949 chunks, and the search wiring; the `source_type` boost was nevertheless inert in this run because of a key mismatch ( $A_{10} = A_8 = 0.6237$ ). Reported  $nDCG@10$  follows the standard TREC formulation (gain by relevance, log-position discount).

**Wave A headline (canonical, 2026-05-19):**  $A_8$  full stack with active salience reaches  $nDCG@10 = 0.6237$  on the entity-flavored golden set ( $n=100$ ), a +78.8% relative improvement over the G3 baseline (0.3488) measured before Wave A deployment, and +9.4% over the mid-deployment G4 checkpoint (0.5702). The best single-component ablation, `section_boost` alone (A3), reaches 0.6228, 99.86% of the full stack’s score. Section-aware ranking accounts for nearly all of the full stack’s gain over unboosted hybrid retrieval within G5 V3 (0.5126), though not for most of the +78.8% over G3 (§5.1.3).

Progression vs prior ablation generations:

| Generation                  | Date       | $A_8$ $nDCG@10$ | $\Delta$ vs G3 baseline | Notes                                                                                                     |
|-----------------------------|------------|-----------------|-------------------------|-----------------------------------------------------------------------------------------------------------|
| G3 baseline<br>(pre-Wave A) | 2026-05-15 | 0.3488          | —                       | Multiplicative salience, <code>tier_boost</code> on, <code>section_boost</code> only via legacy code path |
| G4<br>mid-deployment        | 2026-05-18 | 0.5702          | +63.5%                  | Additive salience wired but <i>active</i> < <i>shadow</i> puzzle observed                                 |

| Generation                 | Date              | A8 nDCG@10    | $\Delta$ vs G3 baseline | Notes                                                                        |
|----------------------------|-------------------|---------------|-------------------------|------------------------------------------------------------------------------|
| <b>G5 V3<br/>canonical</b> | <b>2026-05-19</b> | <b>0.6237</b> | <b>+78.8%</b>           | Wave A fully<br>deployed; reversal<br><i>active &gt; shadow</i><br>confirmed |

The +78.8% compares the G5 V3 full stack with the G3 configuration (multiplicative salience, tier\_boost on). Within G5 V3, hybrid retrieval with no boosts already scores 0.5126, so the Wave A boost stack adds +21.7% (0.5126  $\rightarrow$  0.6237); the remainder of the +78.8% reflects leaving the G3 configuration (tier\_boost alone scores 0.4059). The sub-claims below measure individual contributions within G5 V3 and are not additive. The 12-configuration matrix is recorded only in an archived internal handoff (`handoffs/_archive/HANDOFF-2026-04-28-a-2026-06-14.md`); no per-configuration result files were versioned.

### 5.1.2 Wave A — Claim 1: additive salience, active versus shadow

The Wave A formula replaces the legacy multiplicative `salience = recency  $\times$  pain  $\times$  importance` with a weighted-additive form:

$$\begin{aligned} \text{salience} &= \text{W\_IMPORTANCE} \cdot \text{importance} + \text{W\_RECENCY} \cdot \text{recency} \\ &\quad + \text{W\_PAIN} \cdot \text{pain} + \text{W\_ACCESS} \cdot \text{access\_score} \\ \text{W\_IMPORTANCE} &= 0.55 \quad \text{W\_RECENCY} = 0.15 \quad \text{W\_PAIN} = 0.10 \quad \text{W\_ACCESS} = 0.20 \end{aligned}$$

**Result.** With `NOX_SALIENCE_MODE=active`, A8 reaches 0.6237 vs. 0.6155 with `shadow` (A7), a +1.3% lift that reverses the G4 puzzle, where shadow had out-ranked active. The multiplicative form concentrated 99.7% of chunks in the [0.05, 0.40] salience range. Pain and recency were nearly constant in this corpus (90.67% of chunks at the default pain; 99.76% aged 7–30 days after a restore), so the multiplicative form reduced to a scaled importance. These three distribution figures are from `docs/audits/2026-05-19-salience-distribution-audit.md`. The additive form also adds an access term (weight 0.20) that the multiplicative form lacked. Which of these differences produces the small gain (+0.0082 nDCG@10, n=100, single run, no significance test) was not isolated.

### 5.1.3 Wave A — Claim 2: section\_boost alone reaches 99.86% of the full-stack score

Isolating `section_boost` alone (A3 ablation: `section` enabled, `tier` off, `source_type` off, `salience` shadow) yields nDCG@10 = 0.6228 = 99.86% of the 0.6237 that A8’s full stack reaches. The V10 section weights (`compiled` = 2.0, `frontmatter` = 1.5, `timeline` = 0.8, `legacy` = 1.0, applied as  $\delta = w - 1$  per §4.1) together with the entity-file format introduced in v3.7 account for nearly all of the gain of the full stack over unboosted hybrid retrieval within G5 V3 (0.5126  $\rightarrow$  0.6228, against 0.5126  $\rightarrow$  0.6237 for the full stack), but not for most of the +78.8% over G3: without `section_boost` (A11) the stack still scores 0.5646. A census on 2026-09-09 counted 865 section-bearing chunks across 239 distinct source files represented in the store, of which 184 still exist on

disk (a deleted source file leaves its chunks behind). Files average 3.62 sections each, not the uniform 3 the format suggests, because `timeline` contributes 1..N. Census script and artifact: `paper/measurement/censo-corpus.py`.

The negative control A11 (full stack minus `section_boost`) drops to 0.5646,  $-9.5\%$  relative to A8; this confirms the contribution is not redundant with semantic embeddings or RRF fusion. In this ablation, section-aware boosting over the entity-file form is the main contributor, and the salience formula is not.

**5.1.4 Wave A — Claims 3 & 4: `tier_boost` and `source_type` calibration** `tier_boost` and `source_type` act as calibration; neither carries the gain that §5.1.3 attributes to `section_boost`. Detail in the supplementary material (§S5.1.4).

#### 5.1.5 EverMemBench Phase D — Gemini-2.5-flash headline (5-batch)

**Config:** phaseB adapter, `top_k=20`, rerank OFF, Gemini-2.5-flash backbone. Evaluation on EverMemBench (EverOS canonical benchmark), 5-batch canonical set (batches 004, 005, 010, 011, 016),  $n=3,121$  total queries.

| Metric                                   | nox-mem (5-batch,<br>Gemini-2.5-flash) | MemOS Table 4<br>(Gemini-3-Flash<br>column) | $\Delta$                           |
|------------------------------------------|----------------------------------------|---------------------------------------------|------------------------------------|
| <b>Overall accuracy</b>                  | <b>62.22%</b>                          | 59.27%                                      | <b>+2.95 pp</b> (backbones differ) |
| Overall, Table 4's aggregation (§5.1.10) | 59.21%                                 | 59.27%                                      | $-0.06$ pp                         |

The 5-batch methodology (§5.8) is canonical for this claim. Single-batch estimates from prior runs showed higher variance; the 5-batch aggregate is the number to use for comparative claims. The phaseB adapter uses the production hybrid search stack (FTS5 + Gemini-embedding-001 3072d + RRF  $k=60$ ) with no generation-side augmentation. Table 4 of the benchmark paper reports MemOS on GPT-4.1-mini, Llama-4-Scout and Gemini-3-Flash, not on Gemini-2.5-flash, so the +2.95 pp compares nox-mem on Gemini-2.5-flash with MemOS on Gemini-3-Flash: a cross-backbone comparison, which we do not attribute to retrieval quality. The same-backbone comparisons are §5.1.6 (GPT-4.1-mini) and §5.1.10 (Gemini-3-Flash).

**5.1.6 EverMemBench Phase H v2 — GPT-4.1-mini cross-backbone (5-batch)** **Config:** phaseB adapter, `top_k=20`, rerank OFF, GPT-4.1-mini backbone (OpenAI). 5-batch,  $n=3,121$ .

| Metric                                            | nox-mem 5-batch | 95% CI (t-dist,<br>n=5)   | MemOS Table 4<br>(GPT-4.1-mini<br>col) | $\Delta$        |
|---------------------------------------------------|-----------------|---------------------------|----------------------------------------|-----------------|
| <b>Overall</b>                                    | <b>51.68%</b>   | [49.87, 53.48]            | 42.55%                                 | <b>+9.13 pp</b> |
| Overall,<br>Table 4's<br>aggregation<br>(§5.1.10) | 51.65%          | —                         | 42.55%                                 | +9.10 pp        |
| F_MH<br>(multi-hop)                               | 3.21% (8/249)   | [−1.64, 8.04] (t,<br>n=5) | 18.88%                                 | −15.67 pp       |
| MA_C<br>(Memory<br>Constancy)                     | 84.60%          | —                         | 69.90%                                 | +14.70 pp       |
| MA_P<br>(Memory<br>Proactivity)                   | 65.40%          | —                         | 51.99%                                 | +13.41 pp       |
| MA_U<br>(Memory<br>Update)                        | 70.03%          | —                         | 45.15%                                 | +24.88 pp       |

**Sub-dimensions:** nox-mem scores above MemOS on seven of the nine; F\_MH and F\_TP are below. The 95% CI lower bound (49.87%) exceeds MemOS's 42.55%, so the Overall margin holds under per-batch variance.

**Outlier detection.** Batch 004 alone reported 54.15% overall (+11.60 pp vs MemOS), a +1.70sigma upper-tail outlier (per-batch distribution: 004=54.15%, 005=50.82%, 010=50.72%, 011=50.87%, 016=51.83%, stdev=1.45 pp). Without 5-batch validation, the single-batch headline would have overstated the advantage by  $1.27\times$ . This is the case that motivates the 5-batch protocol (§5.8).

**5.1.7 EverMemBench Phase G — Cross-encoder rerank trade-off study (5-batch) Config:** MiniLM<sup>73</sup>-L-6-v2 cross-encoder rerank (22M params), top\_k=20 pool rescored, Gemini-2.5-flash backbone. 5-batch, n=3,121.

Cross-encoder reranking<sup>74</sup> shows a 4-dimensional trade-off across retrieval workload types:

<sup>73</sup>Wang, Wei, Dong, Bao, Yang & Zhou, *MiniLM: Deep Self-Attention Distillation for Task-Agnostic Compression of Pre-Trained Transformers*, NeurIPS 2020. arXiv:2002.10957. The cross-encoder checkpoint reranked in §5.1.7 is the `ms-marco-MiniLM-L-6-v2` distillation of this model (22M params).

<sup>74</sup>Reimers & Gurevych, *Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks*, EMNLP 2019. arXiv:1908.10084. Source of the bi-encoder / cross-encoder distinction used throughout §5.1.7 and §7.2 F5.

| Category type                        | $\Delta$ vs Phase D (no rerank)                                                                  | Direction                                                                     |
|--------------------------------------|--------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------|
| Hard-recall: F_MH<br>(multi-hop)     | <b>+1.61 pp</b> (Phase G level 6.83%,<br>95% CI [3.97, 9.69], which contains<br>Phase D’s 5.22%) | marginal gain                                                                 |
| Hard-recall: F_HL<br>(high-level)    | +2.58 pp                                                                                         | marginal gain                                                                 |
| Hard-recall: F_TP<br>(temporal)      | +2.00 pp                                                                                         | marginal gain                                                                 |
| Head-precision: F_SH<br>(single-hop) | +0.40 pp                                                                                         | quasi-neutral                                                                 |
| Head-precision: MC<br>(multi-choice) | −2.63 pp                                                                                         | regression                                                                    |
| Memory Awareness: MA_C               | <b>−4.00 pp</b>                                                                                  | regression, not significant<br>(paired per-batch 95% CI<br>~[−11.0, +3.0] pp) |
| Memory Awareness: MA_P               | <b>−2.80 pp</b>                                                                                  | regression (paired per-batch<br>95% CI ~[−5.5, −0.1] pp,<br>borderline)       |
| Memory Awareness: MA_U               | <b>−3.84 pp</b>                                                                                  | regression, not significant<br>(paired per-batch 95% CI<br>~[−8.8, +1.4] pp)  |
| Overall                              | −0.96 pp                                                                                         | net regression                                                                |

The F\_MH gain of +1.61 pp closes only 11.8% of the MemOS F\_MH gap (Phase D baseline 5.22%  $\rightarrow$  Phase G 6.83% vs MemOS 18.88%, a GPT-4.1-mini figure: Table 4 has no Gemini-2.5-flash column). The Memory Awareness (MA) regression of −3 to −4 pp was not seen in the single-batch gate (batch 004) because the Phase D baseline it was compared against had no MA scores, so no MA delta could be computed (§5.8.3); batch 004 in fact shows the largest MA\_C drop of the five (80%  $\rightarrow$  68%). The −0.96 pp overall regression appears in four of the five batches (batch 005 improves by +2.13 pp) and is not statistically significant (paired per-batch 95% CI ~[−3.4, +1.5] pp); it is 2.3 $\times$  smaller than the single-batch −2.24 pp estimate. The paired intervals in this subsection are computed from the per-batch values in `eval/evermembench/RESULTS-PATHB-FULL.md` (Phase D) and `eval/evermembench/RESULTS-PHASEG-5BATCH.md` (Phase G), and for MA from `eval/evermembench/results/analysis-phase{D,3}-batch-*.txt`; they supersede the “CI excludes 0” of `RESULTS-PHASEG-5BATCH.md`, which compared levels rather than paired differences.

**Default:** off; available behind a flag (`--rerank`, `NOX_RERANKER_ENABLED=1` or `/api/answer?mode=exploratory`). Search p50 rose from ~1.1 s to 4.8 s on batch 004 (+3.7 s; `eval/evermembench/RESULTS-PHASEG.md`).

**5.1.8 EverMemBench — standalone retrieval-augmentation knobs (5-batch)** Three retrieval-augmentation knobs (KG path expansion, an adaptive query classifier (AC) that routes queries to cross-encoder rerank, and multi-query fan-out (MQ)) were measured standalone on the 5-batch protocol; MAP is reported with the combinations in §5.1.9. Each is individually small and none

reaches significance alone; the per-knob table and the per-backbone interaction matrix are in the supplementary material. From this study the main text uses the composability result of §5.1.9 and the backbone-conditional behaviour of §5.5.5–§5.5.6.

**5.1.9 Wave B + Wave C composability — sub-additive F\_MH** The standalone knobs (§5.1.8) and a separate MAP (section-bypass) run identified four mechanisms, each with an F\_MH lift of its own. Wave B (two-knob combinations) and Wave C (the three-knob combination) measure whether these lifts stack or overlap. All runs in this subsection use gpt-4.1-mini on the 5-batch protocol. F\_MH is scored on about 50 questions per batch (249 in total), so one question moves a batch score by 2 pp.

**KG + MQ (same retrieval stage):** KG path and MQ expansion co-fire on 90.8% of EverMemBench queries. MQ fires on 99.3% of them, so this is essentially KG’s own firing rate (91.4%); that the two act on the same evidence is a hypothesis. The combination reaches F\_MH 8.02%, +4.81 pp over the Phase H v2 baseline, against +6.42 pp if the standalone lifts added (eval/evermembench/RESULTS-WAVE-B-KG-MQ.md).

**KG + MAP (different retrieval stages):** KG (entity-walk) and MAP (section bypass) act at different points in retrieval:

| Configuration                | F_MH         | $\Delta$ vs Phase H v2 |
|------------------------------|--------------|------------------------|
| Phase H v2 baseline          | 3.21%        | —                      |
| Phase KG standalone          | 6.02%        | +2.81 pp               |
| Phase MAP standalone         | 7.22%        | +4.02 pp               |
| <b>Phase KG+MAP composed</b> | <b>7.25%</b> | <b>+4.04 pp</b>        |

MAP’s +4.02 pp is the difference eval/evermembench/RESULTS-PHASEMAP-5BATCH.md reports against the unweighted per-batch mean of the baseline, 3.20%; against the weighted 3.21% it is +4.01 pp. The combination keeps MAP’s standalone lift (+4.04 against +4.02 pp) and adds almost none of KG’s: summing the standalone lifts would predict +6.83 pp. KG+MAP closes ~26% of the MemOS F\_MH gap, at an MA composite cost of −5.02 pp against the baseline; the run’s own MA-recovery gate failed (eval/evermembench/RESULTS-WAVE-B-KG-MAP.json).

**KG + MQ + MAP (Wave C):** Wave C tested the triple composition (batches run sequentially, outlier-aware aggregation). It reaches 7.23% F\_MH (+4.02 pp), statistically indistinguishable from the KG+MAP doublet, against +10.44 pp if the three standalone lifts added; per batch it ranges from 2% to 18% (t-based 95% CI [−0.74, 15.21]; eval/evermembench/RESULTS-WAVE-C-TRIPLE.json stores a z interval with population SD, [2.20, 12.27]). Adding MQ on top of KG+MAP yields no incremental lift.

**What the composability runs show, and what they do not:**

1. Same-stage retrieval knobs plausibly overlap (MQ fires on 99.3% of queries, hence wherever KG fires), and their combination is sub-additive.
2. The different-stage pair (KG + MAP) is sub-additive too: it keeps the larger standalone lift and little more.
3. No measured combination exceeded about 8% F\_MH on this backbone (the best is KG+MQ at 8.02%). With ~50 F\_MH questions per batch and t-based confidence intervals 8–16 pp wide, this is an observed plateau for these knobs, not an established ceiling of the retrieval stage.
4. Orchestration-stage mechanisms (§5.5) and backbone changes (§5.1.10) are the other directions measured.

**5.1.10 Backbone Matrix — Gemini-3-flash on EverMemBench, against MemOS on the same and on a different backbone Config:** phaseB adapter, top\_k=20, rerank OFF, Gemini-3-flash backbone (frontier reasoning tier). 5-batch n=3,121.

| Metric                               | nox-mem<br>(Gemini-3-<br>flash) | MemOS,<br>Table 4,<br>Gemini-3-<br>Flash | $\Delta$ same<br>backbone | MemOS,<br>Table 4,<br>GPT-4.1-<br>mini | $\Delta$ across<br>backbones | $\Delta$ vs<br>nox-mem<br>gpt-4.1-mini<br>(Phase H<br>v2) |
|--------------------------------------|---------------------------------|------------------------------------------|---------------------------|----------------------------------------|------------------------------|-----------------------------------------------------------|
| <b>Overall</b>                       | <b>63.28%</b>                   | 59.27%                                   | <b>+4.01 pp</b>           | 42.55%                                 | +20.73 pp                    | +11.60 pp                                                 |
| Overall,<br>Table 4's<br>aggregation | 63.77%                          | 59.27%                                   | +4.50 pp                  | 42.55%                                 | +21.22 pp                    | +12.12 pp                                                 |
| <b>MA composite</b>                  | <b>88.42%</b>                   | 86.70%                                   | <b>+1.72 pp</b>           | 55.68%                                 | +32.74 pp                    | +15.08 pp                                                 |
| MA_C                                 | 89.20%                          | 81.84%                                   | +7.36 pp                  | 69.90%                                 | +19.30 pp                    | +4.60 pp                                                  |
| MA_P                                 | 90.00%                          | 87.59%                                   | +2.41 pp                  | 51.99%                                 | +38.01 pp                    | +24.60 pp                                                 |
| MA_U                                 | 86.06%                          | 90.67%                                   | −4.61 pp                  | 45.15%                                 | +40.91 pp                    | +16.03 pp                                                 |

nox-mem values: `eval/evermembench/RESULTS-BACKBONE-MATRIX.json`;  
 MemOS values: Table 4 of the EverMemBench paper<sup>75</sup>, where the MA composite is the mean of MA\_C, MA\_P and MA\_U.

**Aggregation.** Table 4’s “Average” is the unweighted mean of its nine sub-dimensions (it reproduces the published 42.55 and 59.27). Our “Overall” weights each sub-dimension by its query count and also counts F\_HL (388 of 3,121 queries), a category Table 4 does not report. The row “Table 4’s aggregation” averages the nine sub-dimensions Table 4 reports, from the same counts: it uses Table 4’s aggregation formula, not its population. The five released batches hold 2,733 questions in these nine categories against the paper’s 2,400 after filtering

<sup>75</sup>“MemOS Table 4” throughout §5 means the MemOS row of Table 4 in the EverMemBench paper[<sup>longhorizon</sup>] — not a table in the MemOS paper itself. The MemOS system is Li, Xi, Li, Chen, Chen, Song, Niu, Wang *et al.*, *MemOS: A Memory OS for AI System*, arXiv:2507.03724, 2025; we compare against its published numbers and did not re-run it (§5.8.5).

(ours/theirs: Single-hop 247/213, Constraint 500/402, Proactivity 500/427, Update 287/268, Style 181/176, Skill 221/169, Role 248/196; Multi-hop 249 and Temporal 300 match). It moves the margin by under 0.5 pp here, and from +2.95 to  $-0.06$  pp for the Gemini-2.5-flash run of §5.1.5. MA composites are unweighted on both sides.

**Which comparison to cite.** On the same backbone the margins are small: +4.01 pp Overall and +1.72 pp MA composite, with MA\_U below MemOS. On that backbone Table 4 also reports a full-context baseline (the complete dialogue history, no memory system) of 72.61%, above both memory systems, as it is above every memory-augmented system in that column. The larger margins in the GPT-4.1-mini column compare nox-mem on Gemini-3-flash with MemOS on GPT-4.1-mini; they are cross-backbone and are not state-of-the-art results, and §5.5.4–§5.5.6 measure directly how much backbone choice alone can move these metrics (single-stage retrieval knobs transfer at only 0–40% between these two backbones). The same-backbone comparison on GPT-4.1-mini is §5.1.6 (+9.13 pp, 95% CI [49.87, 53.48]). MemOS numbers are published, not re-run (§5.8.5).

**Backbone Matrix interpretation.** The +20.73 pp cross-backbone Overall margin is the sum of the same-backbone GPT-4.1-mini margin (+9.13 pp, §5.1.6) and nox-mem’s own gain from the backbone swap (+11.60 pp); the +32.74 pp MA composite margin is likewise +17.66 pp plus +15.08 pp. The decomposition holds by construction, so it says nothing about whether architecture and backbone interact. MemOS gains more from the same swap (+16.72 pp Overall, 42.55%  $\rightarrow$  59.27%) than nox-mem does (+11.60 pp; +12.12 pp in Table 4’s aggregation).

**5.1.11–5.1.12 Cross-backbone analysis and the F\_MH retrieval-bound finding** Reported in full in the supplementary material (§S5.1.11, §S5.1.12): backbone-portability analysis and the gpt-4.1-mini-era strategic implication of the F\_MH retrieval-bound result. Neither is referenced elsewhere in this manuscript. The conclusion they bear on, that the F\_MH gap is not a wholesale reasoning failure and that task setup is the leading, not established, account, is stated in §5.4.

## 5.2 Classical multi-hop QA — competitive without fine-tuning, below current SOTA

The EverMemBench F\_MH gap leaves two explanations open: nox-mem’s multi-hop reasoning is limited, or the EverMemBench F\_MH track poses a corpus-specific structural challenge. To separate them, we ran nox-mem against two canonical multi-hop QA benchmarks where the task structure is well-known and reader SOTA numbers are published: MuSiQue (multi-hop questions decomposable into sub-questions) and HotPotQA (multi-hop questions over Wikipedia with distractor paragraphs). Both are standard adversarial multi-hop setups and widely used reference benchmarks for retrieval-augmented multi-hop systems.

On both benchmarks nox-mem performs multi-hop QA competently without benchmark-specific fine-tuning, scoring above the specialized multi-hop readers conventionally used as reference points for each dataset and below the current state of the art. The comparison is diagnostic. It checks whether multi-hop composition is a capability floor for the system and makes no leaderboard claim.

**5.2.1 MuSiQue-Ans dev — answer F1 58.62%, above EX(SA), below Beam Retrieval Config:** nox-mem hybrid retrieval (FTS5 + Gemini-embedding-001 + RRF, top\_k=20), GPT-4.1-mini generation backbone, MuSiQue-Ans dev (n=2,417), each question’s own 20 paragraphs ingested into a fresh per-question store (the reading-comprehension setting). Per-question metric: F1 over tokenized answer match.

| System                                                   | Split                 | Answer F1     | $\Delta$ vs nox-mem | Source                                         |
|----------------------------------------------------------|-----------------------|---------------|---------------------|------------------------------------------------|
| Beam Retrieval<br>(Zhang et al.,<br>NAACL 2024)          | <b>test</b>           | <b>69.20%</b> | <b>+10.58 pp</b>    | Table 4<br>(arxiv:2308.08973)                  |
| <b>nox-mem</b><br><b>(hybrid, no</b><br><b>rerank)</b>   | <b>dev</b>            | <b>58.62%</b> | —                   | this work<br>(§5.2.1)                          |
| EX(SA)<br>(Trivedi et<br>al. 2022)                       | dev                   | 49.80%        | −8.82 pp            | MuSiQue paper<br>Table 4<br>(arxiv:2108.00573) |
| IRCoT QA,<br>GPT3 CoT<br>reader (Trivedi<br>et al. 2023) | dev (500-q<br>subset) | 36.50%        | −22.12 pp           | IRCoT paper<br>Table 4<br>(arxiv:2212.10509)   |

Splits differ in the top row. Beam Retrieval reports answer F1 on the MuSiQue-Ans *test* set (their Table 4); its development-set table (Table 3) reports *retrieval* EM/F1 (77.37/89.77 at beam size 1, 79.31/90.51 at beam size 2), not answer F1, so no split-matched answer figure is published for that system. Dev/test differences for the MuSiQue paper’s own models range from 0.8 (EX(SA), 49.8  $\rightarrow$  49.0) to 5.0 points (SA, 47.3  $\rightarrow$  52.3; its Tables 4 and 5, arXiv:2108.00573v3), below the  $\sim$ 10-point gap but not negligible.

nox-mem lands above the benchmark’s strongest published reader and the later IRCoT system, and 10.58 pp below Beam Retrieval, the current published state of the art on this metric. Two cautions apply to the margins over EX(SA) and IRCoT. First, both are 2022–2023 systems while our reader is GPT-4.1-mini: the comparison confounds retrieval design with three years of backbone progress, and we do not claim the delta measures the former. Second, EX(SA) is 49.80 on dev and 49.0 on test, where the MuSiQue paper’s own SA model scores 52.3 and the strongest earlier system in Beam Retrieval’s Table 4 (arXiv:2308.08973v2), RoHTmix, scores 63.6 (Beam Retrieval: 69.2), so “strongest specialized reader in the MuSiQue paper” is a statement about that paper, not about the field. Protocol: like Beam Retrieval and the MuSiQue paper’s models, we work over each question’s own 20 candidate paragraphs, so top\_k=20 returns nearly the

whole candidate set (support\_hit@20 99.96%); IRCoT instead retrieves open-domain from a 139,416-paragraph corpus built from all MuSiQue questions (arXiv:2212.10509v2, App. A), so the margin over IRCoT also reflects the easier retrieval setting. The number supports a narrower claim, which is enough for §5.4: multi-hop composition is not a capability floor for this pipeline. Because retrieval returns nearly every candidate paragraph (support\_hit@10 99.88%), this result measures the reader over the per-question candidate set; it does not test hybrid retrieval recall or RRF fusion.

The per-hop breakdown (eval/musique/RESULTS-MUSIQUE.md) is above EX(SA)’s per-hop answer F1 at every hop count (2-hop 59.42 vs 57.9; 3-hop 52.93–64.27 vs 47.9; 4-hop 47.84–52.35 vs 28.1, arXiv:2108.00573v3, Section 8.1), with the smallest margin on 2-hop questions; no per-template breakdown was computed.

**5.2.2 HotPotQA distractor — answer F1 73.37%, above the original baseline, below Beam Retrieval and FE2H** Config: nox-mem hybrid retrieval (FTS5 + Gemini-embedding-001 + RRF, rerank off) with top\_k=5 (§5.2.1 used top\_k=20), GPT-4.1-mini generation backbone (temperature 0), HotPotQA dev distractor<sup>76</sup>, all 7,405 questions (3 errors), each question’s 10 paragraphs (2 gold, 8 distractors) ingested into a fresh per-question store. Per-question metric: ans\_F1 over tokenized answer match.

| System                                                                            | Split             | Answer F1     | $\Delta$ vs nox-mem | Source                            |
|-----------------------------------------------------------------------------------|-------------------|---------------|---------------------|-----------------------------------|
| Beam Retrieval (Zhang et al., NAACL 2024)                                         | <b>blind test</b> | <b>85.04%</b> | <b>+11.67 pp</b>    | Table 5 (arxiv:2308.08973)        |
| FE2H on ALBERT                                                                    | <b>blind test</b> | 84.44%        | +11.07 pp           | hotpotqa.github.io leaderboard    |
| <b>nox-mem (hybrid, no rerank)</b>                                                | <b>dev</b>        | <b>73.37%</b> | —                   | eval/hotpotqa/RESULTS-HOTPOTQA.md |
| Original dataset baseline, Clark & Gardner (2017) architecture (Yang et al. 2018) | dev               | 58.28%        | −15.09 pp           | HotPotQA paper (arxiv:1809.09600) |

Splits differ in the top two rows. The official HotPotQA leaderboard reports the blind test set only; our figure is dev distractor. No dev-set answer F1 is published for the blind-test entries, so the size of the dev/test difference is unknown.

<sup>76</sup>Yang, Qi, Zhang, Bengio, Cohen, Salakhutdinov & Manning, *HotpotQA: A Dataset for Diverse, Explainable Multi-hop Question Answering*, EMNLP 2018. arXiv:1809.09600. Used in §5.2.2, §5.4.

The answer F1 of 73.37% is above the dataset’s original baseline (58.28%) without specialized training or HotPotQA-specific fine-tuning, and 11.67 points below the leaderboard’s top entry (Beam Retrieval, 85.04%) and 11.07 below FE2H (84.44%, rank 4; leaderboard read 2026-10-02). That baseline is a reimplementation of the Clark & Gardner reader<sup>77</sup>. The result corroborates §5.2.1 in the narrow sense that matters for §5.4: general-purpose hybrid retrieval with an off-the-shelf reader clears classical multi-hop QA competently, well short of purpose-built systems that optimize the retrieval chain end-to-end for these datasets.

**5.2.3 What the classical-QA results do and do not license for the F\_MH narrative** The MuSiQue and HotPotQA results decouple two quantities: - **Multi-hop reasoning quality** is a property of the reader (generation backbone) plus the retrieval candidate pool. - **EverMemBench F\_MH** measures something different: section-anchored entity-chain composition over very long conversation histories, scored CORRECT/WRONG by an LLM judge, without partial credit.

On benchmarks where multi-hop reasoning quality is the question and each question comes with its own 10–20 candidate paragraphs, gold included (MuSiQue, HotPotQA), nox-mem performs in the same regime as published specialized systems without matching the leaders. That licenses only a bounded claim: multi-hop composition is not where this pipeline fails. The 10–12 point shortfall against Beam Retrieval means classical multi-hop still has headroom for this system too, so the EverMemBench F\_MH gap cannot be attributed to task structure alone on the strength of these two results. The same-metric evidence that the F\_MH track is intrinsically hard is separate and stronger: the best published memory-augmented system on it reaches 18.88% (MemOS on GPT-4.1-mini), scored the same way. §5.4 develops what remains.

---

### 5.3 LoCoMo cross-bench — evidence hit@10 and token-F1 (not ranked)

LoCoMo (Maharana et al. 2024; arxiv:2402.17753) is the canonical long-conversation memory benchmark with 10 public conversations (`locomo10.json`; 19–32 sessions and 369–689 turns each; 1,986 question-answer pairs). It provides a retrieval metric and an end-to-end F1 metric; Mem0 reports an overall LLM-as-a-Judge score (J) of 66.88% for its base system and 68.44% for its graph variant (Mem0<sup>g</sup>); its paper reports token-F1 only per question category, with no overall F1. nox-mem was evaluated on both tracks with hybrid retrieval + GPT-4.1-mini.

---

<sup>77</sup>Clark & Gardner, *Simple and Effective Multi-Paragraph Reading Comprehension*, 2017. arXiv:1710.10723. The reader architecture reimplemented as the original HotpotQA baseline that §5.2.2 compares against.

**5.3.1 Evidence hit@10 — strict 74.52% (not comparable to Mem0’s LLM-judge score)** **Config:** nox-mem hybrid retrieval (top\_k=20 requested, scored at 10), oracle-free, all 10 conversations of the public release (1,986 QA; 1,966 with gold evidence).

| Track                   | nox-mem (strict) | nox-mem (adjacency-2) | Mem0 published J | note                                 |
|-------------------------|------------------|-----------------------|------------------|--------------------------------------|
| Overall retrieval@10    | <b>74.52%</b>    | 87.44%                | 66.88% (J)       | different metric, not a head-to-head |
| Multi-hop retrieval@10  | <b>82.21%</b>    | <b>92.91%</b>         | —                | —                                    |
| Single-hop retrieval@10 | 80.36%           | 92.03%                | —                | —                                    |
| Temporal retrieval@10   | 77.96%           | 84.74%                | —                | —                                    |

The strict retrieval@10 of 74.52% is not comparable to Mem0’s published end-to-end LLM-judge score (J) of 66.88% (retrieval@10 vs J are different metrics, not a same-corpus head-to-head; for the like-for-like nDCG@10 comparison where Mem0 wins LoCoMo under its native embedder, see §6.3). Evidence hit@10 is not an upper bound on token-F1: in the F1 push, adversarial questions score 65.78% F1 against 60.18% hit@10. It indicates how often at least one gold turn reaches the prompt. On multi-hop questions at least one gold turn is in the top 10 for 82.21% (strict; 92.91% adjacency-2), but evidence recall@10 is 51.59%, so about half of the evidence a multi-hop answer needs is retrieved. No retrieval metric was measured on EverMemBench, so this does not compare the two benchmarks’ retrieval difficulty (sources: `eval/locomo/RESULTS-LOCOMO.md`, `results/RESULTS-FULL-1986q.json`, `results/RESULTS-FULL-SOTA-PUSH-1986q.json`).

**5.3.2 F1 push — 51.85% token-F1 (not ranked against published scores)** We ran a prompt-level F1 push with the same retrieval pool to quantify the gap between evidence hit@10 and end-to-end F1: nox-mem reaches a token-F1 of 51.85%. We do not rank it against published systems. The comparison table of the Mem0 paper<sup>78</sup> reports overall LLM-as-a-Judge (J) scores (Mem0 66.88, Mem0<sup>g</sup> 68.44, Zep 65.99, LangMem 58.10, OpenAI memory 52.90, full context 72.90) and token-F1 only per question category, so none of them is an overall F1 comparable to ours. Verbosity explains only part of the gap between strict evidence hit@10 (74.52%) and token-F1 (51.85%): the F1 push already uses a 1–5-word answer prompt, which accounted for the step from 34.90% to 50.38%, and the remainder includes composition errors on multi-hop, commonsense and temporal questions. The Mem0 paper reports

<sup>78</sup>Chhikara, Khant, Aryan, Singh & Yadav, *Mem0: Building Production-Ready AI Agents with Scalable Long-Term Memory*, arXiv:2504.19413, 2025. Implementation: `mem0ai/mem0` — open-source memory layer for LLM agents (PostgreSQL + Qdrant backend, OpenAI embeddings by default), `github.com/mem0ai/mem0`. Used in §1.4, §5.3.2, §6.3, Table 2.

token-F1 per category only (Mem0 28.64–48.93; Mem0<sup>g</sup> 24.32–51.55; its Table 1, arXiv:2504.19413v1), which gives no evidence on whether its extraction prompts close this gap.

Injecting each conversation’s session dates into the prompt of temporal questions, with a ‘D Month YYYY’ format hint, lifted temporal F1 from 28.27% to 44.21% (+15.94 pp; +1.47 pp overall). A post-hoc date normalizer lowered temporal F1 in the 100-question smoke and was disabled in the full run (eval/locomo/RESULTS-LOCOMO-SOTA-PUSH.md; temporal\_norm\_enabled: false in results/RESULTS-FULL-SOTA-PUSH-1986q.json).

**5.3.3 Path to  $\geq 55\%$  F1 — not measured** No run measured a route to  $\geq 55\%$  token-F1 on LoCoMo. The run report names retrieval changes (iterative retrieval) or a stronger generator as candidates (eval/locomo/RESULTS-LOCOMO-SOTA-PUSH.md); neither was tested on LoCoMo.

#### 5.4 EverMemBench F\_MH paradox — refined

Three independent multi-hop measurements change how the EverMemBench F\_MH absolute number should be read:

| Benchmark                      | Multi-hop metric                              | nox-mem score                                          | Conventional reference readers              | Published SOTA (split noted)                                           | Position vs references                               |
|--------------------------------|-----------------------------------------------|--------------------------------------------------------|---------------------------------------------|------------------------------------------------------------------------|------------------------------------------------------|
| <b>MuSiQue-Ans dev</b>         | answer F1 (multi-hop decomposable)            | <b>58.62%</b>                                          | 36.50% (IRCoT) – 49.80% (EX(SA))            | 69.20% (Beam Retrieval, <i>test</i> )                                  | +8.82 pp vs EX(SA);<br><b>–10.58 pp vs SOTA</b>      |
| <b>HotPotQA dev distractor</b> | answer F1 (multi-hop bridge)                  | <b>73.37%</b>                                          | 58.28% (original baseline)                  | 85.04% (Beam Retrieval, <i>blind test</i> )                            | above original baseline;<br><b>–11.67 pp vs SOTA</b> |
| <b>LoCoMo</b>                  | evidence hit@10 strict                        | <b>74.52%</b>                                          | 66.88% (Mem0 LLM-judge J, different metric) | —                                                                      | <b>any-hit rate; not comparable to Mem0 J</b>        |
| <b>LoCoMo (F1 push)</b>        | token-F1                                      | 51.85%                                                 | 66.88% (Mem0 J, different metric)           | —                                                                      | not ranked                                           |
| <b>EverMemBench F_MH</b>       | judged, binary (multi-hop chain on long conv) | 3.21% (gpt-4.1-mini) / 6.02% (Gemini-3-flash), 5-batch | —                                           | 18.88% (MemOS, GPT-4.1-mini) / 10.84% (MemOS, Gemini-3-Flash), Table 4 | <b>–15.67 / –4.82 pp, same backbone</b>              |

If nox-mem’s multi-hop reasoning were structurally weak, the MuSiQue and HotPotQA results would sit near the original benchmark readers rather than

well above them. They do not: the pipeline composes multi-hop answers competently on both. That rules out a wholesale reasoning failure as the explanation for F\_MH, which is what this argument requires. It does not establish that classical multi-hop is saturated for this system; the 10–12 point shortfall against Beam Retrieval says the opposite. The contrast with EverMemBench F\_MH is therefore between *competent-with-headroom* and 3–6%, not between *solved* and *broken*. The quantitative evidence that the F\_MH track is intrinsically hard comes from the track itself, not from cross-metric comparison: the best published memory-augmented system (MemOS, GPT-4.1-mini column) attains 18.88% on this same track and metric (10.84% for MemOS in the Gemini-3-Flash column; LLM-judged; the benchmark paper does not name its judge model); full context on Gemini-3-Flash reaches 26.51%. The LoCoMo evidence hit@10 of 74.52% strict (82.21% on multi-hop) shows that at least one gold turn usually reaches the prompt, while multi-hop evidence recall@10 is 51.59%.

**The proposed explanation.** Four features of the EverMemBench task setup are candidates for the F\_MH 3–6% numbers. We present them as the leading account rather than an established attribution: nothing below separates a corpus effect from a system limitation, and §5.2 leaves 10–12 points of headroom on classical multi-hop. The benchmark’s oracle runs (GPT-4.1-mini F\_MH 2.41% with full context in Table 4 → 97.99% given ground-truth evidence in Table 5) point at evidence retrieval and attribution, the memory system’s job, not at the scoring rule:

1. **Very long conversation chains:** EverMemBench F\_MH questions require chaining evidence across speakers, groups and days in a corpus of about 1M tokens per project (arXiv:2602.01313v3), far beyond MuSiQue ( $\leq 4$  supporting paragraphs) or HotPotQA (2 gold paragraphs).
2. **All-or-nothing scoring:** following the benchmark’s protocol, an LLM judge labels each F\_MH answer CORRECT or WRONG for semantic equivalence with the gold answer (our runs: its harness, Gemini-2.5-flash as judge). Wording is not penalised, but a partly resolved chain scores zero. MuSiQue F1 and HotPotQA ans\_F1 give partial credit.
3. **Entity-anchor sparsity:** EverMemBench questions often lack explicit entity tokens that nox-mem’s section/source-type boost framework can latch onto. The MAP (section-bypass) mechanism (§5.1.9) was designed to address this sparsity.
4. **Memory-vs-retrieval mismatch:** EverMemBench is a *memory* benchmark with implicit world-state updates; the chunks that answer F\_MH questions may not be the chunks that explicit retrieval would surface. This is the architectural distinction MemOS optimises for.

**Implication for the orchestration experiments (2026-05-31).** Retrieval-stage knobs (KG, MQ, MAP) and their combinations reached at most about 8% F\_MH on gpt-4.1-mini, and every combination was sub-additive (§5.1.9). That leaves two directions for the remaining EverMemBench F\_MH gap: (a) orchestration-stage multi-round refinement matching the long-chain structure

(IterB ReAct), and (b) a backbone change (Backbone Matrix §5.1.10: Gemini-3-flash raises nox-mem’s F\_MH from 3.21% to 6.02%). The §5.5 IterC mechanism-class finding shows that orchestration mechanisms do not transfer equally to EverMemBench F\_MH (parallel decomposition vs sequential refinement). In the §5.5.2 IterB ReAct result, on the strongest backbone (Gemini-3-flash bare), the multi-round retrieve-reason loop lifts F\_MH +2.01 pp (8.03% from 6.02% bare baseline), a directional change that is not statistically significant; its absolute level matches the best retrieval-stage combination on gpt-4.1-mini (KG+MQ, 8.02%), a comparison across backbones. The paradox is therefore refined rather than dissolved: task setup remains the leading, not established, account of EverMemBench F\_MH, and the backbone and orchestration directions each move it by a few points.

### 5.5 Orchestration — IterC Self-Ask F\_HL +35.84 pp, IterB ReAct on Gemini-3-flash, and mechanism-class distinction

This subsection covers orchestration-stage mechanisms, which sit above the retrieval-stage knobs of Wave A/B/C (§5.1.9). This revision reports two of them: §5.5.1 IterC (parallel decomposition via Self-Ask, F\_HL +35.84 pp on the 5-batch protocol) and §5.5.2 IterB (sequential refinement via ReAct, a directional F\_MH lift on the strongest backbone). §5.5.3 records the mechanism-class distinction drawn from the IterC/IterB contrast. §5.5.4 reports per-knob measurements on Gemini-3-flash that replaced the earlier projection; the composition of IterB with the retrieval-stage knobs itself remained untested (the test did not complete, §5.5.8).

#### 5.5.1 IterC — Self-Ask parallel decomposition (F\_HL +35.84 pp)

IterC implements a Self-Ask-style sub-query loop: the generation model decomposes the query into sub-questions, each sub-question is retrieved separately, and a final synthesis step composes the answer.

| Metric                             | IterC (5-batch)                                                                  | Phase H v2 baseline | $\Delta$                                    |
|------------------------------------|----------------------------------------------------------------------------------|---------------------|---------------------------------------------|
| <b>F_HL</b><br><b>(high-level)</b> | <b>58.52%</b>                                                                    | 22.68%              | <b>+35.84 pp</b> (not a pre-set gate)       |
| F_MH<br>(multi-hop)                | 2.81%                                                                            | 3.21%               | −0.40 pp (pre-set gate<br>≥ +2 pp: not met) |
| F_SH<br>(single-hop)               | 70.89%                                                                           | 80.97%              | −10.08 pp                                   |
| MA composite                       | 68.98%                                                                           | 73.34%              | −4.36 pp (gate ≥ −3<br>pp: not met)         |
| Overall                            | 53.13%                                                                           | 51.68%              | +1.45 pp (gate ≥ −2<br>pp: met)             |
| Cost / latency                     | 1 decomposer + ~3<br>sub-answer + 1 final<br>LLM call per query; p95<br>3,688 ms | baseline            | added cost (gate p95<br>≤ 5,000 ms: met)    |

Two of the four pre-set gates were met (Overall and latency).

The F\_HL +35.84 pp lift is the largest F\_HL change among the mechanisms measured in this paper. F\_HL (high-level synthesis questions) may benefit from up-front parallel decomposition because the synthesis target is a composition over independent sub-facts; this is a post-hoc reading, not a tested mechanism. IterC also departs from Self-Ask as published (Press et al.<sup>79</sup>): there, the model asks itself and answers follow-up questions before the final answer, on multi-hop compositional questions, which is the class where IterC showed no lift. The F\_MH no-lift (−0.40 pp) is what forced the mechanism-class distinction documented in §5.5.3.

IterC is off by default and available behind a flag (NOX\_ITERC\_ENABLED=1 in the evaluation adapter), because of its added cost and the F\_MH no-lift; this later project decision overrides the POC’s own ‘document and defer’ recommendation. The mechanism-class distinction reframed IterB ReAct as the leading EverMemBench F\_MH candidate, which §5.5.2 tests: the lift is directional, not significant.

### 5.5.2 IterB ReAct — directional F\_MH lift on the best backbone

IterB ReAct lifts F\_MH +2.01 pp on Gemini-3-flash (8.03% vs 6.02%; t-based 95% CI of IterB [6.27, 9.79], of the paired per-batch difference [0.25, 3.76]). The paired per-batch interval excludes zero, but it rests on five batch differences (+4, 0, +2, +2, +2.04 pp; one question moves a batch by 2 pp). Both runs answer the same 249 F\_MH questions, so the question-level test is McNemar’s on discordant pairs. The per-question pairing is not archived; with 20 vs 15 correct the net difference is 5 questions, and the exact two-sided McNemar p is at least 0.0625 for any pairing (reached only if no question went from correct to wrong). We therefore call the lift directional, not significant. Against the same bare baseline, MA composite falls 3.53 pp (88.42% → 84.89%, a borderline fail of the −3 pp gate) and Overall falls 0.58 pp, which is why IterB is opt-in. Detail in the supplementary material (§S5.5.2).

**5.5.3 Mechanism-class distinction — parallel decomposition vs sequential refinement** Self-Ask (parallel decomposition) and ReAct (sequential refinement) are different mechanism classes and do not substitute for each other. Detail in the supplementary material (§S5.5.3).

**5.5.4 Knob matrix on Gemini-3-flash (detail in supplement)** The full per-backbone × per-knob matrix (5-batch, n=3,121) is in the supplementary material (§S5.5.4). On Gemini-3-flash, the arithmetic sum of the three standalone lifts (KG −0.01 pp + AC +0.81 pp + MQ +1.21 pp) is +2.01 pp, 24% of the +8.43 pp the same three knobs sum to on gpt-4.1-mini (§5.5.5). No knob combination was run, and each Gemini-3-flash 95% CI contains the 6.02% bare baseline.

<sup>79</sup>Press, Zhang, Min, Schmidt, Smith & Lewis, *Measuring and Narrowing the Compositionality Gap in Language Models*, 2022. arXiv:2210.03350. The decomposition implemented as IterC in §5.5.

**5.5.5 Single-stage knob backbone portability** Re-running the three principal standalone retrieval knobs (§5.1.8) on Gemini-3-flash gives these transfer rates:

| knob              | gpt-4.1-mini    | Gemini-3-flash  | transfer   |
|-------------------|-----------------|-----------------|------------|
| KG path           | +2.81 pp        | <b>−0.01 pp</b> | <b>0%</b>  |
| AC threshold=5    | +2.01 pp        | <b>+0.81 pp</b> | 40%        |
| MQ standalone     | +3.61 pp        | <b>+1.21 pp</b> | 34%        |
| <b>3-knob sum</b> | <b>+8.43 pp</b> | <b>+2.01 pp</b> | <b>24%</b> |

Generalization principle: any retrieval-stage lift of the form “knob X delivers +N pp on backbone Y” is backbone-conditional, and cross-backbone generalization requires explicit re-measurement. Setup, mechanism interpretation and CIs: supplement §S5.5.5.

**5.5.6 MQ across metric axes and backbones** MQ’s F\_MH lift attenuates across backbones (+3.61 → +1.21 pp). Its MA composite moves from −1.38 pp to +0.12 pp and MA\_U by +3.10 pp, but both Gemini-3-flash MA 95% CIs contain the bare baseline, so the data show that the gpt-4.1-mini MA regression did not recur; they do not show a measured MA gain. Per-knob evaluation on a single metric axis can therefore hide compensating effects on orthogonal dimensions. Full axis table: supplement §S5.5.6.

**5.5.7 Architectural composability vs mechanism composability** The IterB adapter contains explicit guards at three locations in `eval/evermembench/adapter_nox_mem.py` that short-circuit the standalone retrieval knobs when IterB is active, so IterB could not be combined with the Wave C knobs without a code change, whatever the mechanisms would do together. The earlier composability projection implicitly assumed architectural composability; the code evidence shows that assumption was false. Code excerpts and design rationale: supplement §S5.5.7.

**5.5.8 IterB + KG + rerank composition test — aborted, not completed** The IterB + KG + rerank composability test was dispatched on the production VPS on 2026-05-31 and aborted after 48 h of sustained CPU steal, measured at 51–97% while the ONNX cross-encoder rerank was active. The hypothesis was not tested, so there is no result, positive or negative; completing it requires dedicated CPU. Failure timeline, attempted mitigations and the preserved branch: supplement §S5.5.8.

## 5.6 Cross-bench validation — LongMemEval<sup>80</sup> (n=300)

Setup: Phase D production config (FTS5 + Gemini-embedding-001 + RRF, rerank OFF, top\_k=20), GPT-4.1-mini backbone, Gemini-2.5-flash judge, ora-

<sup>80</sup>Wu, Wang, Yu, Zhang, Chang & Yu, *LongMemEval: Benchmarking Chat Assistants on Long-Term Interactive Memory*, ICLR 2025. arXiv:2410.10813. The cross-bench validation set of §5.6.

cle session retrieval, stratified n=300 queries.

| Metric                                                                                                 | Score                                                                                         |
|--------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------|
| nDCG@10 (oracle retrieval, n=297 scored)                                                               | <b>1.0000</b> (session_hit@10 = 1.0000, Wilson 95% lower bound 0.9872)                        |
| Recall@10                                                                                              | 1.0000                                                                                        |
| Task accuracy (n=201 judged; 96 judge errors and 3 generator failures excluded; single batch, seed 42) | <b>68.16%</b> (Wilson 95% CI [0.6143, 0.7421]; gemini-2.5-flash judge; not 5-batch-validated) |

Per-category breakdown and fingerprint consistency with EverMemBench:

| Category                                 | LongMemEval score      | Strength | EverMemBench counterpart     |
|------------------------------------------|------------------------|----------|------------------------------|
| single-session-assistant                 | 87.10%                 | strong   | F_SH (strong)                |
| single-session-user                      | 86.67%                 | strong   | F_SH (strong)                |
| knowledge-update                         | 82.05%                 | strong   | MA_U (strong)                |
| abstention (_abs)                        | 82.61%                 | strong   | (no EverMemBench equivalent) |
| variant across the six categories, n=23) |                        |          |                              |
| multi-session                            | 55.81%                 | moderate | F_MH (weak)                  |
| temporal-reasoning                       | 54.76%                 | moderate | F_TP (weak)                  |
| single-session-preference                | 31.25% (n=16, wide CI) | weak     | (preference handling weak)   |

The per-category pattern is consistent with EverMemBench Phase D + H v2: strong on single-context factual recall, abstention and knowledge update; moderate on multi-session and temporal reasoning; weak on preference handling. Retrieval is at ceiling in every LongMemEval category here (nDCG@10 = Recall@10 = 1.0000), so these differences arise downstream of retrieval (in generation, judging or question difficulty) and cannot be attributed to the retrieval architecture. Both benchmarks also share the generator (gpt-4.1-mini) and the judge (gemini-2.5-flash), so they are not independent confirmations.

nDCG@10 rose from 0.9126 in an earlier oracle run (2026-05-19, n=100, before both the FTS5 query-sanitize fix and the Unicode-aware sanitize fix) to 1.0000 here (+8.74 pp, +9.6% relative, oracle ceiling). The two runs differ in sample size and in other pipeline changes made between them, so the rise is consistent with the fix but not attributed to it alone (eval/longmemeval/RESULTS-CROSSBENCH-2026-05-29.md).

Caveat: oracle session retrieval is an upper-ceiling measurement. Comparison to gbrain<sup>81</sup> (May 2026 run, v0.28.8: 97.6% any-hit R@5 on LongMemEval-S, rescored by its authors on 2026-08-31 to 83.40% recall\_all@5) requires the

<sup>81</sup>garrytan/gbrain-evals, *LongMemEval-S benchmark report*, docs/benchmarks/2026-05-07-longmemeval-s.md (first published May 2026; rescored 2026-08-31). github.com/garrytan/gbrain-evals. Cited in §5.6; a benchmark-report reference, so the repository path is the identifier.

`s_cleaned` non-oracle follow-up (not yet run). No competitor was run under this protocol, so §5.6 makes no comparative claim on task accuracy or abstention; it reports nox-mem’s per-category task accuracy and abstention handling as descriptive results.

---

## 5.7 Operational characteristics — latency, cost, and footprint (self-hosted)

The production boost stack (§4.1: section and source-type deltas summed, the source-type delta skipped for section-bearing chunks when the query names at most two entities, plus the additive salience delta) has been deployed since 2026-05-21. Beyond accuracy, we characterize the production deployment by its latency, cost, and memory footprint.

**Measurement host.** The operational measurements of this section were taken on a single 4-vCPU / 16 GB virtual private server (`RESULTS-PRODUCTION-SOTA.json` meta) under Debian Linux, serving six agents over a private overlay network; production has since moved to a 2-vCPU / ~8 GB host (§7.1 L3). Other hosting details are omitted. The latency percentiles and resident set sizes of this section were measured on that host and are relative to its hardware; the cost per query is not.

### 5.7.1 Latency — sub-10 ms KG path

| Path                                             | p50           | p95    | p99    | Notes                                                                                                                                                 |
|--------------------------------------------------|---------------|--------|--------|-------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>KG path</b><br>(entity-walk)                  | <b>2.5 ms</b> | 6.1 ms | 7.9 ms | SQL + regex over <code>kg_relations</code> , no LLM call; n=120, 2026-05-29, <code>benchmark/latency-cost/results/RESULTS-PRODUCTION-SOTA.json</code> |
| Hybrid search<br>(FTS5 + dense + RRF, no rerank) | 529 ms        | 698 ms | 744 ms | n=100 after a 10-query warm-up, 2026-05-29, same artifact (corpus 69,135 chunks); the Gemini query-embedding call dominates                           |

| Path                                            | p50                                 | p95       | p99       | Notes                                                                                                                                                                             |
|-------------------------------------------------|-------------------------------------|-----------|-----------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Hybrid search,<br>earlier run                   | ~940 ms                             | ~2,342 ms | ~2,523 ms | n=95,<br>2026-05-18,<br><a href="#">paper/publication/results/latency-benchmark-summary.json</a> ;<br>same path, a<br>different query<br>mix (see<br>below)                       |
| Hybrid +<br>cross-encoder<br>rerank<br>(MiniLM) | +3,674 ms p50<br>(1,109 → 4,783 ms) | 6,784 ms  | 8,696 ms  | Opt-in,<br>exploratory;<br>eval-harness<br>run on Ever-<br>MemBench<br>batch 004<br>( <a href="#">eval/evermembench/RESULTS-PHASEG.md</a> ),<br>not the<br>production<br>API path |

The KG path is in the sub-10 ms class at 2.5 ms p50 (n=120, table above). Among the systems compared here, none reports retrieval latency in this band, but the comparison is not like-for-like and we do not treat it as a measured contrast. Zep’s published ‘<100 ms’ latency claim (no percentile stated) is a vendor figure, not independently verified by us (§5.8.5), and is therefore never used as a measured operand. Mem0’s documentation states ‘<200 ms’ without a percentile, and MemOS publishes no latency figure; neither number comes from our harness. What we measured is the mechanism. The single-process embedded architecture of nox-mem (better-sqlite3 + sqlite-vec in-process) removes inter-service hops: the KG path makes no network call beyond the measured localhost HTTP round trip (estimated 1–3 ms), while the hybrid path still pays one remote query-embedding call (~400–600 ms). **Correction (v1.0.2).** Revisions before v1.0.2 quoted an unarchived 2026-06-15 re-check (KG path 2.9 ms, hybrid 653 ms p50); no figure here rests on it. The two archived hybrid runs (529 ms and ~940 ms p50, eleven days apart) used different query mixes, so their difference is not a drift measurement. The 2026-05-18 run is dominated by long decision, procedure, incident and architecture queries (per-category p50 ~900–1,100 ms). On the short and entity categories the gap between the runs stays within ~100 ms (short 578 vs 505 ms; entity 504 vs 603 ms), while the long-query category differs by ~390 ms (1,017 vs 624 ms p50), so query mix alone does not account for the whole difference. The Gemini query-embedding call dominates both runs; the KG path makes no such call.

### 5.7.2 Cost — \$0/query KG path; the managed-service column is modeled, not quoted

| Component                                       | nox-mem                                                           | Mem0 Cloud (modeled per-call rate; see text) |
|-------------------------------------------------|-------------------------------------------------------------------|----------------------------------------------|
| Retrieval API cost (KG path)                    | \$0.00                                                            | ~\$0.001/query (modeled)                     |
| Retrieval API cost (hybrid w/ Gemini embedding) | \$0.0000015/query                                                 | ~\$0.001/query (modeled)                     |
| Ingest API cost (per chunk)                     | \$0.00 for FTS5 indexing; the chunk embedding is billed per token | varies                                       |
| Total cost per 1M queries (hybrid)              | \$1.50                                                            | ~\$1,000 (modeled)                           |

The KG path costs \$0 per query because the entity walk uses only local SQL + regex with no LLM call (re-confirmed 2026-06-15). The hybrid path costs \$0.0000015/query (gemini-embedding-001 at \$0.15/1M input tokens as listed at the time of writing  $\times$  ~10 tokens/query; the 2026-05-29 artifact recorded \$0.13/1M, i.e. \$0.0000013/query). The Mem0 column is a modeling assumption. Its ~\$0.001/call rate is the reading recorded in `benchmark/latency-cost/results/RESULTS-PRODUCTION-SOTA.json` on 2026-05-29, and the Mem0 pricing page read on 2026-10-04 shows subscription plans (\$19–\$249/month above a free tier) and no per-call price, so the rate is not a quoted price. We draw no ratio from it.

### 5.7.3 Footprint — 399 MB RSS, single-process, self-hosted

| Scaling             | Idle RSS      | 10 $\times$ concurrent                                                | Notes                                      |
|---------------------|---------------|-----------------------------------------------------------------------|--------------------------------------------|
| nox-mem-api process | <b>399 MB</b> | 423 MB peak (+24 MB; the artifact’s separate delta field reads 15 MB) | better-sqlite3 + sqlite-vec single-process |
| Scaling pattern     | flat          | quasi-flat                                                            | No per-request memory blow-up              |

The process runs without database sidecars. The 399 MB idle footprint was measured on a 4-vCPU, 16 GB VPS; we have not measured it on a smaller tier, and we did not measure the other systems’ footprints (the author’s estimates are in supplement §S6.8).

## 5.8 Methodology, 5-batch protocol, and caveats

**5.8.1 5-batch + 95% CI canonical methodology** All EverMemBench claims in §5.1.5–§5.1.10 use the 5-batch canonical protocol:

- **5-batch set:** batches 004, 005, 010, 011, 016 (610–633 queries each, total  $n=3,121$ )
- **Aggregate metric:** mean across 5 batches per category

- **CI:** 95% confidence interval via t-distribution (n=5), reported as [lower, upper]
- **Claim threshold:** improvement claimed only when CI lower bound exceeds baseline mean (this ignores baseline variance; §5.5.2 adds a question-level test)

Single-batch gates were the prior protocol; they are now explicitly deprecated for any decision to enable a change by default.

### 5.8.2 Why single-batch gates can overstate effects (1.3–5.8× observed)

Phase G batch 004 reported F\_MH +8.00 pp (labelled “breakthrough”); the 5-batch reality was +1.61 pp, a 5× overstatement. Phase H v2 batch 004 reported +11.60 pp overall vs MemOS; the 5-batch reality was +9.13 pp (1.27× overstatement; the 5-batch margin remains positive). The root cause is per-batch variance in EverMemBench. Per-batch standard deviations of one arm’s level are large relative to the effects: in Phase G, F\_MH 2.30 pp, F\_HL 5.76 pp, MA\_C 6.23 pp, MA\_P 2.77 pp, MA\_U 5.46 pp (Phase H v2: MA\_U 9.54 pp). These describe dispersion of a level, not of the paired per-batch difference, so they indicate scale rather than a significance threshold. Batch 004 sat in the upper tail in both Phase G (+1.4 SD on F\_MH) and Phase H v2 (+1.7 SD on Overall); without the 5-batch protocol the single-batch figures would have overstated both effects. Single-batch gates also hide regressions: MA regressions were invisible in the Phase G batch-004 gate because the Phase D baseline it was compared against had no MA scores, so no MA delta could be computed; the −3 to −4 pp MA cost appeared only in the 5-batch re-run.

#### Overstatement rates observed:

| Phase      | Metric  | Single-batch | 5-batch  | Overstatement |
|------------|---------|--------------|----------|---------------|
| Phase G    | F_MH    | +8.00 pp     | +1.61 pp | 5×            |
| Phase G    | F_TP    | +11.67 pp    | +2.00 pp | 5.8×          |
| Phase H v2 | Overall | +11.60 pp    | +9.13 pp | 1.27×         |

**5.8.3 Memory Awareness in every report** Memory Awareness (MA\_C, MA\_P, MA\_U) is reported for every change that touches reranking, query routing or context assembly, because the Phase G single-batch gate missed a −3 to −4 pp MA regression when its baseline carried no MA scores (§5.8.2).

### 5.8.4 Scope of EverMemBench, LoCoMo, and classical-QA claims

- **EverMemBench Phase D headline (+2.95 pp vs MemOS)** compares nox-mem on Gemini-2.5-flash with MemOS on Gemini-3-Flash, so it is a cross-backbone comparison (−0.06 pp in Table 4’s aggregation, §5.1.10). The clearest same-backbone margin is the Memory Awareness composite on GPT-4.1-mini (73.34% vs 55.68%, +17.66 pp); on Gemini-3-Flash it narrows to +1.72 pp (§5.1.10).

- **EverMemBench Phase H v2 headline (+9.13 pp vs MemOS GPT-4.1-mini)** is CI-verified, but the absolute score (51.68%) is not high; MemOS itself is only 42.55%. GPT-4.1-mini has the weakest full-context baseline in Table 4 (37.44%); on it MemOS (+5.11 pp) and Zep (+2.52 pp) beat full context, while MemoBase (−3.18 pp) and Mem0 (−0.36 pp) do not.
- **EverMemBench Backbone Matrix (Gemini-3-flash):** against MemOS on the same backbone, +4.01 pp Overall (+4.50 pp in Table 4’s aggregation) and +1.72 pp MA composite, with that backbone’s full-context baseline (72.61%) above both systems. The +20.73 pp Overall / +32.74 pp MA composite figures compare against MemOS on GPT-4.1-mini and are cross-backbone; they decompose exactly into the GPT-4.1-mini same-backbone margin plus nox-mem’s own backbone gain (§5.1.10).
- **MuSiQue-Ans dev answer F1 58.62%** (§5.2.1) and **HotPotQA distractor dev answer F1 73.37%** (§5.2.2) sit above the specialized readers these datasets are conventionally compared against (IRCoT, EX(SA), the original HotpotQA baseline) and roughly 10–12 points below current published SOTA (Beam Retrieval), without specialized fine-tuning. Each question brings its own 10–20 candidate paragraphs, so these runs measure the reader over that set, not retrieval. This bounds the pipeline’s multi-hop composition as competent rather than state-of-the-art, and is the sense in which §5.4 uses it.
- **LoCoMo evidence hit@10 strict 74.52%** (§5.3) is the share of questions with at least one gold turn in the top 10, not an upper bound on token-F1 (and not comparable to Mem0’s published LLM-judge score (J) of 66.88%, a different metric); the token-F1 push of 51.85% is not ranked against published J scores, and no run measured a route to  $\geq 55\%$  (§5.3.3).
- **KG path, MAP, MQ, adaptive classifier, and IterC** are opt-in features, not defaults. Each addresses a known structural gap; combined effects of KG, MQ and MAP are measured in §5.1.9, and the adaptive classifier and IterC were not tested in combination with them.
- The Unicode-aware FTS5 sanitize fix is a prerequisite for all scores reported here; pre-fix numbers from the 2026-05-19 oracle run (nDCG@10 0.9126 LongMemEval) would have been reported as lower and should not be compared directly.
- **Knob portability (§5.5.5):** the single-stage knob lifts of §5.1.8 were measured on gpt-4.1-mini and hold for that backbone; on Gemini-3-flash they transfer at about 0–40%. A claim of a knob lift must name the backbone; §5.5.4’s measured matrix supersedes the earlier projection.
- **The earlier IterB composability projection (IterB + Wave C  $\rightarrow$  ~12.07% F\_MH) is superseded.** The projection assumed both backbone-portability (refuted by the portability study, §5.5.5) and architectural composability (refuted by adapter guard discovery in §5.5.7). A revised projection is ~8–9% F\_MH, untested (§5.5.8; table in supplement

§5.5.4).

- **IterB composition test (§5.5.8):** aborted on infrastructure, so the hypothesis is untested; batch 004 (n=49) is preserved but not 5-batch valid.

### 5.8.5 Limitations and open work

- **EverMemBench F\_MH absolute gap vs MemOS in Table 4** (3.21% vs 18.88% on GPT-4.1-mini; 6.02% vs 10.84% on Gemini-3-Flash) remains. The §5.4 reframing argues it is not primarily a multi-hop reasoning failure and names task setup as the leading, not established, account. The track’s difficulty is visible on the same metric in the 18.88% (LLM-judged) that the best published memory-augmented system reaches on it, rather than inferred from the classical benchmarks. IterB (§5.5.2) and a backbone change (§5.1.10) each move it by a few points; knob combinations plateau near 7–8%, sub-additively (§5.1.9).
- **LoCoMo end-to-end accuracy** remains open: nox-mem’s 51.85% token-F1 has no same-metric published comparator (Mem0’s 66.88% is J). The EverMemBench composability results (§5.1.9) give no indication that stacking retrieval-stage knobs would close this gap; composition orchestration (§5.5) is the open path.
- **Zep’s published ‘<100 ms’ latency figure** is a vendor figure without a stated percentile, not independently verified. nox-mem KG path p50 = 2.5 ms (§5.7) is measured on production VPS with the harness instrumented end-to-end.
- **GPT-5 / Claude columns** are blocked by API key constraints in the current eval setup. Backbone Matrix is currently three-cell (Gemini-2.5-flash, GPT-4.1-mini, Gemini-3-flash); GPT-5 and Claude entries can be added if access opens.
- **Standalone-knob portability to other backbones** beyond Gemini-3-flash is unverified. The 0–40% transfer pattern (24% in aggregate) of §5.5.5 is based on three knobs on one backbone pair. Additional backbone pairs (Claude Sonnet 4.6, GPT-5, Gemini 4) require independent re-baseline before composability projections can be made.
- **EverMind-AI / EverMemBench reference baselines** rely on MemOS Table 4 published numbers (arXiv:2602.01313v3); we have not re-run MemOS internally on the canonical 5-batch subset, and the five batches’ category counts differ from the paper’s filtered set (2,733 vs 2,400 questions, §5.1.10).

## 6. Cross-System Comparison (Pre-specified)

### 6.1 Methodology summary

§6 covers the cross-system comparison between nox-mem and five competing persistent memory systems for AI agents. The comparison is pre-specified: its execution plan, `specs/2026-05-23-Q4-comparison-execution-plan.md`, was

first committed to the public repository on 2026-05-21, before the first run (a capped smoke on 2026-05-23, §6.6); it was not registered with an external registry (§6.7). The comparison principles (§6.5), anti-cherry-pick statement (§6.6) and plan provenance (§6.7) were fixed before execution; the tables in §6.2/§6.3/§6.4 and the later-run subsections §6.3.1–§6.3.4 were written after execution, and §6.5 carries dated annotations of what the runs could not meet. The pre-specified success criterion (§6.7) requires nox-mem to rank among the top three on at least two of the four key metrics (nDCG@10, R@10, MRR, latency).

## 6.2 Competitors

The five competitors were selected by prioritizing GitHub stars, recent commit activity, and functional overlap with the nox-mem scope. Versions in the table are those recorded for each run; the pre-specified plan named version ranges (“Latest stable”, “v0.27+”, “Latest”) rather than exact pins, and the rc4 Mem0 arm ran under mem0ai 2.0.10 (§6.3.2 (a)).

| System            | Repo                 | Install path                                    | Version pinned                                                                                       | Default config                                                                                      |
|-------------------|----------------------|-------------------------------------------------|------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------|
| Mem0              | mem0ai/mem0          | <code>pip install mem0ai==0.1.114</code>        | 0.1.114                                                                                              | OpenAI text-embedding-3-small 1536d + faiss (Chroma swapped to avoid telemetry thread-leak, §6.9)   |
| Zep               | getzep/zep           | Docker compose (zep + postgres)                 | 0.27.2; run 2026-09-10 on a host with a real Docker daemon, not in the 2026-06-15 canonical (§6.3.4) | Local self-host mode; text-embedding-3-small 1536d, no reranker                                     |
| Letta (ex-MemGPT) | letta-ai/letta       | <code>pip install letta (+ click&lt;8.2)</code> | 0.6.6                                                                                                | SQLite backend; agent-OS retrieval                                                                  |
| agentmemory       | rohitg00/agentmemory | daemon (REST :3111)                             | as-of 2026-06-15                                                                                     | union store + namespace filter                                                                      |
| EverMind-AI       | EverOS (PyPI)        | <code>pip install everos</code>                 | 1.3.1; run 2026-09-10, not in the 2026-06-15 canonical (§6.3.3)                                      | local-first library; gemini-embedding-001 3072d + Qwen/Qwen3-Reranker-4B rerank (mandatory, §6.3.3) |

Each system runs with its publicly documented default configuration (principle 3 of §6.5), so no competitor is adversarially tuned to underperform.

### 6.3 Per-system per-dataset results

This is the canonical cross-system  $\times$  cross-dataset table. The K cutoff is fixed at 10 across all systems; latency is measured externally (wall clock around the adapter call); cost is derived from per-system logs (API calls  $\times$  published pricing).

**Preliminary smoke (2026-05-24), superseded.** An initial 20-query smoke on an eval-isolated DB validated the harness end-to-end (nox-mem, full corpus: nDCG@10 = 0.4509 as rescored by `aggregate.py` from `output/nox_mem.json` (an earlier reported 0.6380 is not reproducible from that artifact), gold-hit 13/20; Mem0 at a 500-chunk cost-control cap, not directly comparable). Its only role was pipeline validation. The canonical run (below) and rc4 (§6.3.2) supersede it, and the capped-corpus “concentration vs coverage” reading no longer bears on the reported results. The per-dataset breakdown below is from the canonical run (2026-06-15); rc4 (§6.3.2) extends it to the full n=2,482 set.

**Canonical run, 2026-06-15 (dedicated RunPod pod, n=100/dataset, same-namespace fair; its per-query outputs were not retained when the pod was terminated, so the figures below are carried from the run’s report and cannot be rescored from this repository).** Sustained CPU steal on the shared production host had aborted another long benchmark (the IterB composition test, §5.5.8; §7.1 L5), so the canonical cross-system run was executed on a dedicated pod on 2026-06-15. **Two of the five competitors produced head-to-head quality numbers** (Mem0, agentmemory; nox-mem is the system under test, not a competitor); **the other three are documented deployment non-runs** with explicit reasons (Zep, Letta, EverMind-AI; §6.3.1). Protocol: n=100 queries per dataset, k=10; retrieval quality (nDCG@10 / recall@10 / MRR) under each system’s native default embedding provider; same-namespace fair re-query, in which the union store is queried at k=30 and filtered to the active dataset namespace before re-scoring the top-10, a step that removes the cross-dataset distractor confound that inflated an early union-store reading (14.6% of nox-mem’s raw top-10 were off-dataset chunks). Per-system latency percentiles were not captured uniformly in this run; nox-mem operational latency is reported independently in §5.7 (KG path 2.5 ms p50; hybrid 529 and ~940 ms p50 in two archived runs).

#### LongMemEval n=100 (canonical):

| System         | nDCG@10       | R@10   | MRR           | Cost/query                                                    | Coverage /<br>config                                   |
|----------------|---------------|--------|---------------|---------------------------------------------------------------|--------------------------------------------------------|
| <b>nox-mem</b> | <b>0.5234</b> | 0.6535 | <b>0.5494</b> | ~\$0.0000015<br>(one Gemini<br>query<br>embedding,<br>§5.7.2) | 100%;<br>hybrid<br>FTS5 +<br>Gemini-<br>3072d +<br>RRF |

| System          | nDCG@10                                                                                                                                                                                     | R@10   | MRR    | Cost/query                                                     | Coverage /<br>config                                                         |
|-----------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------|--------|----------------------------------------------------------------|------------------------------------------------------------------------------|
| Mem0            | 0.4764                                                                                                                                                                                      | 0.6372 | 0.5107 | OpenAI embed<br>per query (OSS<br>library, no<br>subscription) | 100%; faiss<br>backend,<br>OpenAI<br>text-<br>embedding-<br>3-small<br>1536d |
| agentmemory     | 0.2803                                                                                                                                                                                      | n/c    | n/c    | \$0 (local)                                                    | daemon<br>REST;<br>weak<br>retriever                                         |
| Zep             | [not in this<br>run - Docker<br>impossible on<br>the<br>unprivileged<br>pod (§6.3.1);<br>measured<br>2026-09-10<br>over the full<br>n=2,482 set<br>on a Docker-<br>capable host,<br>§6.3.4] | —      | —      | —                                                              | —                                                                            |
| Letta           | [GAP - see<br>§6.3.1:<br>agent-OS<br>latency ~16<br>min/query;<br>impractical<br>for n=100]                                                                                                 | —      | —      | —                                                              | —                                                                            |
| EverMind-<br>AI | [not in this<br>run - barrier<br>of §6.3.1<br>lifted later;<br>measured<br>2026-09-10<br>over the full<br>n=2,482 set,<br>§6.3.3]                                                           | —      | —      | —                                                              | —                                                                            |

#### LoCoMo n=100 (canonical):

| System | nDCG@10       | R@10   | MRR    | Cost/query                                                     | Coverage /<br>config                                      |
|--------|---------------|--------|--------|----------------------------------------------------------------|-----------------------------------------------------------|
| Mem0   | <b>0.4686</b> | 0.6171 | 0.4656 | OpenAI embed<br>per query (OSS<br>library, no<br>subscription) | ~95%<br>(thread-<br>leak ingest<br>loss); faiss,<br>1536d |

| System                              | nDCG@10                                                                                                                     | R@10   | MRR    | Cost/query                                                    | Coverage /<br>config                                   |
|-------------------------------------|-----------------------------------------------------------------------------------------------------------------------------|--------|--------|---------------------------------------------------------------|--------------------------------------------------------|
| <b>nox-mem</b>                      | 0.4263                                                                                                                      | 0.5504 | 0.4464 | ~\$0.0000015<br>(one Gemini<br>query<br>embedding,<br>§5.7.2) | 100%;<br>hybrid<br>FTS5 +<br>Gemini-<br>3072d +<br>RRF |
| agentmemory                         | 0.1587                                                                                                                      | n/c    | n/c    | \$0 (local)                                                   | daemon<br>REST;<br>weak<br>retriever                   |
| Zep / Letta<br>/<br>EverMind-<br>AI | [GAP of this<br>run - see<br>§6.3.1; Zep<br>and<br>EverMind-AI<br>have since<br>produced<br>numbers<br>(§6.3.3,<br>§6.3.4)] | —      | —      | —                                                             | —                                                      |

**Split result (mandatory per §6.6).** Each top-tier system wins one benchmark: nox-mem wins LongMemEval (+0.047 nDCG@10, +0.039 MRR) and Mem0 wins LoCoMo (+0.042 nDCG@10). agentmemory is a distant third on both, reflecting a weaker retriever. Neither system dominates. The nox-mem LoCoMo figure rose from 0.4173 (raw union store) to 0.4263 after the namespace fix. Mem0 still wins, so the namespace confound does not account for the LoCoMo gap in this as-configured setting (whether it survives embedder matching is tested in §6.3.2, where the ordering reverses). Per §6.6, both datasets are reported side by side; we do not cherry-pick the one nox-mem wins.

**Caveats (per-system composition, declared per §6.5).** (a) Embedding providers differ by native default: nox-mem uses Gemini 3072d, Mem0 OpenAI 1536d, and agentmemory its own default. This is the “as-configured” fair-comparison stance (§6.5, principle 5); the controlled-embedding experiment that equalizes this confound is reported in §6.3.2 (rc4, all-Gemini) and inverts the LoCoMo result. (b) Store composition differs: nox-mem and agentmemory were queried from a union store with namespace filtering, while Mem0 used single-namespace stores (its LoCoMo store ingest leaked threads and lost ~5% of vectors; §6.9). (c) Coverage: nox-mem 100% both datasets; Mem0 100% LongMemEval / ~95% LoCoMo; agentmemory full. These differences are reported here, consistent with §6.6.

### 6.3.1 Documented gaps — one system without a benchmark number

Per §6.6, systems that fail setup receive an explicit reason rather than silent omission. The 2026-06-15 run hit three:

- **Zep** — requires a Docker stack (Zep + Postgres). Docker is impossible on the unprivileged benchmark pod: `dockerd` fails on `iptables ... Permission denied` (no `NET_ADMIN`), and every documented workaround (`--iptables=false --bridge=none, --storage-driver=vfs`) fails on kernel-namespace syscalls blocked by the pod seccomp profile. The cause is a privilege constraint rather than a configuration error; Zep requires a host with real Docker (privileged VM or its hosted Cloud).

Zep is a non-run of the 2026-06-15 canonical run specifically. It did run earlier, and the artifact is in this repository: `eval/q4-comparison/output/zep.json` (2026-05-25, `zep-python==1.5.0 + ghcr.io/getzep/zep:0.27.2` OSS via Docker) holds 20 queries, 20 with results, 0 errors. They come from the superseded 2026-05-24/25 smoke series (§6.3, “Preliminary smoke”; Zep ingested the full corpus there, the 500-chunk cap applied to Mem0 only), run on a host that did have Docker rather than on the benchmark pod. The smoke is not comparable to the canonical run (n=20 vs n=100/dataset, no same-namespace re-query), so its numbers are not used in §6.3. A comparable Zep run now exists (2026-09-10, full n=2,482, §6.3.4) and lives at a different path, `output-2026-09-10/`; `output/zep.json` is left where it is because this paper cites it.

- **Letta (ex-MemGPT)** — installs and runs natively (CLI unblocked via `click<8.2`, server starts without Docker), but is an agent-OS: retrieval routes through a full LLM agent turn (`archival_memory_search` tool call), reported at ~16 min/query (indicative: no archived artifact holds this timing), so n=100 × 2 datasets would take days. Ingest also degrades catastrophically (stalls at ~94% after ~12 h on a sequential archival-memory insert). Letta *validates* (it runs) but is impractical for a 100-query benchmark in this configuration.
- **EverMind-AI / EverOS** — at the 2026-06-15 canonical run it ran only as an HTTP service and required two third-party credentials not available to that run (OpenRouter for LLM extraction + DeepInfra for embedding/rerank), plus authorization for an external-repo `pip install -e`; it was out of scope for that time-box.

**Zep was a gap of this list and no longer is.** The barrier lay in the host rather than the software: `dockerd` cannot start on the unprivileged pod. On a host with a real Docker daemon and a paid OpenAI key, the same OSS stack ran the full n=2,482 canonical set on 2026-09-10 with zero errors, reported in §6.3.4. The bullet above describes the 2026-06-15 environment.

**EverMind-AI / EverOS was a gap of this list and no longer is.** The barrier in the bullet above was lifted upstream: `pip install everos` now resolves to a local-first library (v1.3.1) with no mandatory OpenRouter credential. It was run on 2026-09-10 over the full n=2,482 set and produced a number, reported with its confounds in §6.3.3. The entry above describes the version that

existed on 2026-06-15.

**What is measured of EverOS, and what is not (2026-09-10).** Credentials were obtained after the canonical run, and the corpus was ingested through EverOS’s in-process `service.knowledge` API: 6,822 documents retained, measured on the searchable index after the final sync, from 6,830 offered lines carrying 6,822 distinct ids. Four of the 6,830 attempts failed, all four with `DuplicateDocumentError` on the eight ids that name two distinct documents each; no id is absent from the index and none of the four is the gold document of any query (predicate: `intersect(corpus_ids - indexed_ids, union(gold_chunk_ids)) = empty`).

One configuration fact of that service bounds any future retrieval number and is recorded here because it was measured: EverOS reranks with a `rerank_n` of 50 while exposing a parameter named `top_k_cap = 100`, and every query in the sweep returns exactly 50 hits under `method=hybrid`. The hit count saturates at the ceiling on 100% of queries and never reaches the value the parameter’s name advertises. Any `nDCG@10` from that service is therefore computed over a candidate set truncated at 50, and the name of the larger parameter overstates the limit that binds. That sweep has since closed, and its number is reported in §6.3.3.

LightRAG and HippoRAG2 are not §6 competitors. They are §5 KG/RAG references whose LLM-per-chunk graph-build cost (LightRAG warns >6 h on default) places them outside the memory-system comparison; they are deferred as optional baselines.

**6.3.2 Embedding-matched variant (rc4 — all-Gemini, full eval)** The §6.3 split is reported under each system’s *native default* embedder (the “as-configured” stance, §6.5, principle 5). Because nox-mem defaults to Gemini 3072d and Mem0 to OpenAI 1536d, the most obvious confound on the split is the embedding provider itself. To probe it we ran an embedding-matched variant (rc4) in which both systems use the same embedder: `gemini-embedding-001` at 3072d (the model and dimensionality nox-mem runs in production), with Mem0 re-pointed to it via an external config patch (`lib/all_gemini_config`). The swap was not zero-edit: the Mem0 adapter also required a one-time API-compatibility fix for the mem0 2.x `search/get_all` signatures (declared as confound (a) below). The variant also replaces the `n=100` sample with the full evaluation set (`n=2,482`: 1,982 LoCoMo + 500 LongMemEval) over the full 6,822-chunk corpus, scored identically to §6.3 (binary-relevance `nDCG@10`, `k=10`, same gold encoding). The evaluation gold is fully covered: every gold chunk exists in the corpus on both datasets. That is *corpus/gold* coverage, which differs from Mem0 *store* coverage (Mem0’s ingest log records 4 transient 503 errors; those 4 chunks were absent from its store while the run queried it and were back-filled only after it finished; confound (e) below). The comparison is embedding-matched (same model + dimensionality) and is not an all-else-equal controlled experiment: four residual confounds are declared below, and a

fifth, the embedding task-type asymmetry, was probed by a dedicated ablation, under which nox-mem’s lead persisted.

| System<br>(all-Gemini<br>3072d)                   | LongMemEval<br>nDCG@10 (n=500) | LoCoMo nDCG@10<br>(n=1,982) | Overall (n=2,482)     |
|---------------------------------------------------|--------------------------------|-----------------------------|-----------------------|
| <b>nox-mem</b><br>(hybrid FTS5 +<br>Gemini + RRF) | <b>0.5255 ± 0.033</b>          | <b>0.4952 ± 0.017</b>       | <b>0.5013 ± 0.015</b> |
| Mem0 (Gemini<br>embedder,<br>Chroma)              | 0.4030 ± 0.035                 | 0.4407 ± 0.017              | 0.4331 ± 0.016        |

Intervals are 95% confidence (mean per-query nDCG@10  $\pm 1.96 \cdot \text{SEM}$  (standard error of the mean), binary relevance). The nox-mem / Mem0 intervals are disjoint on both datasets and overall, so the gap is not attributable to query-sampling noise (it remains subject to the four residual configuration confounds below; with the one asymmetry that favors nox-mem by design, task type, removed from both arms in an ablation, the lead persisted).

**Result: the split does not survive embedding matching.** With the embedding model and dimensionality equalized (the four residual confounds below declared; with a fifth, task type, removed in an ablation, the lead persisted), nox-mem outperforms Mem0 on both benchmarks (LongMemEval +0.123, LoCoMo +0.055 nDCG@10) and on all five represented query categories (§6.4). The ordering on LoCoMo therefore reverses when the embedder is matched. This does not attribute the §6.3 Mem0 win to the embedder, nor the rc4 nox-mem lead to the retrieval architecture: the run also changed the Mem0 version, its vector backend, the query sample and corpus retention (confounds (a)–(c) and (e) below), and nothing here separates their effects.

**Residual confounds, declared per §6.6: embedding matching is not a clean architecture isolation.** Four factors remain that prevent attributing the inversion purely to the embedding; the last of them, (e), is quantified below, and its net effect on the comparison is not isolated (a further one, task-type asymmetry, was probed by ablation, and the lead persisted; see below). **(a) Mem0 version, established from the artifacts together with the parameterisation.** `meta.version = "mem0ai==0.1.114"` in `output/rc4/mem0.json` is the adapter’s declared `VERSION_PIN`, which records intent rather than a runtime read, so table §6.2 and the artifact are a single source. The run persisted its version twice elsewhere, in strings exclusive to the 2.x line: `output/rc4-run.log` carries mem0’s own warning that “*the ‘chroma’ vector store does not support keyword search*”, and `.mem0-chroma-rc4/chroma.sqlite3` carries `text_lemmatized` on 6,830 of 6,830 metadata rows. Neither string occurs anywhere in the released 0.1.114 source distribution; we checked this with positive controls (`def search` appears in 23 of its files, `user_id` in 21). The run executed under mem0ai 2.0.10; under 2.0.10 a

0.1.x-shaped `search(query=..., user_id=..., limit=k)` is not absorbed by `**kwargs`: 2.0.10 calls `_reject_top_level_entity_params` on entry and raises `ValueError: Top-level entity parameters frozenset({'user_id'}) are not supported in search()`. The adapter wraps the call in `except Exception → RuntimeError` and `runner.py` counts every such exception into `n_errors`; the artifact carries `error: null` on all 2,482 queries and `n_errors: 0`. The clean exit is therefore a discriminator: under the 0.1.x form all 2,482 entries would carry a `RuntimeError`. The run issued the 2.x form, at the intended parameterisation: `filters={"user_id": "q4-eval"}, top_k=10, threshold=0.0`. No re-execution figure is reported: a replay run during revision did not retain its script or per-query output. Any future run must persist `validate()`'s `mem0.__version__` into `meta`; here the version was recoverable from a log line and a metadata column by luck rather than by design. **(b) Vector backend:** canonical Mem0 used a faiss store; rc4 Mem0 uses a fresh Chroma collection (required to avoid a dimension clash with the 1536d OpenAI run), so the backend changed along with the embedder. **(c) Sample scope:** §6.3 sampled `n=100/dataset`; rc4 scores the full `n=2,482`: a larger and differently distributed query set rather than a like-for-like re-score of the same 100. rc4 is therefore best read as “*same embedding model and dimensionality, Mem0 at the mem0ai 2.0.10 it is now established to have run under (a), and at its default backend*” → nox-mem leads. It is not a surgical architecture-only isolation. (The 4 chunks missing from Mem0's store at query time affect one query, whose gold chunk Mem0 fails to retrieve in top-10 even when present; we verified zero aggregate effect.) **(e) Corpus retention differed by adapter.** The offered corpus is 6,830 documents carrying 6,822 distinct ids: 8 pairs of documents with *different text* share an id. nox-mem's eval loader inserts with `INSERT OR IGNORE`, so it retained 6,822 and silently kept only the first document of each colliding pair; Mem0's Chroma collection held 6,826 rows (6,818 distinct ids) when the run queried it, keeping both documents of every colliding pair; the 4 chunks lost to transient 503s were back-filled at 15:02 UTC, after the run finished at 14:57:04 UTC, which is why the preserved store now counts 6,830 rows. Measured 2026-09-10 on the run's own artifacts: `cache/rc4-nox-hybrid.db eval_chunks = 6,822; .mem0-chroma-rc4/chroma.sqlite3 embeddings = 6,830` with 6,822 distinct `chunk_id`. Scoring is by id, and each retrieved id counts once, at the rank of its first occurrence. Before v1.0.6 the scorer counted a repeated id at each occurrence: in 3 LongMemEval queries Mem0 returned both documents that share a gold id, which overstated Mem0 and moved nine printed values of §6.3.2–§6.4 (Mem0 scores, one interval, and the margins against nox-mem) by at most 0.004; no order or sign changed. 10 of 2,482 queries (0.40%) carry a gold id in a colliding pair, and on those nox-mem held one candidate text where Mem0 held two. The net effect of this retention difference on retrieval quality is not isolated: holding both documents of a pair may let Mem0 cover a gold passage that nox-mem's single copy misses, while a repeated id spends a slot of Mem0's top 10 (20 of its 2,482 lists repeat an id, 17 of them only a non-gold one), and a shared id does not distinguish the relevance

of the two texts behind it. Which document of a pair survives INSERT OR IGNORE is deterministic but arbitrary, since file order decides it. One external datum bears on how to read this, from a **different run** (the 2026-09-10 EverOS ingest of §6.3.1, not rc4, and therefore not a fourth column here): offered the same corpus, EverOS also retained **6,822**, reaching that number by rejecting four of the eight duplicate second writes with an explicit error and accepting the other four without one (an asymmetry we do not explain). Two systems, two mechanisms, one count, which makes 6,822 the consequence of honouring id uniqueness rather than an idiosyncrasy of our loader. It does not remove the rc4 asymmetry: Mem0 held 6,826 rows at query time against nox-mem’s 6,822. The stripped per-query outputs are in the evidence dataset 10.5281/zenodo.23146656, with a verifier that checks 132 Section 6 values and statements against the supplied outputs, metadata, logs and store extracts, plus three arithmetic differences of printed figures; twelve groups of Section 6 figures cannot be recomputed from it and are listed there with the reason; the run is reproducible from `eval/q4-comparison/runner_rc4.py` and `eval/q4-comparison/run_rc4_full.sh`.

**Task-type ablation: confound (d) probed; the lead persisted.** The one configuration asymmetry that *avored* nox-mem in rc4 was the embedding task type: nox-mem passes Gemini’s RETRIEVAL\_DOCUMENT/RETRIEVAL\_QUERY task types (retrieval-optimized embeddings), while Mem0’s embedder call does not. To probe it we re-ran nox-mem with a generic Gemini embedding (NOX\_EMBED\_GENERIC\_TASKTYPE=1: no task type, exactly how Mem0 calls the same model) against the same Mem0 baseline, over the full n=2,482 in a symmetric “*neither system sets a task type*” comparison holding confounds (a)–(c) constant. nox-mem’s overall nDCG@10 falls only 0.5013 → 0.4979 (−0.34 pp) and still outperforms Mem0 (0.4331) on overall, both datasets (LoCoMo 0.4920 vs 0.4407; LongMemEval 0.5215 vs 0.4030), and all five categories. With task types removed from both arms, nox-mem’s lead persisted. The 0.34 pp difference does not isolate or bound the task-type contribution, because the generic run differed in more than the task type: the re-ingested generic corpus reached 99.03% gold coverage, with 23 of 2,370 distinct gold chunks absent against 100% in the task-type run, and the run log records 33 dense-search errors (504) at query time, on each of which the adapter fused the FTS5 list alone for that pass. Missing documents and a lost dense pass can raise some rankings as well as lower others. With confounds (a)–(c) and (e) still in place, the ablation does not identify which difference produces the lead. The ablation is reproducible from `eval/q4-comparison/run_rc4_ablation.sh`.

**LoCoMo claim taxonomy (orientation).** “Who wins LoCoMo” depends entirely on metric and configuration; the paper reports four distinct, non-contradictory readings, tabulated here to prevent conflation:

| §      | Metric                   | Configuration                                                       | n     | Result                                                                                                                            |
|--------|--------------------------|---------------------------------------------------------------------|-------|-----------------------------------------------------------------------------------------------------------------------------------|
| §5.3.1 | retrieval@10<br>(strict) | nox-mem<br>standalone                                               | —     | nox-mem<br>74.52% (vs<br>Mem0's<br><i>published</i><br>LLM-judge<br>score J 66.88%<br>— different<br>metric, not<br>head-to-head) |
| §5.3.2 | token-F1                 | nox-mem<br>standalone                                               | —     | nox-mem<br>51.85%, not<br>ranked (Mem0's<br><i>published</i><br>66.88% is J, a<br>different metric,<br>§5.3.2)                    |
| §6.3   | nDCG@10                  | native<br>embedders (nox<br>Gemini-3072d /<br>Mem0<br>OpenAI-1536d) | 100   | <b>Mem0</b> 0.4686<br>vs nox-mem<br>0.4263                                                                                        |
| §6.3.2 | nDCG@10                  | matched<br>embedder (both<br>Gemini-3072d)                          | 2,482 | <b>nox-mem</b><br>0.4952 vs Mem0<br>0.4407                                                                                        |

The §6.3 → §6.3.2 reversal occurs under embedding matching, together with the four residual confounds above, so it is not attributed to the embedder alone (with the task-type asymmetry removed in an ablation, the lead persisted); the §5.3 retrieval-vs-F1 contrast is an orthogonal metric distinction, not a contradiction.

**6.3.3 EverOS measured (2026-09-10) — a competitor that outperforms nox-mem, and the pipeline confound that qualifies it** The EverMind-AI gap of §6.3.1 closed upstream. `pip install everos` resolves to a local-first library (v1.3.1) that needs no HTTP service and no mandatory OpenRouter credential, and it was run over the same corpus and the same  $n = 2,482$  query set as rc4 (§6.3.2), scored by the same aggregator under the same binary-relevance nDCG@10 at  $k = 10$ .

| System                          | Overall<br>nDCG@10<br>(n=2,482) | LoCoMo<br>(n=1,982) | LongMemEval<br>(n=500) | R@10   | MRR    | p50 latency                                  |
|---------------------------------|---------------------------------|---------------------|------------------------|--------|--------|----------------------------------------------|
| <b>EverOS</b>                   | <b>0.6455</b>                   | <b>0.6585</b>       | <b>0.5942</b>          | 0.7629 | 0.6403 | 1,592 ms                                     |
| <b>1.3.1</b><br>(2026-09-10)    |                                 |                     |                        |        |        |                                              |
| nox-mem<br>(rc4,<br>2026-06-29) | 0.5013                          | 0.4952              | 0.5255                 | 0.6656 | 0.4749 | 515.6 ms,<br>not<br>transport-<br>normalized |

| System                           | Overall<br>nDCG@10<br>(n=2,482) | LoCoMo<br>(n=1,982) | LongMemEval<br>(n=500) | R@10   | MRR    | p50 latency                                  |
|----------------------------------|---------------------------------|---------------------|------------------------|--------|--------|----------------------------------------------|
| Mem0<br>(rc4,<br>2026-06-<br>29) | 0.4331                          | 0.4407              | 0.4030                 | 0.5842 | 0.4092 | 349.8 ms,<br>not<br>transport-<br>normalized |

EverOS outperforms nox-mem here on both datasets (+0.163 LoCoMo, +0.069 LongMemEval, +0.144 overall), across 2,482 queries with zero errors.

**The confound, and what we do not claim about it.** The two pipelines differ in shape, and the difference is the cross-encoder re-ranking stage<sup>82</sup>. EverOS requires a cross-encoder: `_require_search_providers()` in `everos/service/knowledge.py` raises `ProviderNotConfiguredError` before any search path when a reranker is absent, so `Qwen/Qwen3-Reranker-4B` is its minimum viable configuration, not a generous setting we chose for it. nox-mem’s rc4 run has no cross-encoder stage at all: what its adapter calls rerank is RRF re-ordering plus a 1-hop KG-neighbourhood multiplier. This is a property of the systems compared rather than an experimenter’s choice, and its magnitude is not measured in this benchmark. The only measurement we hold of a cross-encoder’s contribution inside nox-mem is §5.1.7:  $-0.96$  pp overall,  $+1.61$  pp on multi-hop,  $-2.80$  to  $-4.00$  pp on Memory Awareness. That study used MiniLM-L-6-v2, 22 M parameters and roughly two orders of magnitude smaller than a 4 B reranker, on a different benchmark, under different metrics, through a different code path. It does not transfer, and we do not offer it as a rebuttal. We record it because it is the only evidence we hold on this term; at a 180-fold parameter gap it barely serves as a floor. What it supports is the negative claim: we do not know the magnitude, and nothing in this paper licenses attributing the 0.144 gap to the reranker, in whole or in part. Settling the question requires adding the stage to the nox-mem adapter and re-running the same corpus (§7.2); the stage does not exist in this harness, so no flag can switch it on.

**Three further asymmetries, declared per §6.5.** (a) *Run date*: rc4 ran 2026-06-29, EverOS 2026-09-10; corpus, query set, embedding model and dimensionality (`gemini-embedding-001`, 3072 d) are the same, but the dates differ. (b) *Operational axis*: EverOS’s p50 (1,592 ms, this harness) is above both archived standalone nox-mem hybrid runs (529 and  $\sim 940$  ms p50, §5.7; a different harness, so indicative only), and every EverOS query is paid on two providers (DeepInfra rerank + Gemini), against \$0 on the nox-mem KG path

<sup>82</sup>Nogueira & Cho, *Passage Re-ranking with BERT*, 2019. arXiv:1901.04085. The work that established the cross-encoder as a re-ranking stage over a first-stage candidate pool: the MS MARCO result cited in §7.2 F5, and the stage whose **presence or absence** is the confound of §6.3.3–§6.3.4 — EverOS requires one, nox-mem fuses by RRF[`~rrf`] without one, Zep has neither. Our own measurement of the stage (§5.1.7,  $-0.96$  pp) used a 22 M distillation[`~minilm`],  $\sim 180\times$  smaller than the 4 B model EverOS runs[`~qwen3embed`], so it is not carried over as a bound.

and one embedding call on its hybrid path (§5.7.2). (c) *Configuration provenance*: the artifact’s `meta` records what was asked of the harness and nothing about model, dimensionality or reranker; the configuration named above was captured from the live process’s environment before it exited. That omission is recorded as a debt, not reconstructed after the fact.

**How this number relates to the vendor’s own.** EverMind-AI reports EverMemOS<sup>83</sup> at 93.05% on LoCoMo and 83.00% on LongMemEval. Those are LLM-judged answer accuracy; ours is retrieval nDCG@10 over a fixed corpus, so the two are not the same quantity and 0.6455 neither confirms nor contradicts them. Two structural points still apply, and they cut in opposite directions. Against the vendor: their headline numbers are self-reported, and the benchmark on which several of them are obtained, EverMemBench<sup>84</sup>, is authored by the same group. A reviewer of §6 should hold our own EverMemBench numbers (§5.1) to the same standard, which is why §6.6 forbids us from reporting a benchmark we win without the one we lose. In the vendor’s favour: the measurement above is, as far as we can establish, an independent third-party run of their system, and it places EverOS first.

Artifact: `eval/q4-comparison/output-2026-09-10/_aggregate.json`  
(`everos==1.3.1`, `n_queries 2,482`, `n_errors 0`).

**6.3.4 Zep measured (2026-09-10) — the last Docker gap closed; Zep ranks third** The Zep gap of §6.3.1 came from the host, not from Zep’s software: `dockerd` could not start on the unprivileged benchmark pod. Given a host with a real Docker daemon and a paid OpenAI key, the same OSS stack of the 2026-05-25 smoke (`zep-python==1.5.0 + ghcr.io/getzep/zep:0.27.2`, Zep + Postgres/pgvector) ran the full canonical set, `n = 2,482`, with zero errors, in 4 h 13 min.

---

<sup>83</sup>Hu, Gao, Zhou, Xu, Bai, Li, Zhang, Li, Zhang, Bing & Deng, *EverMemOS: A Self-Organizing Memory Operating System for Structured Long-Horizon Reasoning*, arXiv:2601.02163, 2026. Implementation: [github.com/EverMind-AI](https://github.com/EverMind-AI) (~5k stars, Apache 2.0). Publishes EverMemBench dataset and an EvoAgentBench-framed evolution loop. nox-mem’s EverMemBench numbers are in §5.1.5–§5.1.10; re-running the comparators in the same harness is F4 in §7.2. It is also the system paper for the system measured in §6.3.3. Its reported 93.05% LoCoMo / 83.00% LongMemEval are LLM-judged answer accuracy, a different quantity from the retrieval nDCG@10 of §6.3.3, and are not compared to it.

<sup>84</sup>Hu et al., *Evaluating Long-Horizon Memory for Multi-Party Collaborative Dialogues* (the EverMemBench paper), 2026. arXiv:2602.01313v3. Named in §1.5 as the multi-party long-horizon setting. §5.1.5–§5.1.10 report nox-mem numbers **on** this benchmark and §6.3.3 reports a number **of** the system whose authors also wrote it; both facts are disclosed rather than left implicit, and the disclosure applies symmetrically to our own use of it.

| System                                | Overall<br>nDCG@10<br>(n=2,482) | LoCoMo<br>(n=1,982) | LongMemEval<br>(n=500) | R@10   | MRR    | p50 latency                                  |
|---------------------------------------|---------------------------------|---------------------|------------------------|--------|--------|----------------------------------------------|
| EverOS<br>1.3.1<br>(2026-09-10)       | 0.6455                          | 0.6585              | 0.5942                 | 0.7629 | 0.6403 | 1,592 ms                                     |
| nox-mem<br>(rc4,<br>2026-06-29)       | 0.5013                          | 0.4952              | 0.5255                 | 0.6656 | 0.4749 | 515.6 ms,<br>not<br>transport-<br>normalized |
| <b>Zep<br/>0.27.2</b><br>(2026-09-10) | <b>0.4546</b>                   | <b>0.4793</b>       | <b>0.3567</b>          | 0.6108 | 0.4279 | <b>6,002 ms</b>                              |
| Mem0<br>(rc4,<br>2026-06-29)          | 0.4331                          | 0.4407              | 0.4030                 | 0.5842 | 0.4092 | 349.8 ms,<br>not<br>transport-<br>normalized |

Zep ranks third in this table on overall nDCG@10, behind EverOS and nox-mem and ahead of Mem0, and its LongMemEval number (0.3567) is the lowest any system in this table records on either dataset: below its own LoCoMo by 0.123, the widest per-dataset spread here. Its p50 of 6,002 ms is  $3.8\times$  EverOS’s 1,592 ms in the same harness, and far above nox-mem’s standalone hybrid p50 (529 and ~940 ms in two archived runs, §5.7; a different harness); the p99 is 11,846 ms.

**Why the latency is a property of the surface measured rather than of our host.** We measured Zep’s per-session memory surface: under the one-conversation-per-session mapping a query fans out to 510 sessions and the client merges the results. Zep 0.27.2 also exposes a corpus-wide document-collection search (`/api/v1/collection/{name}/search`), which we did not populate or measure; the latency above is a property of the surface we measured, not a ceiling of Zep. We ran that fan-out in parallel, at the highest worker count the server pool sustained, so the number above comes from the fastest configuration we could give it. A single-session Zep deployment would not pay this cost; this corpus is not a single session.

**What Zep does and does not do in the pipeline (per §6.3.3’s confound).** Zep embeds the query through OpenAI (`text-embedding-3-small`, 1,536 d) and does not rerank. The adapter confirms there is no cross-encoder stage: it calls `search_memory` with a plain `MemorySearchPayload(text=...)` and no reranking option. Separately, our `zep-config.yaml` disables the Summarizer, Entity and Intent extractors (they require LLM completions), a departure from Zep’s enrichment pipeline declared here. So the three-way shape is: EverOS reranks with a 4 B cross-encoder<sup>85</sup> and cannot run without one (§6.3.3),

<sup>85</sup>Zhang, Li, Long, Zhang, Lin *et al.*, *Qwen3 Embedding: Advancing Text Embedding and Reranking Through Foundation Models*, 2025. arXiv:2506.05176. Named in §7.2 F5 as the current open-

nox-mem fuses by RRF with a KG neighbourhood term and no cross-encoder, and Zep does neither. We do not convert that ordering into an explanation of the scores; the magnitude of the pipeline effect is unmeasured here, as declared in §6.3.3.

**Declared, not corrected: 47 session-level failures.** Across the run’s 1,265,820 session sweeps (2,482 queries  $\times$  510 sessions, 0.0037%), 47 failed in 46 distinct sessions: 35 client-side timeouts, 8 OpenAI 500s on `/v1/embeddings` after six retries, and 4 **Bad file descriptor** from our connection pool. None is a Zep retrieval failure, and none crossed the per-query abort threshold; a crossing would have become a query error, and `n_errors` closed at 0. Each removed one of 510 sessions from that query’s candidate pool.

Artifact: `eval/q4-comparison/output-2026-09-10/_aggregate.json` (`system: zep, n_queries 2,482, n_errors 0`). **Caveat.** This artifact is distinct from `eval/q4-comparison/output/zep.json`, which is the superseded 2026-05-25 smoke (20 queries, no nDCG) cited elsewhere in this paper; the two are different runs and the older path is kept unmoved because it is cited.

## 6.4 Per-category breakdown

Per-query-category breakdown from the rc4 embedding-matched run (§6.3.2; all-Gemini 3072d,  $n=2,482$  across both datasets). The canonical 2026-06-15 run reported only dataset-level metrics, so the breakdown is populated from rc4, where per-category gold labels (the datasets’ *native* question types, mapped to six canonical buckets) are wired into the harness. Only nox-mem and Mem0 participate: agentmemory’s server-side embedder cannot be re-pointed to Gemini, Letta produced no number, and the 2026-09-10 EverOS and Zep runs were not broken down by category.

| Category    | n   | nox-mem nDCG@10 | Mem0 nDCG@10 | $\Delta$ |
|-------------|-----|-----------------|--------------|----------|
| single-hop  | 997 | <b>0.5922</b>   | 0.5607       | +0.031   |
| multi-hop   | 415 | <b>0.3641</b>   | 0.3218       | +0.042   |
| temporal    | 454 | <b>0.5502</b>   | 0.4565       | +0.094   |
| adversarial | 524 | <b>0.4370</b>   | 0.2930       | +0.144   |
| open-domain | 92  | <b>0.2592</b>   | 0.2351       | +0.024   |
| numeric     | n/a | n/a             | n/a          | —        |

nox-mem leads every represented category. The largest margins are on adversarial (+0.144) and temporal (+0.094). These are query types where one would expect naive semantic similarity (Mem0’s Gemini-only path) to be weakest and a lexical channel to help, although no per-channel ablation here measures the FTS5 channel’s contribution. The narrowest margin is

---

weight reranker candidate **for nox-mem’s own stack, where it is not run**. It is not, however, absent from this study: EverOS reranks internally with a Qwen3 reranker, and §6.3.1 records the **rerank\_n = 50** ceiling that this imposes on the candidate set measured there. The model appears here as a component of a *compared system*, never of ours.

on open-domain (+0.024, n=92). The lead survives the task-type ablation (§6.3.2): with a generic Gemini embedding, nox-mem still leads all five categories, the last by a near-tie (adversarial 0.4573 vs 0.2930; temporal 0.5571 vs 0.4565; single-hop 0.5773 vs 0.5607; multi-hop 0.3515 vs 0.3218; open-domain 0.2365 vs 0.2351). **Correction (v1.0.3).** v1.0.2 printed this table with LoCoMo’s categories 1–4 permuted; the table and the ablation figures above use the mapping the questions support (1→multi-hop, 2→temporal, 3→open-domain, 4→single-hop) and are recomputed from the rc4 outputs by `paper/measurement/recompute-rc4-categories.py` (output `paper/measurement/out/recompute-rc4-categories.json`). The run-time labeler and the `_aggregate.md` beside the outputs still use the old map and give a different table.

**Bucket composition (declared per §6.6).** The six buckets aggregate the two datasets’ native labels, and two of the mappings affect interpretation. The adversarial bucket (n=524) combines LoCoMo’s 446 native adversarial queries with 78 LongMemEval `knowledge-update` queries. The labeling audit flags this mapping as *ambiguous* in the labeling audit (`eval/q4-comparison/docs/rc2-per-category-mapping.md`) because `knowledge-update` is not adversarial in LoCoMo’s sense, so the adversarial cell (nox-mem’s largest margin, +0.144) should be read as “adversarial + knowledge-update”, not pure adversarial. The `numeric` category receives n/a: neither dataset contributes at least 10 numeric-typed queries (open-domain, by contrast, is LoCoMo-only: LongMemEval has no open-domain type), so per the n<10 rule those cells are not scored rather than extrapolated. LoCoMo buckets are assigned in `paper/measurement/recompute-rc4-categories.py` from each question’s native category; LongMemEval types follow `eval/q4-comparison/lib/category_labeler.py` unchanged.

## 6.5 Fair-comparison principles

The comparison follows principles standardized in the published benchmarking literature (EverMemBench; BEIR, Thakur et al. 2021, arXiv:2104.08663; MTEB, Muennighoff et al. 2023, arXiv:2210.07316):

1. **Identical corpus.** All systems receive the same `chunks.text` ingested via each system’s native API. No system receives an “optimized” version of the corpus.
2. **Identical eval set.** Same queries, same gold sets, same random seed (42 for LongMemEval shuffle).
3. **Native defaults per system.** Each competitor runs with its publicly documented default configuration. Competitors are not adversarially tuned to lose; if the default config is what is published, it is what is evaluated.
4. **K cutoff fixed at 10.** Some systems default to 5 or 20; all are forced to k=10 for comparability.
5. **Native embeddings provider per system.** nox-mem uses Gemini

3072d; each competitor uses its default provider. The **all-Gemini** controlled variant that equalizes this provider was originally planned as an optional side experiment; it was executed (rc4) and is reported in §6.3.2: with both systems on Gemini 3072d over the full  $n=2,482$  set, nox-mem outperforms Mem0 on both datasets and all five represented categories (whereas as-configured, §6.3, Mem0 wins LoCoMo), with four residual confounds declared; with the task-type asymmetry removed in an ablation, the lead persisted (a generic-embedding re-run still wins by at least  $+0.05$  nDCG@10 on both datasets).

6. **Hardware declared per run.** Not uniform across runs: the systems compared within a run share one host (the canonical 2026-06-15 run: a dedicated benchmark pod; rc4: one host for both arms; the 2026-09-10 EverOS and Zep runs: a later host, Zep’s with a working Docker daemon), with localhost between systems except for external embedding and reranking API calls. Retrieval-quality metrics do not depend on hardware, and latency is not compared across runs (§6.3.3).

Each adapter passes a pre-run smoke test:

```
result = adapter.search("test query", k=5)
assert len(result) >= 1
assert all('id' in r and 'score' in r for r in result)
```

Any adapter that fails the smoke test is documented as a gap ([FAILED: <reason>]) rather than omitted, consistent with §6.6.

## 6.6 Anti-cherry-pick statement

To prevent retroactive selection bias:

- **All 6 categories reported.** None is omitted because the result is unfavorable.
- **Both datasets reported.** LongMemEval and LoCoMo side by side:  $n=100$  each in the canonical run (§6.3), and in full ( $n=500 + 1,982 = 2,482$ ) in rc4 (§6.3.2). We do not cherry-pick whichever benefits nox-mem.
- **Latency: not met for the cross-system comparison.** Per-system latency percentiles were not captured uniformly in the 2026-06-15 run (§6.3), so this section reports no cross-system latency at all, rather than a favourable subset of it. nox-mem’s own paths are reported separately in §5.7 with p50 and p95 (KG path 2.5 / 6.1 ms; hybrid 529 / 698 ms, and  $\sim 940$  / 2,342 ms in an earlier run); p99 is not reported for every path, and the opt-in cross-encoder row carries p50 only.
- **Per-category transparency.** The §6.4 table reports every category for the two systems run under the matched embedder (nox-mem, Mem0); the other systems have no per-category breakdown.
- **Gaps documented.** Systems that fail setup receive an explicit note; the comparison runs without the missing system and records the gap.

- **Capped smoke, per dataset (2026-05-23, rev3; superseded).** At a 500-chunk cap and  $n = 20$ , nox-mem scored nDCG@10 0.0466 FTS5-only and 0.0918 hybrid (LoCoMo-only 0.1835), against mem0@500 at 0.1315 (LoCoMo-only 0.2631). The cap exhausts on LoCoMo chunks and zeroes the 10 LongMemEval queries for both systems; at  $n = 20$  the aggregate difference is inside the inconclusive interval, and no result rests on this run.

## 6.7 Pre-specification

This section’s methodology was fixed in `specs/2026-05-23-Q4-comparison-execution-plan.md`, first committed to the public repository on 2026-05-21, before the first run on 2026-05-23; the comparison is pre-specified, not registered with an external registry. The plan is an internal execution document: it named version ranges rather than pins (§6.2), listed the all-Gemini variant as an optional side experiment, and used its success criterion (below) as an internal planning gate. The 2026-05-24 preliminary smoke (20 dry-run-sample queries, eval-isolated DB) validated the pipeline end-to-end (nox-mem nDCG@10 = 0.4509, gold-hit 13/20; `eval/q4-comparison/output/nox_mem.json`). The partial cross-system smoke added a mem0 row ( $n=20$ , 500-chunk corpus cap): nDCG@10 = 0.1315, p50 = 263 ms, gold-hit 3/20 (`output/mem0.json`). Both are superseded (§6.3) and no result rests on them. The canonical run was executed on 2026-06-15 on a dedicated pod ( $n=100$ /dataset,  $k=10$ , same-namespace fair), away from the CPU steal on the shared host that had aborted the IterB composition test (§5.5.8, §7.1 L5). It produced real numbers for 3/6 systems (nox-mem, Mem0, agentmemory) and three documented gaps (§6.3.1), updating the §6.3 cells. The **all-Gemini** variant (§6.5, principle 5), a planned side experiment, was subsequently executed as rc4 (§6.3.2), equalizing the embedding provider over the full  $n=2,482$  set, and supplied the §6.4 per-category breakdown; it was not a post-hoc methodology change, and its residual confounds are declared in §6.3.2 per §6.6, with the task-type asymmetry probed by a dedicated ablation, under which the lead persisted. No methodology from §6.5–§6.6 was altered after the plan was committed; the same-namespace re-query is a fair-comparison refinement (confound removal, §6.3) consistent with §6.5, not a change to corpus, eval set, or gold. Principles (§6.5), anti-cherry-pick (§6.6), and the general structure of this section are immutable post-run. Any methodological adjustment identified during execution is documented as an explicit follow-up rather than retroactively applied here.

The pre-specified success criterion requires nox-mem to rank among the top three on at least two of the four key metrics (nDCG@10, R@10, MRR, latency). In the canonical run only nox-mem, Mem0 and agentmemory produced numbers, so the criterion holds by construction on the three quality metrics and carries no evidential weight; latency was not captured cross-system (§6.6). We therefore do not report the criterion as passed, and rest the comparison on the per-dataset results of §6.3–§6.3.4.

## 6.8 Operational dependencies per memory system

The quality comparison (§6.3–§6.6) reports retrieval quality on matched corpora. Table 2 lists two operational properties of each system’s default self-host configuration that can be read from its source or documentation: the services it runs and the third-party keys it cannot run without. The EverOS row describes its 2026-06-15 configuration, not the library measured in §6.3.3.

**Table 2 — Services and mandatory third-party keys in each system’s default self-host configuration.**

| System                                                           | Services                                                | Mandatory third-party keys                                                              | Sources                                       |
|------------------------------------------------------------------|---------------------------------------------------------|-----------------------------------------------------------------------------------------|-----------------------------------------------|
| nox-mem                                                          | 1 (SQLite file + Node process)                          | 0 (offline-OK; embeddings optional)                                                     | This work; <sup>86</sup>                      |
| mem0                                                             | 2 (Postgres + Qdrant)                                   | 1 (OpenAI for embeddings)                                                               | mem0<br>docker-compose defaults <sup>87</sup> |
| Letta                                                            | 3 (Letta server + Postgres + OpenAI)                    | 1 (OpenAI)                                                                              | Letta self-host guide <sup>88</sup>           |
| Zep OSS                                                          | 2 (Zep + Postgres; 3 with the local embedder)           | 1 (a paid LLM key: OpenAI <i>or</i> Anthropic; the server aborts at startup without it) | Zep v0.27.2 source <sup>89</sup>              |
| EverOS /<br>EverMind-AI<br>(docker-compose,<br>as of 2026-06-15) | 5 (MongoDB + Elasticsearch + Milvus + Redis + Postgres) | 2–3 (LLM + embedding + optional reranker)                                               | EverMind-AI<br>docker-compose <sup>90</sup>   |

<sup>86</sup>The ~399 MB RSS figure of §5.7.3 and §6.8 is **measured 2026-05-29** on the production VPS at 69,135 chunks (field `rss_idle_mb` of the artifact cited in §5.7.1), which attributes the rise from the earlier reading to corpus growth. The earlier reading, **2026-05-24** (single production process, uptime 9h28min, ~62k chunks live per that artifact’s `note_341mb_paper` field, 100% vector coverage), was ~341 MB: main process RSS = 349,276 KB via `ps -eo pid,rss,vsz,comm,args | grep dist/api-server.js`. The cgroup `MemoryCurrent` reported by `systemctl show nox-mem-api -p MemoryCurrent` is 727,064,576 bytes (~727 MB); the ~386 MB delta vs process RSS is SQLite memory-mapped I/O (chunks table, FTS5 index, vec0 index) — kernel-managed page cache, reclaimable on memory pressure, not exclusive process memory.

<sup>87</sup>mem0 default self-host requires Postgres + Qdrant + an OpenAI key for embeddings (or a configured alternative provider). Counts: 2 services + 1 mandatory third-party key. Source: mem0 README and `docker-compose.yml` defaults at [github.com/mem0ai/mem0](https://github.com/mem0ai/mem0).

<sup>88</sup>Letta default self-host requires the Letta server, Postgres, and an OpenAI key (or alternative LLM provider) for the agent loop. Counts: 3 components + 1 mandatory third-party key. Source: Letta self-host documentation at [docs.letta.com](https://docs.letta.com) and [github.com/letta-ai/letta](https://github.com/letta-ai/letta).

<sup>89</sup>Zep OSS requires the Zep service container + Postgres, plus a third container (the local embedder) if embeddings are not routed to a paid vendor. **One paid LLM key is mandatory in every configuration:** `pkg/llms/llm_base.go` constructs an LLM client unconditionally — including for the empty service string, which falls through to OpenAI — and both back-ends `log.Fatal` on an empty key (`llm_openai.go`: `ZEP_OPENAI_API_KEY is not set`; `llm_anthropic.go`: `ZEP_ANTHROPIC_API_KEY is not set`). The embedder, by contrast, is not vendor-locked: `pkg/llms/embeddings.go` routes `Service`: `local` to `pkg/llms/embeddings_local.go`, which POSTs to the local NLP service, and `ZepAnthropicLLM.EmbedTexts` returns “not implemented. use a local embedding model”. Counts: 2 services + 1 mandatory paid LLM key. Source: `get-zep/zep` at tag v0.27.2 — the image we ran — read 2026-09-10.

<sup>90</sup>EverMind-AI / EverOS docker-compose declares MongoDB + Elasticsearch + Milvus + Redis

| System   | Services                                                                   | Mandatory third-party keys         | Sources                                |
|----------|----------------------------------------------------------------------------|------------------------------------|----------------------------------------|
| LightRAG | 1 (in-process library; JSON, NanoVectorDB and NetworkX storage by default) | 1 (LLM provider for KG extraction) | LightRAG source defaults <sup>91</sup> |

Only the nox-mem row is measured; the others are read from each project’s source or documentation, and neither column says anything about retrieval quality. nox-mem runs as one Node process over one SQLite file, needs no third-party key when embeddings are off (FTS5-only retrieval, §4), installs with two commands (`npm i`, `nox-mem reindex`) and idles at a 399 MB resident set (§5.7.3). Zep needs a paid LLM key in every configuration, while its embedder can run keyless against a local service at the cost of a third container (<sup>92</sup>); Letta documents OpenAI as its default provider. The author’s estimates of the other systems’ idle RAM, cold start and setup steps were not measured and are in the supplement (§S6.8).

## 6.9 What running the benchmark required

Running the benchmark also recorded what each system needed in order to produce a number. These are observations from one environment, not measurements of retrieval quality.

On the 2026-06-15 pod, nox-mem ingested the full offered corpus into a single SQLite file in one process with no incidents (the offered corpus is 6,830 documents carrying 6,822 distinct ids, all 6,822 retained by its eval loader, as counted on the 2026-06-29 rc4 artifacts, §6.3.2). The other systems needed the following:

- **Mem0** exhausted the pod’s process-ID limit three times. Its default telemetry (PostHog) leaks one thread per operation, and at benchmark

+ Postgres = 5 services, plus 2-3 third-party API keys for LLM, embedding, and (optional) reranker. Counts confirmed against the published `docker-compose.yml` in the EverMind-AI repo as of **2026-06-15**. That file no longer resolves at the repository root (checked 2026-09-10); the project now ships as a local-first library (v1.3.1, §6.3.3).

<sup>91</sup>LightRAG’s default storage backends are in-process: `JsonKVStorage`, `NanoVectorDBStorage`, `NetworkXStorage` and `JsonDocStatusStorage` (`lightrag/lightrag.py`, main branch of `github.com/HKUDS/LightRAG`, read 2026-10-04). Neo4j and external vector databases are optional backends. One LLM provider key is needed for entity/relation extraction during indexing. Counts: 1 service (the host process) + 1 mandatory third-party key.

<sup>92</sup>Zep OSS requires the Zep service container + Postgres, plus a third container (the local embedder) if embeddings are not routed to a paid vendor. **One paid LLM key is mandatory in every configuration:** `pkg/llms/llm_base.go` constructs an LLM client unconditionally — including for the empty service string, which falls through to OpenAI — and both back-ends `log.Fatal` on an empty key (`llm_openai.go`: `ZEP_OPENAI_API_KEY is not set`; `llm_anthropic.go`: `ZEP_ANTHROPIC_API_KEY is not set`). The embedder, by contrast, is not vendor-locked: `pkg/llms/embeddings.go` routes `Service: local` to `pkg/llms/embeddings_local.go`, which POSTs to the local NLP service, and `ZepAnthropicLLM.EmbedTexts` returns “not implemented. use a local embedding model”. Counts: 2 services + 1 mandatory paid LLM key. Source: `get-zep/zep` at tag `v0.27.2` — the image we ran — read 2026-09-10.

scale this wedged the host until ingest and search were split into separate processes, telemetry was disabled (`MEMO_TELEMETRY=False`), and the vector backend was swapped from Chroma to faiss. Its LoCoMo store still lost ~5% of vectors to the leak before the workaround stabilized.

- **Zep** did not run *on this pod*: Docker is impossible on an unprivileged RunPod kernel (§6.3.1). That is a property of the environment, not of Zep: it ran on hosts with a working Docker daemon, both in the superseded 2026-05-25 smoke (artifact in `eval/q4-comparison/output/zep.json`) and again on 2026-09-10. Operationally, Zep is constrained by the key (§6.8), not by Docker.
- **Letta** ran but at ~16 min/query, so a 100-query benchmark would be a multi-day run.
- **agentmemory** silently persisted store state across sessions, contaminating a LoCoMo run with leftover LongMemEval vectors until the store was reset.

**Scope of this subsection (2026-09-10).** The non-runs above are non-runs *of the 2026-06-15 canonical pod*, and the list is shrinking as upstream barriers lift. EverOS left it first (§6.3.3), taking the count of non-running competitors from three to two. Zep left it on the same day (§6.3.4), brought up on a host with a working Docker daemon and a paid key and swept over the full  $n=2,482$  set, which takes the count to one: Letta. The operational finding that survives either departure is the one verified in source rather than inferred from our environment: Zep cannot boot without a paid LLM key (§6.8).

---

## 7. Limitations and Future Work

### 7.1 Limitations

#### L1 — Explicit-ingestion dependency (no zero-shot corpus coverage)

nox-mem retrieves only what has been explicitly ingested via `ingestFile()`, `ingest-entity`, or the inotifywait watcher pipeline. There is no mechanism to answer queries over arbitrary external corpora at query time. This is a deliberate design constraint. The system is optimized for an agent’s *own* accumulated memory rather than for general-purpose retrieval augmentation, so coverage is bounded by ingestion discipline. A corpus that has never been ingested produces zero recall regardless of query quality. Users bootstrapping the system must explicitly run `nox-mem reindex` over existing files before the hybrid search layer is useful. See §3.1 (ingestion pipeline).

#### L2 — Gemini API dependency for embeddings (cost + outbound network)

The semantic retrieval layer (Layer 2) depends on Google’s `gemini-embedding-001` model (3072 dimensions). This introduces two constraints: (a)

every vectorization call requires outbound network access and a valid `GEMINI_API_KEY`, meaning an air-gapped deployment falls back to FTS5-only retrieval with no semantic recall; (b) API cost scales with corpus size: a full re-vectorization pass re-embeds every chunk (69,135 on 2026-05-29, §5.7.1), so its cost and duration grow linearly with the corpus; no timed re-vectorization run is archived. Provider independence is a stated design goal; local embedding substitution (e.g., `nomic-embed-text` via Ollama) is architecturally feasible but not validated against the canonical eval set. Every deployment that enables embeddings supplies its own Gemini API key; within Google’s free quota there is no per-query charge, and beyond it the cost is the per-token price given in §5.7.2.

**L3 — Single-instance architecture (no distributed sharding or replication)** The system runs on a single SQLite file per agent database. WAL mode provides concurrent read safety, but there is no horizontal sharding, no replication across nodes, and no distributed coordination layer. The current production corpus (79,220 chunks summed across 7 databases, 67,724 of them in the main store, on a single 2-vCPU / ~8 GB virtual server, as of 2026-09-09; the §5.7 latency and footprint figures were measured on the 4-vCPU / 16 GB host in use on 2026-05-29) operates comfortably within these bounds, but the architecture does not generalize to multi-tenant deployments or corpora significantly exceeding the single-node memory/storage envelope. Distributed SQLite extensions (e.g., `cr-sqlite` CRDT-based replication) exist but are explicitly out of scope for v1. The single-instance design is a deliberate architectural decision.

**L4 — No write-side concurrency control (last-writer-wins)** Chunk ingestion operates under an optimistic concurrency model: `ingestFile()` deletes existing chunks for the source file and re-inserts in a single transaction, but there is no row-level locking or version fence against concurrent ingest of the same source file from two processes. In practice the inotify-wait watcher and manual CLI calls rarely overlap, and the WAL journal prevents data corruption; however, two concurrent ingest calls on the same file produce non-deterministic chunk counts. The `withOpAudit()` wrapper (`staged/1.7a/edits/op-audit.ts`) does not add a mutual-exclusion layer for ingest. It targets destructive bulk operations (in the staged sources, `reindex`, `compact`, `kg-merge` and `graphify-ingest` among them); nightly consolidation is not wrapped (§3.4.5). Production mitigations are operational (systemd service prevents concurrent watcher processes; cron stagger of 5 minutes between agents), not architectural.

**L5 — Evaluation sample size and canonical run gap (resolved 2026-06-15 / 2026-06-29)** The 2026-05-24 cross-system run was a 20-query methodology smoke over an eval-isolated DB (5,882 LoCoMo + 940 LongMemEval chunks), not the canonical run; the canonical run was moved off the shared production host, where sustained CPU steal (51–97% over 48 h; batch 005 at

0/50 after 23 h) had just aborted another long benchmark, the IterB composition test of §5.5.8. The canonical run was subsequently executed on dedicated infrastructure on 2026-06-15 (n=100/dataset, 3/6 systems producing numbers + 3 documented gaps, §6.3), and the controlled-embedding variant (rc4) was run on 2026-06-29 at full scale (n=2,482, both systems on Gemini 3072d, §6.3.2 / §6.4). The §6 competitive figures are therefore settled, not [deferred]. The residual limitation is the set of confounds declared on rc4 in §6.3.2, not sample size, and with the one asymmetry that favors nox-mem by design (embedding task type) removed in an ablation, the lead persisted. The earlier nox-mem smoke figure (nDCG@10 = 0.4509 as rescored from its artifact; 0.6380 as first reported) is not directly comparable to the G5 V3 entity-eval figure (0.6237) because the eval corpus and query set differ.

**L6 — Cross-system comparison is methodologically partial** One of five competitors produced no benchmark number. After the canonical run (2026-06-15, n=100/dataset) and the rc4 embedding-matched run (2026-06-29, full n=2,482), Mem0 and agentmemory produced real numbers alongside nox-mem; the 500-chunk cap was a property of the superseded 2026-05-24 smoke only and no longer applies. The standing limitation is deployability coverage: Letta (agent-OS, ~16 min/query) produced no number in the unprivileged single-node environment (§6.3.1). That is a deployability gap on the operational axis and makes no retrieval-quality claim against Letta. Two systems that were gaps of the canonical run have since closed: EverMind-AI / EverOS and Zep, both swept over the full n=2,482 set on 2026-09-10 once the barriers of 2026-06-15 (third-party credentials; a host with a real Docker daemon) were lifted; the results are in §6.3.3 (EverOS) and §6.3.4 (Zep). The rc4 head-to-head additionally carries the four residual confounds declared in §6.3.2 (a fifth potential confound, embedding task-type asymmetry, was probed by ablation, and the lead persisted). See §6.6 (anti-cherry-pick statement).

**L7 — Latency comparison conflates transport classes** Cross-system latency was not captured uniformly. The canonical (2026-06-15) and rc4 runs did not record per-system latency percentiles under a normalized transport, and the only side-by-side figures, from the superseded 2026-05-24 smoke, compared nox-mem localhost retrieval against a Docker-in-Docker HTTP call. Those are different transport classes and do not support a head-to-head speed claim. nox-mem’s operational latency is therefore reported standalone in §5.7 (KG path p50 2.5 ms; hybrid p50 529 and ~940 ms in two archived runs), not as a cross-system comparison. A normalized-transport latency benchmark (all systems behind one gateway) is future work.

**L8 — Pain signal is directional but not statistically significant in isolation** The “pain-weighted hybrid memory” framing rests primarily on the additive salience formula (§5.1.2) and section-aware ranking (§5.1.3). The pain dimension contributes  $W\_PAIN = 0.10$  of the salience weight, but its isolated

causal contribution has not been validated to statistical significance: the E10 pain ablation ([paper/publication/paper-draft-sec4-7.md](#) §5.5) reports  $\Delta = +0.0065$  with 95% CI  $[-0.0143, +0.0338]$  on  $n = 31$ , directional but not significant. Most production chunks carry the default `pain = 0.2` (89% in the snapshot used for the calibration test), but low variance is not shown to be the bottleneck: a follow-up that replaced the real distribution with wider artificial spreads (uniform, bimodal, log-scale;  $n = 60$ , FTS5  $\times$  pain only) did not improve on it, and only 8.3% of those queries had a gold chunk in the BM25 candidate pool ([paper/publication/results/E10-pain-calibration-test.md](#)). A definitive ablation needs queries whose gold chunks the first stage reaches, not only a wider pain range. See §5.1.2.

---

## 7.2 Future Work

**F1 — Signed audit chain: an offline verifier (not yet built)** Encrypted storage (SQLCIPHER) and an Ed25519-signed checkpoint chain over destructive operations (`reindex`, `consolidate`, `crystallize`) have been implemented; the chain makes the audit log tamper-evident without a central trust authority. A standalone, offline verifier for that chain (`nox-mem audit verify`) is planned but not built, and no date is committed.

**F2 — Shadow tracker: empirical A/B for ranking changes** Two observability dashboards ([/observability/health.html](#), [/observability/evals.html](#)) are deployed (supplement §S5.7.4). A planned extension would capture production queries, execute them against a candidate ranking config in parallel, and accumulate query-level nDCG deltas before any promotion decision, then turn this into a pre-promotion gate: any ranking change (boost weight adjustment, mutex threshold, salience weight) that has not accumulated  $\geq 50$  shadow queries with  $p < 0.05$  improvement is blocked from reaching the production endpoint. This closes the observability gap. Currently the shadow phase is a flag toggle governed by procedure, not an integrated eval pipeline or an automated gate.

**F3 — Per-method benchmark Phase B: cross-method nDCG optimization** The cross-system benchmark (§6) establishes the baseline. Phase B targets per-query-type boost calibration: given that, in internal ablations not reported in this paper, keyword and natural-language queries responded differently to boosting ([audits/2026-05-21-G10c-per-style-mutex-ablation.md](#)) and single-hop and multi-hop queries showed opposing trade-offs under the source-type mutex ([audits/2026-05-21-G10b-per-category-mutex-ablation.md](#)), a routing layer that selects ranking parameters based on query-type classification has measurable potential upside.

**F4 — EverMemBench: comparators re-run in the same harness** §5.1.5–§5.1.10 measure nox-mem on EverMemBench’s five released batches, but compare it against the MemOS numbers published in the benchmark paper’s Table 4 rather than against a re-run (§5.8.5). Running MemOS, and EverMind-AI’s own systems (for which the EverMemOS paper<sup>93</sup> reports 93.05% on LoCoMo and 83.00% on LongMemEval, and the HyperMem paper<sup>94</sup> reports 92.73% on LoCoMo; all three are LLM-judged answer accuracy, not retrieval nDCG), through the same harness, batches, answer model and judge would turn those published-number comparisons into measured ones.

**F5 — Neural reranker: cross-encoder rerank post-RRF** The default retrieval stack terminates at RRF fusion (§4.1). A cross-encoder rerank stage already exists as an opt-in path and was measured in §5.1.7: with a 22 M-parameter MiniLM cross-encoder it cost −0.96 pp Overall on EverMemBench (−2.80 to −4.00 pp on Memory Awareness, +3.7 s p50) and was rejected as default. The open item is a larger reranker (the open-weight Qwen3 reranker<sup>95</sup> is the current candidate) evaluated on the same 5-batch protocol. Cross-encoder re-ranking was established by Nogueira & Cho<sup>96</sup>, who report a 27% relative MRR@10 gain over the previous state of the art on MS MARCO passage ranking, with a different metric, first stage and corpus from ours, so no gain is assumed here. The provider-independence goal favors a locally-runnable cross-encoder (e.g., `cross-encoder/ms-marco-MiniLM-L-6-v2` via sentence-transformers, ~91 MB) over a cloud inference call, keeping the retrieval stack fully offline-capable.

<sup>93</sup>Hu, Gao, Zhou, Xu, Bai, Li, Zhang, Li, Zhang, Bing & Deng, *EverMemOS: A Self-Organizing Memory Operating System for Structured Long-Horizon Reasoning*, arXiv:2601.02163, 2026. Implementation: [github.com/EverMind-AI](https://github.com/EverMind-AI) (~5k stars, Apache 2.0). Publishes EverMemBench dataset and an EvoAgentBench-framed evolution loop. nox-mem’s EverMemBench numbers are in §5.1.5–§5.1.10; re-running the comparators in the same harness is F4 in §7.2. It is also the system paper for the system measured in §6.3.3. Its reported 93.05% LoCoMo / 83.00% LongMemEval are LLM-judged answer accuracy, a different quantity from the retrieval nDCG@10 of §6.3.3, and are not compared to it.

<sup>94</sup>Yue, Hu, Sheng, Zhou, Zhang, Liu, Guo & Deng, *HyperMem: Hypergraph Memory for Long-Term Conversations*, 2026. arXiv:2604.08256. Cited in §7.2 F4 for its self-reported LoCoMo accuracy, which F4 proposes to re-measure in a shared harness.

<sup>95</sup>Zhang, Li, Long, Zhang, Lin *et al.*, *Qwen3 Embedding: Advancing Text Embedding and Reranking Through Foundation Models*, 2025. arXiv:2506.05176. Named in §7.2 F5 as the current open-weight reranker candidate **for nox-mem’s own stack, where it is not run**. It is not, however, absent from this study: EverOS reranks internally with a Qwen3 reranker, and §6.3.1 records the `rerank_n = 50` ceiling that this imposes on the candidate set measured there. The model appears here as a component of a *compared system*, never of ours.

<sup>96</sup>Nogueira & Cho, *Passage Re-ranking with BERT*, 2019. arXiv:1901.04085. The work that established the cross-encoder as a re-ranking stage over a first-stage candidate pool: the MS MARCO result cited in §7.2 F5, and the stage whose **presence or absence** is the confound of §6.3.3–§6.3.4 — EverOS requires one, nox-mem fuses by RRF[`~rrf`] without one, Zep has neither. Our own measurement of the stage (§5.1.7, −0.96 pp) used a 22 M distillation[`~minilm`], ~180× smaller than the 4 B model EverOS runs[`~qwen3embed`], so it is not carried over as a bound.

**F6 — Scale validation: 250k chunk corpus** The main store held 67,724 chunks with complete vector coverage as of 2026-09-09, which is below the ~100k-vector scale at which we expect, without having measured it, exact-search latency to start to matter. An internal record placed the main store near 95k as of 2026-06-04; that figure is not re-verifiable, and the store has shrunk since through documented cleanup (removal of duplicates and of retired static imports). A 250k chunk corpus would validate: (a) sqlite-vec ANN recall at scale (current exact-search; approximate search may become necessary past ~100k vectors (unmeasured)); (b) salience formula stability (the recency component decays over a longer history window); (c) FTS5 BM25 IDF calibration (with more documents, rare-term IDF weights shift). This scale-validation item has no committed spec yet; it is gated on `NOX_SALIENCE_MODE=active` remaining stable in production.

**F7 — Multilingual corpus coverage: Portuguese and Spanish** The current evaluation corpus is English-dominant (LongMemEval and LoCoMo are English datasets; the internal entity-eval golden set mixes English and Portuguese). The FTS5 tokenizer (`unicode61 remove_diacritics 2` since the schema-V5 migration, which removed the English Porter stemmer because it mis-stemmed Portuguese) folds accents but applies no Portuguese or Spanish stemming, so inflected forms do not match lexically (e.g., “decisão” and “decisões” are indexed as the distinct terms “decisao” and “decisoos”). The Gemini semantic layer partially compensates via cross-lingual embedding space, but there is no explicit multilingual evaluation. A Portuguese golden set is a natural next step given the production operational language of the corpus; Quati<sup>97</sup>, a Brazilian-Portuguese IR dataset annotated by native speakers, is a strong candidate benchmark for that evaluation. This evaluation is deferred.

**F8 — Feedback from use outside the author’s deployment** Use of the system by others is expected to surface real-world limitation patterns not visible in the synthetic golden sets (e.g., corpora with high image-to-text OCR content, multi-language mixes, or very short memory fragments < 20 words that the current chunker merges).

---

## 8. Conclusion

nox-mem demonstrates that persistent, searchable, and shareable memory for AI agent fleets is achievable with commodity infrastructure: a single VPS, one SQLite file per store, and a provider-agnostic embedding layer, which used Gemini in every configuration measured here; FTS5-only retrieval is a valid keyless degraded mode (§4). In production, hybrid retrieval recovered queries that FTS5 alone missed (§4.2), and in the capped  $n = 20$  smoke of §6.6 it scored

---

<sup>97</sup>Bueno *et al.*, *Quati: A Brazilian Portuguese Information Retrieval Dataset from Native Speakers*, STIL 2024. arXiv:2404.06976. Cited in §7.2 F7 as the candidate Portuguese evaluation set.

nDCG@10 0.0918 against 0.0466 for FTS5-only; no FTS5-only or dense-only ablation was run on the canonical LoCoMo/LongMemEval sets. Multilingual behaviour is not among the claims: the evaluation corpus is English-dominant and Portuguese/Spanish coverage is declared future work (§7.2 F7). The LLM-powered knowledge graph extracts typed entities and relations that the earlier regex path did not, and its relations carry confidence decay and a TTL (§2.2), the mechanism meant to keep the graph current without manual curation; this paper does not measure how well it does so. No head-to-head extraction-yield ratio against the regex path is reported here. The Wave A empirical evaluation (§5) established nDCG@10 = 0.6237 on the entity-flavored golden set (+78.8% relative over the G3 baseline), with `section_boost` identified as the dominant driver (A3 alone reaches 99.86% of the full-stack score) and the additive salience formula supported by the `active > shadow` reversal (0.6237 vs 0.6155, +1.3%, a single n = 100 run without a confidence interval; §5.1.2). The conditional source-type mutex (supplement §S5.1.4; deployed 2026-05-21) consolidates the production boost stack (§4.1: section and source-type deltas summed, the source-type delta skipped for section-bearing chunks when the query names at most two entities, plus the additive salience delta), recovering multi-hop and adversarial regressions with contained dilution on single-hop.

The cross-system comparison of §6 is pre-specified (execution plan committed to the public repository before the first run; not registered with an external registry; `specs/2026-05-23-Q4-comparison-execution-plan.md`). After a methodology smoke (2026-05-24, n=20) and a move off the shared production host (§7.1 L5), the canonical run executed 2026-06-15 (n=100/dataset, 6 systems: 3 produced numbers, 3 documented gaps; §6.3), the all-Gemini variant (rc4), a planned side experiment, executed 2026-06-29 (full n=2,482; §6.3.2 / §6.4), and two of those gaps (EverOS, Zep) were measured on 2026-09-10 over the same corpus and n=2,482 set: EverOS outperforms nox-mem on both datasets (overall nDCG@10 0.6455 vs 0.5013; the contribution of its mandatory cross-encoder is unmeasured, §6.3.3) and Zep ranks third, behind EverOS and nox-mem and ahead of Mem0 (§6.3.4). Under each system’s native embedder nox-mem and Mem0 split: Mem0 wins LoCoMo and nox-mem wins LongMemEval (§6.3); with the embedding provider equalized, nox-mem outperforms Mem0 on both datasets and all five represented categories (§6.3.2). Both readings are reported side by side per §6.6, with four residual confounds declared; with the task-type asymmetry removed in an ablation, the lead persisted. The pre-specified success criterion of §6.7 (top-3 on at least 2 of the 4 key metrics) is not reported as passed (§6.7): in the canonical run only nox-mem, Mem0 and agentmemory produced numbers, so top-3 was guaranteed and carries no evidential weight; over the full n = 2,482 set nox-mem ranks second of four on nDCG@10, R@10 and MRR (§6.3.4), and latency was not compared under a normalized transport (§7.1 L7).

A cross-agent layer searches all agent databases in one query and attributes each result to its agent (§4.3).

**Repository:** [github.com/totobusnello/memoria-nox](https://github.com/totobusnello/memoria-nox)

---

## Disclosure of Generative AI Use

Generative AI tools were used in preparing this work in two roles, reported here in line with arXiv’s guidance that significant use of text-to-text generative AI be reported.

*Engineering and writing assistance.* Anthropic’s Claude models (Opus and Sonnet versions), used through the Claude Code command-line agent, assisted with writing and running code, evaluation scripts, and the mechanical checks that test this manuscript’s claims against the experiment artifacts, and with drafting and editing the manuscript text. Most commits that changed the manuscript source record a Claude model as co-author.

*Review.* Drafts and revisions were reviewed read-only by LLM-based reviewers from other model families (GLM, Grok, Kimi and Codex) to check statements against the stored artifacts. Findings were checked against the artifacts before being accepted or rejected; `paper/CHANGELOG.md` records these rounds in its v1.0.0, v1.0.2 and v1.0.3 entries.

Models that are components of the system or of the evaluation (the embedding model, the knowledge-graph extraction model, and the answer-generation backbones) belong to the method and are specified where they are used.

The research questions, the design decisions and the reported numbers are the author’s, and the author takes responsibility for the full text.

---

## Appendices — moved out

The manuscript carries no appendix. The seven operational appendices of earlier revisions (A. Knowledge Graph v2, B. Cross-Agent Intelligence, C. MCP Server Interface, D. HTTP API Server, E. Operational Infrastructure, F. Dashboard Integration, G. Evolution History) are included in the supplementary material rather than in this PDF. They document how the deployment is wired rather than what was measured, so no claim made here depends on them.

## References and Footnotes

### Related-systems references

#### Table 2 sources

#### Academic references

Every entry below carries a resolvable identifier (arXiv ID or DOI). Full BibTeX in `paper/refs.bib`. arXiv IDs were verified against the arXiv API on 2026-09-09 — ID, first author and title checked to match, rather than transcribed from memory.

#### Internal references